

Building Large Language Models from Scratch

A grounded eBook generated from the playlist

Pipeline Statistics	
Videos processed	43
Claims extracted	3,873
Superseded claims	684
Grounded sentences	76.9%

Source playlist:

https://www.youtube.com/watch?v=Xpr8D6LeAtw&list=PLPTV0NXA_ZSgsLAr8YCgCwhPIJNNtexWu

Generated by llm-playlist-book pipeline

May 2026

Contents

1	Large Language Models: Foundations and Landscape	1
1.1	What Is a Large Language Model?	1
1.2	Where Do LLMs Fit? The AI Terminology Hierarchy	2
1.3	How LLMs Differ from Earlier NLP Models	2
1.4	The Two-Stage Development Pipeline	2
1.4.1	Stage 1: Pre-Training	3
1.4.2	Stage 2: Fine-Tuning	3
1.5	The Three-Stage Development Workflow	4
1.6	The Transformer Architecture: A Brief Introduction	4
1.7	Zero-Shot and Few-Shot Learning	5
1.8	The Historical Arc: From Transformers to GPT-4	7
1.9	Open-Source vs. Closed-Source Models	7
1.10	Why Build from Scratch?	8
1.11	Chapter Summary	8
2	The Transformer Architecture: From Attention Is All You Need to GPT	9
3	The Original Transformer: Built for Machine Translation	11
3.0.1	The Encoder	11
3.0.2	The Decoder	11
3.0.3	The Eight-Step Pipeline	12
3.1	BERT and GPT: Two Paths from One Architecture	12
3.1.1	BERT: The Encoder-Only Model	12
3.1.2	GPT: The Decoder-Only Model	12
3.2	Beyond Language: Vision Transformers	13
3.3	Inside the Transformer Block	13
3.3.1	Multi-Head Attention	13
3.4	The GPT Model Configuration	15
3.5	The Full GPT Pipeline: From Token IDs to Logits	15
3.5.1	Step 1: Token Embeddings	16
3.5.2	Step 2: Positional Embeddings	16
3.5.3	Step 3: Dropout	16
3.5.4	Step 4: Transformer Blocks	16
3.5.5	Step 5: Output Head — Projecting to Logits	16
3.6	Putting It All Together	17
3.7	Chapter Summary	18

4	Tokenization: From Words to Byte Pair Encoding	19
5	The Role of Tokenization in the LLM Pipeline	20
5.1	Three Families of Tokenization	20
5.1.1	Word-Based Tokenization	20
5.1.2	Character-Based Tokenization	21
5.1.3	Subword-Based Tokenization	21
5.2	Building a Word-Based Tokenizer from Scratch	21
5.2.1	Loading the Raw Text	21
5.2.2	Splitting into Tokens	22
5.2.3	Building the Vocabulary	23
5.2.4	Encoding and Decoding	23
5.3	Handling Unknown Words with Special Tokens	24
5.3.1	Adding Special Tokens to the Vocabulary	24
5.3.2	Updating the Encoder to Use the Unknown Token	24
5.3.3	Other Special Tokens	24
5.4	The Limits of Word-Based Tokenization	25
5.5	Byte Pair Encoding: The Algorithm	25
5.5.1	Step 1: Initialize with Characters	25
5.5.2	Step 2: Count and Merge Pairs	25
5.5.3	Step 3: Apply to New Text	25
5.5.4	A Worked Example	26
5.6	Using tiktoken for GPT-2 Compatible BPE	26
5.6.1	Installation and Basic Usage	26
5.6.2	How the BPE Tokenizer Handles Unknown Words	27
5.7	Putting It All Together: The Full Tokenization Pipeline	27
5.8	Summary	27
6	Data Preprocessing: Input-Target Pairs, Token Embeddings, and Positional Em- beddings	29
6.1	Creating Input-Target Pairs	29
6.1.1	Why LLMs Need a Special Pairing Strategy	29
6.1.2	Context Size and the Sliding Window	30
6.1.3	Loading the Raw Text	31
6.2	The Dataset and DataLoader Abstraction	31
6.2.1	Why Use a DataLoader?	31
6.2.2	Dataset Structure	31
6.2.3	Workers and Parallel Loading	31
6.2.4	Verifying the Output	31
6.3	Token Embeddings: From IDs to Vectors	32
6.3.1	What Is a Token Embedding?	32
6.3.2	The Embedding Weight Matrix	32
6.3.3	Word2Vec: A Historical Perspective	32
6.3.4	<code>torch.nn.Embedding</code> : The PyTorch Lookup Table	32
6.3.5	Building the Token Embedding Layer for GPT	34
6.4	The Problem with Token Embeddings Alone	34
6.5	Positional Embeddings	35
6.5.1	What Is Positional Encoding?	35

6.5.2	Absolute vs. Relative Positional Embeddings	35
6.5.3	The Sinusoidal Formula (Original Transformer)	35
6.5.4	Implementing Learned Positional Embeddings	36
6.6	Combining Token and Positional Embeddings	36
6.6.1	The Addition Operation	36
6.6.2	The Complete Pipeline	37
6.7	Putting It All Together: A Worked Example	37
6.8	Summary	39
7	The Attention Mechanism: From RNNs to Self-Attention	40
7.1	The Problem: Sequential Data and Long-Range Dependencies	40
7.1.1	Recurrent Neural Networks	40
7.1.2	The Vanishing Gradient and Loss of Context	40
7.2	Bahdanau Attention: A First Solution	41
7.3	From Attention to Self-Attention	42
7.4	Simplified Self-Attention: The Core Idea	42
7.4.1	Notation	42
7.5	Step 1: Computing Attention Scores	42
7.6	Step 2: Normalizing to Attention Weights	43
7.6.1	Softmax Normalization	44
7.7	Step 3: Computing Context Vectors	44
7.8	Putting It Together: A Complete Walkthrough	45
7.9	Implementing Simplified Self-Attention in Python	45
7.10	Introducing Trainable Weights	46
7.10.1	The Three Weight Matrices	46
7.10.2	Projecting Inputs Through Weight Matrices	46
7.10.3	Initializing Weight Matrices in PyTorch	47
7.11	Why Scale by $\sqrt{d_k}$?	47
7.12	The Road Ahead	47
8	Self-Attention with Trainable Weights: Keys, Queries, and Values	49
8.1	Why Trainable Weights?	49
8.2	The Three Weight Matrices	50
8.3	Computing Attention Scores	51
8.4	Scaling by $\sqrt{d_k}$: Why It Matters	51
8.5	From Scores to Attention Weights	51
8.6	Computing Context Vectors	52
8.7	Putting It All Together: The Forward Pass	53
8.8	Implementing SelfAttentionV1 in PyTorch	54
8.8.1	Initializing the Weight Matrices	54
8.8.2	The Forward Method	54
8.9	Implementing SelfAttentionV2: Using <code>nn.Linear</code>	54
8.10	Interpreting the Mechanism	54
8.11	The Full Picture	56
8.12	What Comes Next	57
9	Causal Self-Attention and Multi-Head Attention	58
9.1	The Problem with Unrestricted Self-Attention	58

9.2	Building the Causal Mask	58
9.2.1	A Simple Lower-Triangular Mask	59
9.2.2	Why You Must Mask Before Softmax	59
9.3	Adding Dropout to Attention Weights	60
9.4	The CausalSelfAttention Class	60
9.5	From Single-Head to Multi-Head Attention	61
9.5.1	Naive Wrapper Implementation	61
9.6	The Efficient Weight-Split Implementation	61
9.6.1	Initializing the Layers	62
9.6.2	The Forward Method: Step by Step	62
9.7	A Working Example	63
9.8	Comparing the Two Implementations	64
9.9	Summary	65
10	Building the Transformer Block: LayerNorm, GELU, Feed-Forward, and Short-	66
	cut Connections	
10.1	Layer Normalization	66
10.1.1	The Problem: Internal Covariate Shift	66
10.1.2	The Math	66
10.1.3	A Quick Empirical Check	67
10.1.4	Scale and Shift: Trainable Parameters	67
10.1.5	A Note on Bessel's Correction	67
10.1.6	The LayerNorm Class	68
10.2	GELU: The Activation Function of Choice	68
10.2.1	Why Not ReLU?	68
10.2.2	GELU: A Smooth Alternative	68
10.2.3	The GELU Class	69
10.3	The Feed-Forward Network	70
10.3.1	Purpose and Architecture	70
10.3.2	Why Expand and Then Contract?	70
10.3.3	Implementation	70
10.4	Shortcut Connections and the Vanishing Gradient Problem	70
10.4.1	The Vanishing Gradient Problem	70
10.4.2	What Are Shortcut Connections?	71
10.4.3	The Math Behind Shortcut Connections	71
10.4.4	Implementing Shortcut Connections	71
10.5	Assembling the Transformer Block	72
10.5.1	Components	72
10.5.2	Pre-Layer Norm vs. Post-Layer Norm	72
10.5.3	The Stacking Order	72
10.5.4	Putting It All Together	73
10.6	The GPT Architecture in Context	73
10.7	Summary	73
11	Assembling and Running the Full GPT-2 Model	74
11.1	The Big Picture: What the GPT Pipeline Looks Like	74
11.2	The GPT Configuration Object	74
11.3	Building the DummyGPTModel	75

11.4	Component Deep-Dive	75
11.4.1	Token Embeddings	75
11.4.2	Positional Embeddings	75
11.4.3	Combining the Two	75
11.4.4	Dropout	76
11.4.5	The Transformer Blocks	76
11.4.6	Layer Normalization	76
11.4.7	The Output Head	76
11.5	The Full GPT Model Forward Pass	76
11.6	Counting Parameters: 124 M vs. 163 M	77
11.7	Generating Text: The Autoregressive Loop	77
11.7.1	The Five-Step Algorithm	77
11.7.2	The <code>generate_text_simple</code> Function	78
11.7.3	Putting It All Together	78
11.8	Running the Model: A Concrete Example	79
11.9	Why the Output Is Nonsense (For Now)	79
11.10	What Was Built from Scratch	79
11.11	Summary	80
12	Pre-Training: Loss Functions, Training Loop, and Evaluation	81
12.1	The Loss Landscape: Why Cross-Entropy?	81
12.1.1	A Taxonomy of Loss Functions	81
12.1.2	Why Language Modeling Uses Cross-Entropy	82
12.2	From Logits to Loss: The Forward Pass in Detail	83
12.2.1	The Model's Output: Logits	83
12.2.2	Constructing Input-Output Pairs	83
12.2.3	The Dataloader	83
12.3	Computing Cross-Entropy Loss in PyTorch	84
12.3.1	The Flat-Tensor Pattern	84
12.3.2	Selecting the Predicted Token	84
12.3.3	The <code>calculate_loss_batch</code> Function	84
12.4	Perplexity: An Interpretable Loss Metric	84
12.5	The Pre-Training Loop	85
12.5.1	High-Level Structure	85
12.5.2	The Core Update Rule	85
12.5.3	Iterating Over Batches	85
12.5.4	Evaluation During Training	85
12.5.5	Model Parameters: A Quick Accounting	87
12.6	Putting It All Together: Training on "The Verdict"	87
12.6.1	Data Preparation	87
12.6.2	The Training Function	88
12.6.3	Decoding Predictions During Training	88
12.6.4	Observing Overfitting	88
12.7	Summary	90
13	Decoding Strategies, Model Checkpointing, and Loading Pre-Trained GPT-2 Weights	91
13.1	Decoding Strategies	91

13.1.1	Greedy Decoding	91
13.1.2	Multinomial Sampling	91
13.1.3	Temperature Scaling	92
13.1.4	Top-K Sampling	92
13.1.5	Combining Top-K Sampling and Temperature Scaling	93
13.1.6	The <code>generate</code> Function	94
13.2	Model Checkpointing: Saving and Loading Weights	94
13.2.1	PyTorch Model State	94
13.2.2	Saving the Optimizer State	94
13.2.3	Restoring from a Checkpoint	95
13.3	Loading Pre-Trained GPT-2 Weights	95
13.3.1	Downloading the GPT-2 Files	95
13.3.2	What Gets Downloaded	96
13.3.3	Understanding the <code>settings</code> and <code>params</code> Dictionaries	96
13.3.4	GPT-2 Model Variants	97
13.3.5	Loading TensorFlow Weights into PyTorch	97
13.3.6	Shape Validation During Assignment	98
13.3.7	Assigning Feed-Forward Weights	98
13.3.8	Running Inference with Pre-Trained Weights	99
13.4	Why Build Your Own Instead of Using the API?	100
13.5	Summary	100
14	Classification Fine-Tuning: Building a Spam Detector	101
14.1	What Is Classification Fine-Tuning?	101
14.1.1	12.1.1 The Role of the Foundation Model	101
14.2	The SMS Spam Collection Dataset	102
14.2.1	12.2.1 Class Imbalance and Balancing	102
14.2.2	12.2.2 Downloading the Dataset	102
14.2.3	12.2.3 Splitting into Train, Validation, and Test Sets	102
14.3	Building PyTorch Datasets and Data Loaders	103
14.3.1	12.3.1 Tokenization	103
14.3.2	12.3.2 The Spam Dataset Class	103
14.4	Loading Pre-Trained GPT-2 Weights	104
14.4.1	12.4.1 Downloading GPT-2	104
14.4.2	12.4.2 Loading TensorFlow Checkpoints	105
14.5	Modifying the GPT Architecture for Classification	105
14.5.1	12.5.1 The Classification Head	105
14.5.2	12.5.2 Which Token's Output Do We Use?	105
14.6	Freezing Layers and Selective Unfreezing	105
14.6.1	12.6.1 What to Freeze and What to Unfreeze	106
14.6.2	12.6.2 Parameter-Efficient Methods: LoRA and QLoRA	106
14.7	Evaluation Metrics: Accuracy and Loss	107
14.7.1	12.7.1 Classification Accuracy	107
14.7.2	12.7.2 Cross-Entropy Loss	107
14.8	The Fine-Tuning Training Loop	108
14.8.1	12.8.1 Epochs and Batches	108
14.8.2	12.8.2 Evaluation Frequency	108
14.8.3	12.8.3 Per-EPOCH Accuracy Reporting	108

14.8.4	12.8.4 Training Loop Structure	108
14.9	Saving and Loading the Fine-Tuned Model	108
14.10	Evaluating on the Test Set and Running Inference	109
14.10.1	12.10.1 Why a Held-Out Test Set Matters	109
14.10.2	12.10.2 Inference on New Text	109
14.11	Putting It All Together	110
14.11.1	12.11.1 Comparison with Instruction Fine-Tuning	110
14.12	Summary	110
15	Instruction Fine-Tuning: Building a Personal Assistant	111
15.1	Why Instruction Fine-Tuning?	111
15.2	The Three-Stage Pipeline	111
15.3	The Instruction Dataset	112
15.3.1	What the Data Looks Like	112
15.3.2	Loading and Splitting the Data	112
15.4	The Alpaca Prompt Format	112
15.4.1	Implementing <code>format_input</code>	113
15.5	Data Batching: From Text to Tensors	113
15.5.1	Step 1: Format with the Prompt Template	113
15.5.2	Step 2: Tokenization	114
15.5.3	Step 3: Padding to Equal Length	114
15.5.4	Step 4: Creating Target Sequences	114
15.5.5	Step 5: Replacing Padding Tokens with <code>-100</code>	115
15.5.6	The Custom Collate Function	115
15.6	Creating the Data Loaders	116
15.7	Loading the Pre-Trained Model	116
15.8	The Fine-Tuning Training Loop	117
15.8.1	Loss Function	117
15.8.2	Optimizer	117
15.8.3	Parameter Updates	117
15.8.4	Monitoring Training	117
15.8.5	Saving and Loading the Fine-Tuned Model	117
15.9	Evaluating the Fine-Tuned Model	118
15.9.1	Step 1: Extracting Responses	118
15.9.2	Step 2: Qualitative Evaluation	118
15.9.3	Step 3: Automated LLM-Based Scoring	118
15.10	Running Llama 3 Locally with Ollama	118
15.10.1	Starting Ollama	119
15.11	Putting It All Together	119
15.12	Summary and What Comes Next	119
16	Summary, Reflections, and Next Steps	120
16.1	The Three-Stage Pipeline at a Glance	120
16.2	Stage 1 in Depth: Architecture	121
16.2.1	The Attention Mechanism	121
16.2.2	The Transformer Block and Full Architecture	122
16.3	Stage 2 in Depth: Pre-training	122
16.4	Stage 3 in Depth: Fine-tuning	123

16.5	Evaluating What You Have Built	123
16.6	What Was Built from Scratch	123
16.7	Reflections on the Core Ideas	124
16.8	Next Steps and Further Directions	124
16.9	Closing Thoughts	125
A	GPT Model Configuration Reference	126
A.0.1	The GPT_CONFIG_124M Dictionary	126
A.0.2	Hyperparameter Definitions	126
A.0.3	The Full GPT-2 Model Family	128
A.0.4	Training Hyperparameters	129
A.0.5	Quick-Reference Summary	130
B	Environment Setup: PyTorch, TikToken, TensorFlow, and Ollama	131
B.0.1	Device Selection: CPU, CUDA, and Apple Silicon	131
B.0.2	TensorFlow and tqdm: Loading GPT-2 Weights	132
B.0.3	Ollama: Running LLMs Locally	132
B.0.4	Summary	133
C	Datasets Used in This Series	134
C.0.1	The Verdict: Pre-Training Dataset	134
C.0.2	SMS Spam Collection: Classification Fine-Tuning Dataset	135
C.0.3	Instruction Fine-Tuning Dataset	135
C.0.4	GPT-3 Pre-Training Corpora: Real-World Scale	135
C.0.5	Downloading and Licensing	136
C.0.6	Summary	136

Chapter 1

Large Language Models: Foundations and Landscape

If you have spent any time with ChatGPT, Gemini, or similar tools, you have already interacted with a Large Language Model. But knowing how to *use* one and knowing how to *build* one are very different things. This chapter lays the conceptual groundwork for everything that follows: what LLMs actually are, where they sit in the broader landscape of artificial intelligence, how they are trained, and why they represent such a significant departure from the NLP tools that came before them. By the end, you will have a clear mental map of the territory—one that will make every technical detail in subsequent chapters feel purposeful rather than arbitrary.

1.1 What Is a Large Language Model?

Start with the simplest possible definition. A Large Language Model—LLM for short—is a neural network designed to understand, generate, and respond to human-like text. That single sentence contains three important ideas: *neural network*, *massive scale*, and *text*.

At its core, a neural network consists of input data feeding into layers of neurons stacked together, with an output layer. Think of each layer as a transformation: raw numbers go in, slightly more useful numbers come out, and after enough layers the network has learned to map inputs to outputs in surprisingly powerful ways. LLMs are deep neural networks—meaning they have many such layers—trained on massive amounts of data and specifically designed to understand, generate, and respond to human-like text.

The word “large” in the name is not decorative. Model size refers to the number of parameters in a model, and modern LLMs have parameter counts in the billions or even trillions. Those parameters are the learned weights that encode everything the model knows about language, facts, reasoning patterns, and more.

[Diagram: Neural network architecture showing input text flowing through stacked layers with parameters on connections, producing output text.]

1.2 Where Do LLMs Fit? The AI Terminology Hierarchy

The field of AI is littered with overlapping buzzwords—AI, ML, deep learning, generative AI—and it is easy to lose track of how they relate. Here is the precise hierarchy.

Artificial Intelligence is the broadest umbrella term, encompassing any machine that behaves remotely like a human or exhibits some form of intelligence. Inside that umbrella sits Machine Learning, a subset of AI where machines learn and adapt based on how users interact with them. Inside ML sits Deep Learning, which specifically involves neural networks—whereas ML also encompasses other approaches like decision trees.

LLMs are a subset of deep learning that deal exclusively with text and do not involve images. That last qualifier matters: the moment you add images, audio, or video to the mix, you have crossed into a different, though related, territory.

[Diagram: Hierarchical relationship of AI subfields: AI encompasses ML, which encompasses DL, which contains LLMs at the center. Generative AI overlaps the DL and LLM regions.]

That related territory is Generative AI—a mixture of large language models and deep learning that deals with multiple modalities including text, images, sound, and video. Generative AI uses deep neural networks to create new content such as text, images, and various forms of media. So while every LLM is a piece of deep learning, not every piece of generative AI is an LLM.

1.3 How LLMs Differ from Earlier NLP Models

To appreciate what makes LLMs remarkable, it helps to understand what came before them. Earlier NLP models were designed for very specific tasks—a translation model translated, a sentiment analysis model classified sentiment, and that was essentially the end of the story. Each model was a specialist, and specialists are brittle: a translation model cannot answer a question, and a sentiment classifier cannot summarize a document.

LLMs break this mold entirely. A single pretrained LLM can translate, summarize, answer questions, write code, and perform dozens of other tasks—often without any task-specific retraining. This task-agnostic quality is one of the defining characteristics of the LLM era, and understanding *why* it arises requires understanding how LLMs are trained.

1.4 The Two-Stage Development Pipeline

Creating an LLM involves two stages: pre-training and fine-tuning. This pipeline is fundamental enough that the entire book is organized around it, so it is worth spending time here to understand what each stage means and why both are necessary.



Figure 1.1: Two-stage pipeline for building Large Language Models: pre-training produces a foundation model, which is then fine-tuned to create a task-specific model.

1.4.1 Stage 1: Pre-Training

Pre-training means training an LLM on a huge amount of data so that it can perform a wide range of tasks. The key word is *huge*. We are not talking about thousands or even millions of examples—we are talking about hundreds of billions of tokens of text scraped from the internet and other sources.

For the purposes of building intuition, one token is approximately equal to one word. So “hundreds of billions of tokens” translates roughly to hundreds of billions of words—an amount of text no human could read in many lifetimes.

Where does all this text come from? Two prominent sources illustrate the scale:

- **Common Crawl** is a huge and open repository of all the data on the internet.
- **WebText2** is a corpus consisting of Reddit submissions, blog posts, Stack Overflow articles, and code.

The training objective during pre-training is deceptively simple: next-word prediction. Given a sequence of words, the model predicts the next word—also called word completion. LLM pretraining involves this next-word prediction task applied to large text datasets.

What makes this objective so powerful is that it requires no human-labeled data. The text itself provides the supervision: every word in a document is simultaneously a training input (given the words before it) and a training label (the correct next word). This approach is called generative pre-training—an unsupervised learning method where a language model is trained on unlabeled text to predict the next word, using the text itself as training data without requiring external labels. The raw text used for training is regular text without any labeling information, which means training data can be collected at internet scale without expensive human annotation.

In the GPT family of models, this self-supervised structure is made explicit: a sentence is divided into training and testing portions, where the next word serves as the known true label. GPT models are autoregressive, meaning each prediction is conditioned on all previous words in the sequence.

The result of pre-training is a **foundation model** (also called a base model or pre-trained model)—a model trained on an underlying dataset of unlabeled data that has absorbed an enormous amount of world knowledge and linguistic structure, but has not yet been specialized for any particular application.

1.4.2 Stage 2: Fine-Tuning

Once a foundation model exists, it can be adapted for specific tasks through fine-tuning. A pre-trained LLM can be finetuned using smaller labeled datasets for specific tasks like classification, summarization, or question-answering. Fine-tuning uses human-annotated examples to adjust the model’s weights so that it performs well on a given task.

The contrast with pre-training is stark: pre-training uses massive unlabeled datasets; fine-tuning uses small labeled ones. Pre-training builds general capability; fine-tuning sharpens it for a purpose.

It is worth noting what fine-tuning for classification does to a model’s outputs. A classification-finetuned model is restricted to predicting only the classes it has encountered during training; for example, a model trained to classify emails as “spam” or “not spam” cannot output any other response. This is a deliberate constraint—you want a spam classifier to say “spam” or “not spam,” not to compose a poem.

All modern large language models are trained in these two main steps: pre-training on unlabeled data to create a foundational model, followed by fine-tuning on labeled task-specific data.

1.5 The Three-Stage Development Workflow

When building an LLM from scratch, the two-stage pipeline expands into a three-stage development workflow:

1. **Stage 1 — Foundation and Architecture:** This covers data preparation, sampling, and the architectural components of the model. Before training anything, you need to understand how to represent text as numbers, how to feed data into the network, and what the network's internal structure looks like.
2. **Stage 2 — Pre-Training:** With the architecture in place, the model is trained on large unlabeled datasets. At the end of this stage, pre-trained weights—such as those released by OpenAI—can be loaded into the model, and functions to save and load those weights are implemented to avoid retraining from scratch every time.
3. **Stage 3 — Fine-Tuning:** The pre-trained model is adapted for specific applications.

This book follows that three-stage structure. The goal throughout is to understand the nuts and bolts of how large language models are built, rather than simply running pre-built applications.

1.6 The Transformer Architecture: A Brief Introduction

The architectural backbone of virtually every modern LLM is the **transformer**—a deep neural network architecture introduced in 2017. Understanding the transformer is essential to understanding LLMs, and later chapters will build it piece by piece. For now, it is enough to know that the transformer's power comes from a mechanism called **attention**.

The attention mechanism is the core component of the transformer architecture that makes it powerful. Intuitively, attention allows the model to decide which parts of the input are most relevant when producing each part of the output. More precisely, it gives LLMs selective access to the entire input sequence when generating output one word at a time. This is a crucial capability: in a long sentence, the word that best predicts the next word might be ten or twenty words back. The self-attention mechanism captures exactly these long-range dependencies, allowing better prediction of the next word—something earlier architectures struggled with.

Several components work together to make attention function correctly:

- **Attention scores** are a key component of the attention mechanism.
- **Positional encoding** provides information about the order in which words appear in a sentence, since the transformer architecture does not inherently process words in sequence.
- **Vector embeddings** represent words as dense numerical vectors.
- **Tokens** are the basic units of a sentence that the model operates on.

The transformer also uses specialized variants of attention for different purposes. **Multi-head attention** applies the attention mechanism across multiple representation subspaces simultaneously,

allowing the model to attend to different aspects of the input at once. **Masked multi-head attention** is a variant used in transformer decoders to prevent the model from attending to future positions—a necessary constraint during next-word prediction training, since the model should not be able to “cheat” by looking ahead.

The GPT approach specifically combines the transformer architecture with unsupervised pre-training, where labels are derived from the sentences themselves without any pre-labeling of data.

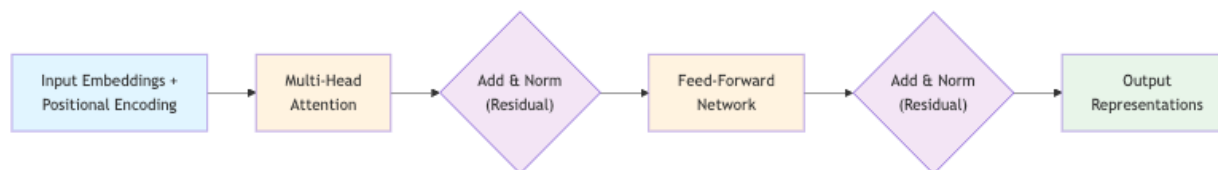


Figure 1.2: Transformer block architecture showing input embeddings with positional encoding flowing through multi-head attention and feed-forward layers with residual connections to produce output representations.

1.7 Zero-Shot and Few-Shot Learning

One of the most striking capabilities of large pre-trained language models is their ability to perform tasks they were never explicitly trained on. This phenomenon has two related names depending on how much task-specific information the model receives at inference time.

Zero-shot learning is the ability to generalize to completely unseen tasks without any prior specific examples. The model predicts the answer given only a description of the task, with no supporting examples whatsoever. For instance, in zero-shot translation, the model is given only the task description “translate English to French” and the input word “breakfast”—no translation examples included.

Few-shot learning is learning from a minimum number of examples provided by the user as input. Rather than updating the model’s weights, these examples are supplied directly in the input prompt.

Between these two extremes sits **one-shot learning**, where the model sees a single example of the task in addition to the task description.

These capabilities are closely related to what researchers call **emergent behavior**—the ability of a model to perform tasks it was not explicitly trained to perform. A model trained purely on next-word prediction somehow learns to translate, reason, and answer questions. This emergence is not fully understood theoretically, but it is empirically well-established.

The concept of few-shot learning as a property of large language models was formally introduced with GPT-3, released in 2020, which demonstrated that language models are few-shot learners. Zero-shot versus few-shot learning remains a key concept in understanding what modern LLMs can do.

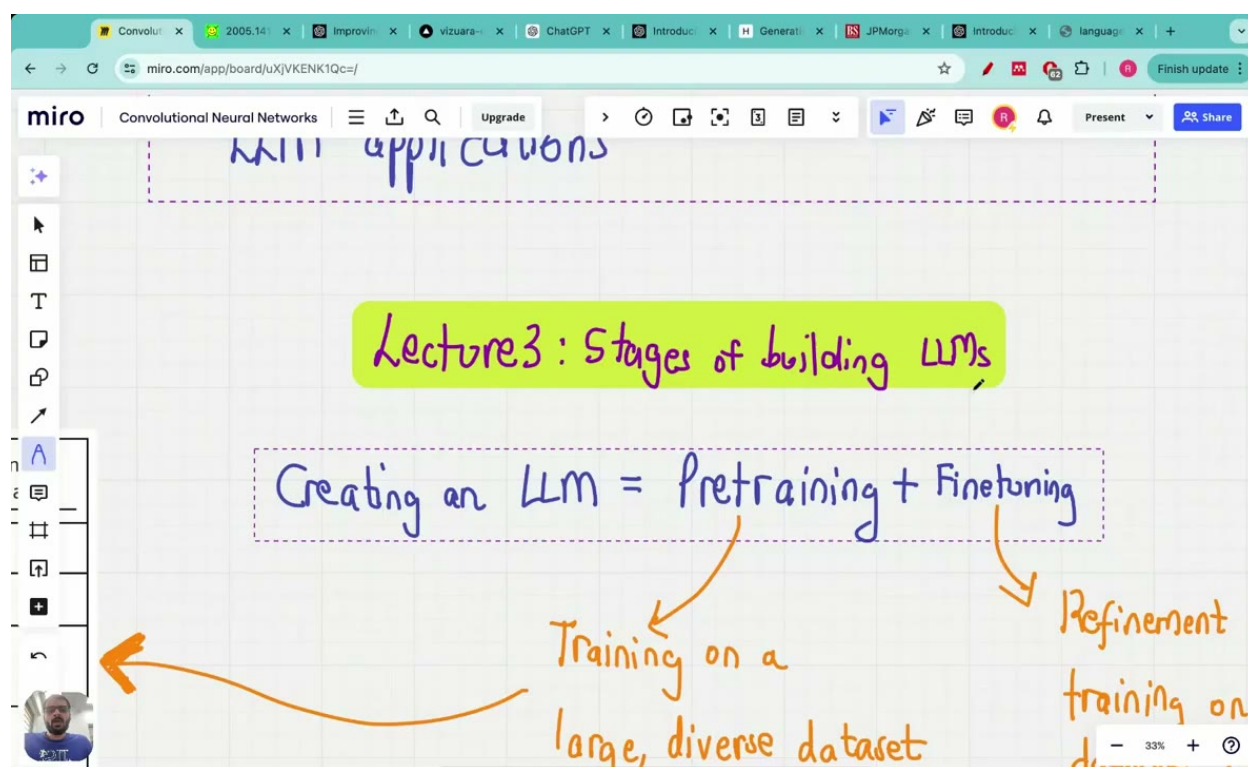


Figure 1.3: Three prompting paradigms: zero-shot (task description only), one-shot (task description plus one example), and few-shot (task description plus multiple examples) for guiding LLM behavior.

1.8 The Historical Arc: From Transformers to GPT-4

To understand where we are, it helps to trace the path that got us here.

The transformer architecture, introduced in 2017, provided the foundation. The GPT family of models then demonstrated that combining transformers with large-scale unsupervised pre-training could produce models with remarkable generalization ability. GPT-3, released in 2020, was a landmark: it showed that scaling up model size and training data produced qualitatively new capabilities, including few-shot learning.

GPT-4 represents the current state of the art in closed-source models—one where the parameters and weights are not publicly known. This opacity is characteristic of the closed-source approach: such models do not release their weights or architecture, and few details about the model itself are made public.

1.9 Open-Source vs. Closed-Source Models

The distinction between open-source and closed-source models is practically important for anyone building with or on top of LLMs.

An open-source model makes its entire architecture available to the public for anyone to see, allowing researchers and engineers to inspect, modify, and build upon it. In 2022, most large language models were closed-source. That landscape has shifted considerably: by 2024, the performance gap between open-source and closed-source models is slowly decreasing, and as of August 2024, open-source language models are comparable to closed-source models on the MMLU benchmark.

Meta’s release of Llama 3 405B is a concrete example of this trend—a model that closes the gap between closed-source and open-weight models. All the information needed to build capable language models is now available as open-source. Modern large language models such as GPT are very powerful and sophisticated, but the techniques used to build them are no longer secret, and the open-source ecosystem has made it possible to study, replicate, and build upon state-of-the-art architectures.

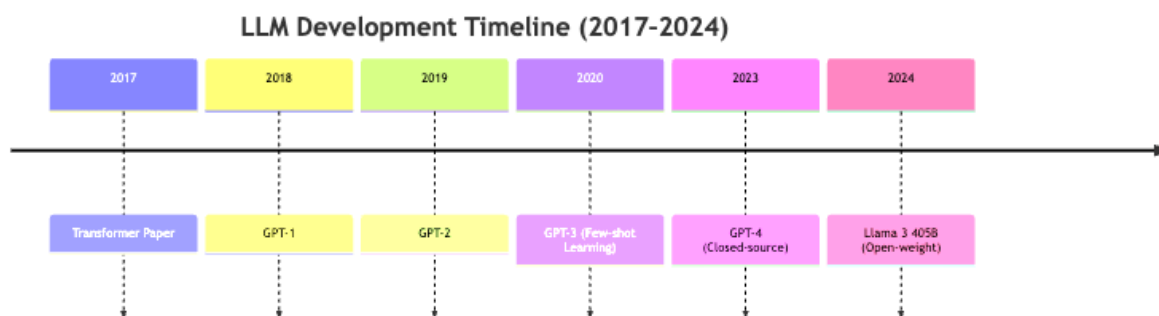


Figure 1.4: Key milestones in large language model development from the Transformer paper (2017) through GPT-4 and open-weight models like Llama 3 (2024).

1.10 Why Build from Scratch?

Given that powerful pre-trained models are freely available, a reasonable question is: why bother building one from scratch? Most existing online courses on building large language models focus on application development rather than teaching how to build an LLM from the ground up. Running a pre-built model is straightforward; understanding what is happening inside it is not.

A common mistake learners make is jumping directly to applications and running code without understanding how LLMs actually work. This shortcut creates a fragile understanding—you can use the tool, but you cannot diagnose problems, adapt architectures, or reason about limitations.

The goal of this book is different: to teach how to build an entire large language model from scratch, ensuring deep understanding of LLM fundamentals rather than focusing on deploying pre-built applications. Every design decision, every hyperparameter, every architectural choice will be motivated from first principles. By the time you finish, you will not just know *that* transformers use multi-head attention—you will know *why*, and you will have implemented it yourself.

1.11 Chapter Summary

This chapter has established the conceptual landscape for everything that follows:

- An LLM is a deep neural network designed to understand, generate, and respond to human-like text, with model size measured in parameters.
- LLMs sit within a hierarchy: AI $\supset$ ML $\supset$ Deep Learning $\supset$ LLMs, with Generative AI overlapping DL and LLMs across multiple modalities.
- Earlier NLP models were task-specific specialists; LLMs are task-agnostic generalists.
- LLM development follows a two-stage pipeline: pre-training on massive unlabeled text using next-word prediction, followed by fine-tuning on smaller labeled datasets for specific tasks.
- The transformer architecture, introduced in 2017, is the backbone of modern LLMs, with the attention mechanism as its core innovation.
- Pre-trained LLMs exhibit zero-shot and few-shot learning—the ability to perform tasks without explicit training examples—a form of emergent behavior.
- GPT-3 (2020) formally established few-shot learning as a property of large language models; GPT-4 represents the closed-source frontier.
- The open-source model landscape has matured rapidly, with performance now comparable to closed-source models on standard benchmarks.

In the next chapter, we begin Stage 1 of the development workflow: preparing text data and understanding how raw text is transformed into the numerical representations that a neural network can process.

Chapter 2

The Transformer Architecture: From Attention Is All You Need to GPT

The story of modern large language models begins with a single paper. In 2017, a research team published “Attention Is All You Need,” introducing the transformer — a deep neural network architecture that would reshape the field of machine learning. In the years since, that paper has accumulated more than one hundred thousand citations, a testament to how thoroughly it changed what was possible. Most modern LLMs rely on the transformer architecture, and the GPT architecture that underlies ChatGPT originated directly from that work.

This chapter builds a complete mental model of the transformer: where it came from, how it is structured, how GPT and BERT diverged from the original design, and how the full pipeline from raw token IDs to output logits fits together. By the end, you will have enough context to understand every design decision in the GPT implementation we build throughout the rest of this book.

To appreciate why the transformer was such a leap forward, it helps to understand what came before it.

The dominant sequence-modeling tools before 2017 were Recurrent Neural Networks (RNNs) and their more capable cousins, Long Short-Term Memory (LSTM) networks. RNNs maintain a feedback loop to incorporate memory, allowing them to carry information from one step of a sequence to the next. LSTM networks extend this idea by maintaining two separate paths — one for short-term memories and one for long-term memories. Despite this improvement, both architectures struggled with what researchers call *long-range dependencies*: the need to understand context from distant parts of a sequence, such as an earlier sentence, to accurately predict the next word in a later one. The feedback-loop design of RNNs means that information from many steps back tends to fade or become distorted by the time it is needed.

Attention mechanisms changed this. They have become an integral part of sequence modeling, allowing the modeling of dependencies without regard to the distance between input or output sequences. The self-attention mechanism specifically allows the model to weigh the importance of different words and tokens relative to each other and to capture long-range dependencies by looking far back in a sequence to identify the next word. Indeed, without masked multi-head attention, large language models would lose their power and revert to relying on recurrent and long short-term

memory networks.

The paper “Attention Is All You Need” is named precisely because of the self-attention mechanism and the intuition behind it. The name is a statement of intent: you do not need recurrence at all.

Chapter 3

The Original Transformer: Built for Machine Translation

The original transformer was proposed for machine translation tasks, specifically for translating English texts into German and French. The transformer architecture consists of two main blocks: an encoder that converts input text to embedding vectors, and a decoder that generates output text from those embedding vectors and partial output.

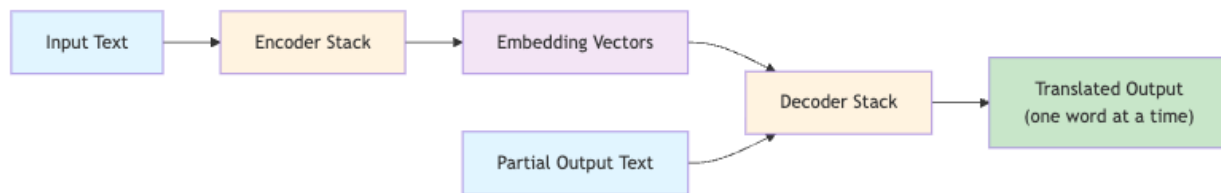


Figure 3.1: High-level encoder-decoder transformer architecture showing input text flowing through the encoder stack to produce embedding vectors, which feed into the decoder stack alongside partial output text to generate translated output sequentially.

3.0.1 The Encoder

The encoder takes tokenized input text and converts it into vector embeddings. Intuitively, its job is to read and understand the source sentence, compressing its meaning into a rich numerical representation that the decoder can consult.

3.0.2 The Decoder

The decoder generates output text one word at a time, receiving both the encoder’s vector embeddings and the partial output text produced so far. Previously generated words are made available to the model at each step, so the process is autoregressive: the output at one step becomes part of the input for the next.

3.0.3 The Eight-Step Pipeline

At a high level, the transformer converts input text to output text through eight steps: tokenization, encoding, decoding with partial output, and word-by-word generation.

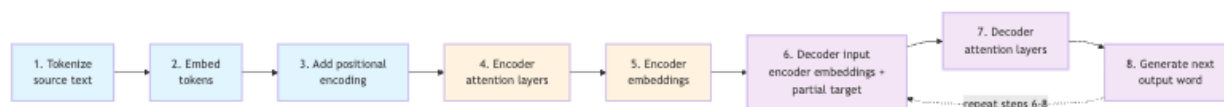


Figure 3.2: Eight-step transformer pipeline: tokenization and embedding of source text, encoder processing with attention layers, and iterative decoder generation with encoder-decoder attention.

After the original transformer was developed, researchers discovered that architectures derived from it could handle many tasks beyond machine translation. Text completion — the predominant role of GPT — was not even in consideration when the transformer paper was originally proposed. The transformer turned out to be a general-purpose sequence-processing engine, and researchers quickly began adapting it.

3.1 BERT and GPT: Two Paths from One Architecture

Both BERT and GPT have the word “transformers” in their names because they originated from the transformer architecture, but they took very different paths from that common ancestor.

3.1.1 BERT: The Encoder-Only Model

BERT stands for Bidirectional Encoder Representations from Transformers. As the name implies, BERT uses only the encoder half of the original transformer and has no decoder. It is a transformer variation that predicts hidden or masked words in a sentence, attending to the sentence from both left and right directions simultaneously. This bidirectional view gives BERT a significant advantage for understanding tasks: it can capture nuances and relationships between words by examining the entire sentence at once.

One practical consequence is that BERT can differentiate between different meanings of the same word — such as “bank” as a financial institution versus “bank” as a river bank — by examining surrounding words. BERT models are therefore commonly used for sentiment analysis, because they can capture the meanings of different words and how they relate to each other. GPT can also perform sentiment analysis, though it is a specialty of BERT.

3.1.2 GPT: The Decoder-Only Model

GPT stands for Generative Pre-trained Transformers. It is a pre-trained foundational model and the architecture that underlies ChatGPT. Where BERT reads in both directions, GPT processes text strictly from left to right, predicting only the next unknown token at each step.

In ChatGPT, when you write an input, GPT gives attention to every preceding token and then predicts what the next word could be — not just the immediately preceding sentence. This is the self-attention mechanism at work: even in a left-to-right model, every token can attend to all previous tokens, not just its immediate neighbor.

[Diagram: Comparison of BERT (bidirectional encoder) and GPT (left-to-right decoder) architectures showing attention flow patterns and output generation.]

The key architectural difference is straightforward: BERT uses only the encoder half of the original transformer; GPT uses only the decoder half, with the cross-attention to an encoder removed since there is no encoder to attend to.

3.2 Beyond Language: Vision Transformers

The transformer is not limited to text. Transformers can be applied to computer vision tasks such as image classification and image segmentation. Vision Transformers (ViT) apply the transformer architecture specifically to tasks such as image recognition and image classification, and they achieve comparable or better results than Convolutional Neural Networks (CNNs) while requiring substantially fewer computational resources for pre-training.

This generality is part of what makes the transformer so important. The same core mechanism — self-attention over a sequence of tokens — works whether those tokens represent words, image patches, or other discrete units.

It is also worth being precise about terminology: “transformers” and “large language models” are not interchangeable. Not all large language models are based on the transformer architecture, and not all transformers are large language models.

3.3 Inside the Transformer Block

We now zoom in from the high-level architecture to the building block that makes it all work. The transformer block is the most important part of the large language model architecture, consisting of multiple components linked together.

The attention mechanism is considered the heart of the transformer, and the transformer block contains a large number of trainable parameters and weights that are optimized during LLM pre-training. The feed-forward network within the block uses GELU activation, and its weights are optimized during training to predict output words correctly. At its simplest, the transformer can be understood as a neural network where parameters are optimized through training; the complexity lies in the specific structure of those parameters and how information flows through them.

3.3.1 Multi-Head Attention

The self-attention mechanism inside the transformer block is implemented as *multi-head attention*. The number of attention heads refers to the count of separate query, key, and value matrix sets created in this process. The governing equations are:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where each individual head is computed as:

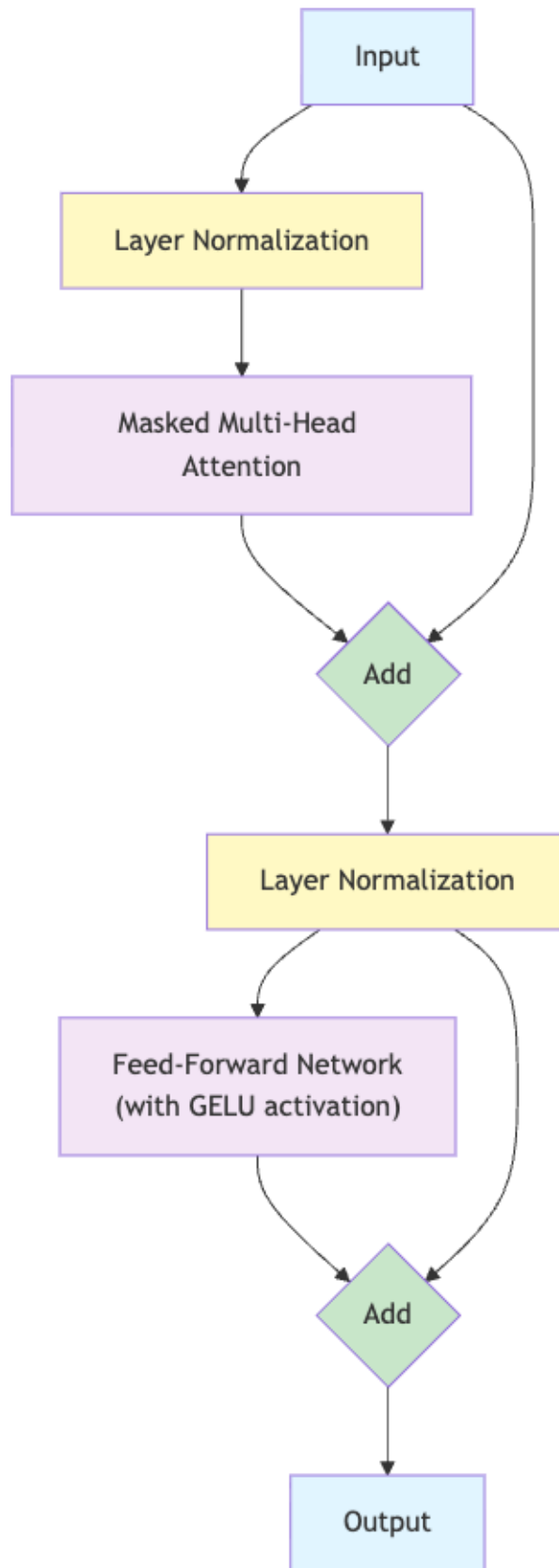


Figure 3.3: Transformer block architecture showing the sequential flow of layer normalization, masked multi-head attention, residual connections, and feed-forward network with GELU activation.

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Running multiple attention heads in parallel allows the model to simultaneously attend to different aspects of the input — one head might track syntactic relationships while another tracks semantic similarity. The results are concatenated and projected back to the model dimension via the output weight matrix W^O .

After all transformer blocks are applied, each input token has a 768-dimensional output vector representation. This dimensionality matches the embedding dimension used throughout the model.

3.4 The GPT Model Configuration

Before tracing the full pipeline, it is useful to examine the concrete hyperparameters that define the GPT-124M model we will implement. Pre-training is training on a large, diverse dataset, and the model configuration determines the network's capacity for that training.

```
GPT_CONFIG_124M = {
    "vocab_size": 50257,           # Vocabulary size
    "context_length": 1024,       # Context length
    "emb_dim": 768,               # Embedding dimension
    "n_heads": 12,                # Number of attention heads
    "n_layers": 12,               # Number of layers
    "drop_rate": 0.1,             # Dropout rate
    "qkv_bias": False             # Query-Key-Value bias
}
```

Each field controls a distinct aspect of the architecture:

- **vocab_size: 50257** — the number of distinct tokens the model knows about; each token ID is an integer in the range $[0, 50256]$.
- **context_length: 1024** — the maximum number of tokens the model can process in a single forward pass.
- **emb_dim: 768** — the dimensionality of every token's vector representation throughout the model. Each token ID in GPT-2 is converted into a vector of 768 dimensions.
- **n_heads: 12** — the number of parallel attention heads in each transformer block.
- **n_layers: 12** — the number of transformer blocks stacked on top of each other.
- **drop_rate: 0.1** — the dropout probability applied during training for regularization.
- **qkv_bias: False** — no bias terms are used in the Query-Key-Value projections.

These parameters were covered in earlier lectures, and we will refer back to them throughout the implementation chapters.

3.5 The Full GPT Pipeline: From Token IDs to Logits

With the configuration in hand, we can trace the complete data flow through the GPT model — the bird's-eye view that ties every component together.

[Diagram: Full GPT pipeline architecture: from token IDs through embedding lookups, transformer blocks, and layer normalization to final logits output.]

3.5.1 Step 1: Token Embeddings

Token IDs are converted to token embeddings before being passed to the GPT model. The architecture workflow begins with token IDs that are converted to input embedding vectors of 768 dimensions. In GPT-2, the embedding layer is trained to learn token representations that capture semantic meaning, with the embedding matrix parameters initialized randomly from a Gaussian distribution and refined during backpropagation.

3.5.2 Step 2: Positional Embeddings

Because the transformer has no built-in notion of order — unlike an RNN, which processes tokens sequentially — position information must be injected explicitly. The positional embedding is implemented as a learned lookup table:

```
self.pos_emb = nn.Embedding(context_length, embedding_dimension)
```

To index into this table, we construct a tensor of position indices:

```
pos = torch.arange(sequence_length)
```

This creates a tensor `[0, 1, 2, ..., sequence_length-1]` that indexes into the positional embedding matrix. The positional embedding for each position is then added to the corresponding token embedding, giving the model a combined representation that encodes both *what* the token is and *where* it appears in the sequence.

3.5.3 Step 3: Dropout

After the token and positional embeddings are summed, dropout is applied for regularization. The output then passes through the transformer blocks.

3.5.4 Step 4: Transformer Blocks

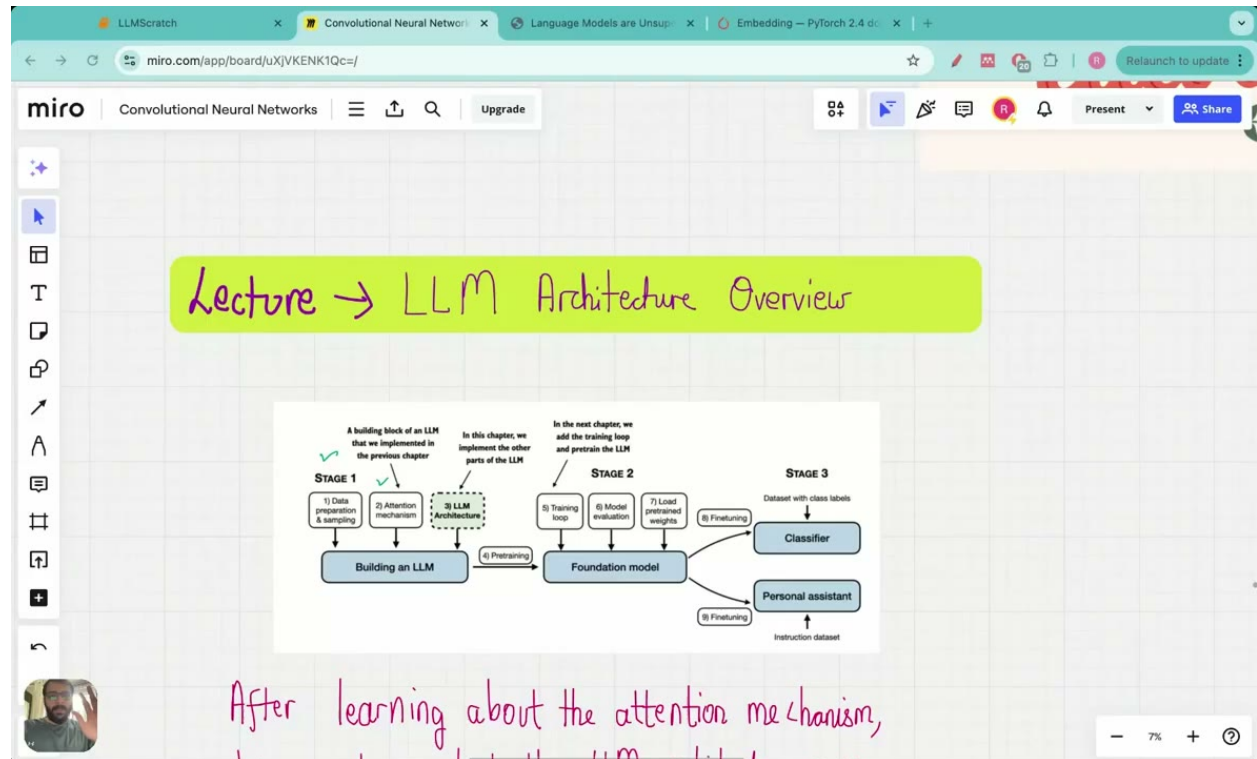
The combined embedding passes through 12 stacked transformer blocks. Each block applies masked multi-head attention followed by a feed-forward network, with layer normalization and residual connections around each sub-layer. The attention mechanism is the heart of each block, and the forward method of the GPT model class takes an input and outputs the next-word prediction by running it through all transformer blocks in sequence.

3.5.5 Step 5: Output Head — Projecting to Logits

After all transformer blocks have processed the sequence, a final linear transformation maps from the embedding dimension back to the vocabulary. The output head applies a linear transformation from the embedding dimension (768) to the vocabulary size (50,257), converting token representations to logits.

Logits are the final output matrices from a transformer model that contain probability scores for each token in the vocabulary at each position in the sequence. The shape of the logits tensor

is therefore `[batch_size, sequence_length, 50257]` — for every position in the input, the model produces a score for every possible next token.



with each row representing a probability distribution over the vocabulary for one sequence position]

To convert logits to actual probabilities, a softmax function is applied, and the token with the highest probability (or a sample from the distribution) becomes the next generated token. Generating text from output tokens using the logits matrix is the final step in the pipeline.

3.6 Putting It All Together

The full architecture can be summarized as a single sequential pass:

1. **Tokenization:** raw text is split into tokens and mapped to integer IDs from a vocabulary of 50,257 tokens.
2. **Embedding:** each token ID is looked up in a learned embedding table, producing a 768-dimensional vector, to which a learned positional embedding is added to encode sequence position.
3. **Dropout:** applied to the summed embeddings during training.
4. **Transformer blocks ($\times 12$):** each block applies masked multi-head attention (12 heads) followed by a GELU-activated feed-forward network, with layer normalization and residual connections throughout. The large number of trainable parameters in these blocks are optimized during pre-training.
5. **Output projection:** a linear layer maps the 768-dimensional representations to 50,257-dimensional logits.

6. **Generation:** logits are converted to probabilities, the next token is selected and appended to the sequence, and the process repeats.

The GPT configuration parameters control every dimension of this pipeline. Changing `n_layers` adds or removes transformer blocks; changing `emb_dim` widens or narrows every vector in the system; changing `n_heads` controls how many parallel attention patterns each block can learn.

3.7 Chapter Summary

This chapter has traced the transformer from its origins as a machine translation system to its role as the foundation of modern LLMs. The key points to carry forward:

- The transformer was introduced in 2017 for machine translation and consists of an encoder and a decoder.
- Researchers subsequently discovered that transformer-derived architectures could handle many tasks beyond translation.
- BERT uses only the encoder and is bidirectional, making it well-suited for understanding tasks such as sentiment analysis.
- GPT uses only the decoder and generates text left-to-right, one token at a time.
- The transformer block — the core repeating unit — combines masked multi-head attention, a GELU-activated feed-forward network, layer normalization, and residual connections.
- The full GPT pipeline runs from token IDs through embeddings, through stacked transformer blocks, to a final linear projection that produces logits over the vocabulary.
- The GPT-124M configuration uses a vocabulary of 50,257 tokens, a context length of 1,024, an embedding dimension of 768, 12 attention heads, and 12 transformer layers.

In the chapters that follow, we will implement each of these components from scratch, starting with the attention mechanism — the heart of the transformer.

Chapter 4

Tokenization: From Words to Byte Pair Encoding

Before a large language model can process text, that text must be converted into a form the model can actually work with: numbers. This chapter traces that conversion from first principles—splitting raw text into tokens—all the way to the production-ready byte pair encoding used by GPT-2.



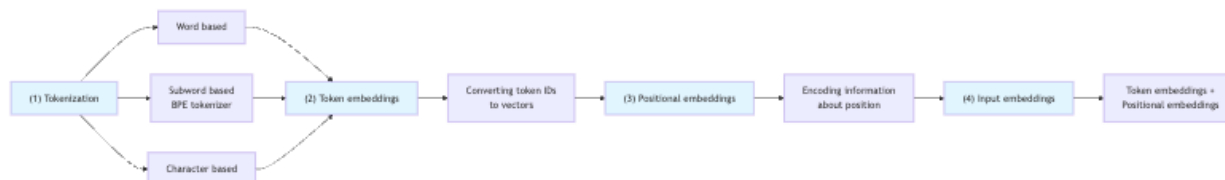
Figure 4.1: LLM data preprocessing pipeline: raw text is tokenized into token IDs, which are then converted to vector embeddings for model input.

Chapter 5

The Role of Tokenization in the LLM Pipeline

Data preprocessing is the procedure of processing text data—sentences, paragraphs, documents—before it is fed as input to a large language model. Within that preprocessing pipeline, tokenization is the very first step: it converts raw text into tokens. Those tokens are then mapped to integer IDs, and those IDs are later converted into vector representations that the model actually trains on.

Concretely, the pipeline involves three stages: tokenizing input text into words, converting those words into token IDs, and converting token IDs into vector representations. This chapter focuses on the first two stages; the third—embedding—is covered in the next chapter.



tokens, (2) tokens $\rightarrow$ [4312, 995] IDs, (3) IDs $\rightarrow$ embedding vectors]

5.1 Three Families of Tokenization

There are three main types of tokenization algorithms: word-based tokenizers, sub-word-based tokenizers, and character-based tokenizers. Each makes a different trade-off between vocabulary size, handling of rare words, and semantic expressiveness.

5.1.1 Word-Based Tokenization

Word-based tokenization is the most intuitive approach: it converts an entire dataset into chunks of words, treating each word as an individual token. In earlier tokenization schemes, every word was treated as a unique token, along with special characters such as commas, full stops, and exclamation marks.

To see this concretely, consider splitting on whitespace:

```
# Source:
import re
text = "Hello_world.This_is_a_test."
tokens = re.split(r'\s+', text)
```

This yields one token per whitespace-separated chunk. Because word-based tokenization treats each unique word as a single token, a large corpus will produce an enormous vocabulary. Worse, any word not seen during training becomes an **out-of-vocabulary (OOV) word**—a word absent from the tokenizer’s vocabulary—and must be handled with special context tokens.

5.1.2 Character-Based Tokenization

At the opposite extreme, character-based tokenization uses individual characters as tokens instead of words. For example, “dinosaur” is split into eight separate character tokens (d, i, n, o, s, a, u, r), whereas word-based tokenization would treat it as a single token.

The upside is a tiny vocabulary: character-based tokenization requires only approximately 256 characters, far fewer than word-based tokenization. It also eliminates the OOV problem entirely, since every possible string can be represented as a sequence of characters. The trade-off is that the model must learn to compose meaning from individual characters at every level, producing very long sequences with little per-token semantics.

5.1.3 Subword-Based Tokenization

Subword-based tokenization sits between the two extremes, using tokens that are neither full words nor individual characters. It combines features of both approaches by selectively splitting words based on frequency, which retains semantic similarity by sharing root-word tokens across related words like “tokens” and “tokenizing”. Byte pair encoding (BPE) is the most widely used algorithm for implementing this approach and is the algorithm used by GPT models.

5.2 Building a Word-Based Tokenizer from Scratch

Although word-based tokenization is not what GPT uses, building one from scratch is the best way to understand the machinery that all tokenizers share: a vocabulary, an encode method, and a decode method. We will use a real text file as our corpus.

5.2.1 Loading the Raw Text

```
# Source:
with open('the-verdict.txt', 'r') as f:
    raw_text = f.read()
```

The variable `raw_text` stores the entire text content read from The Verdict file.

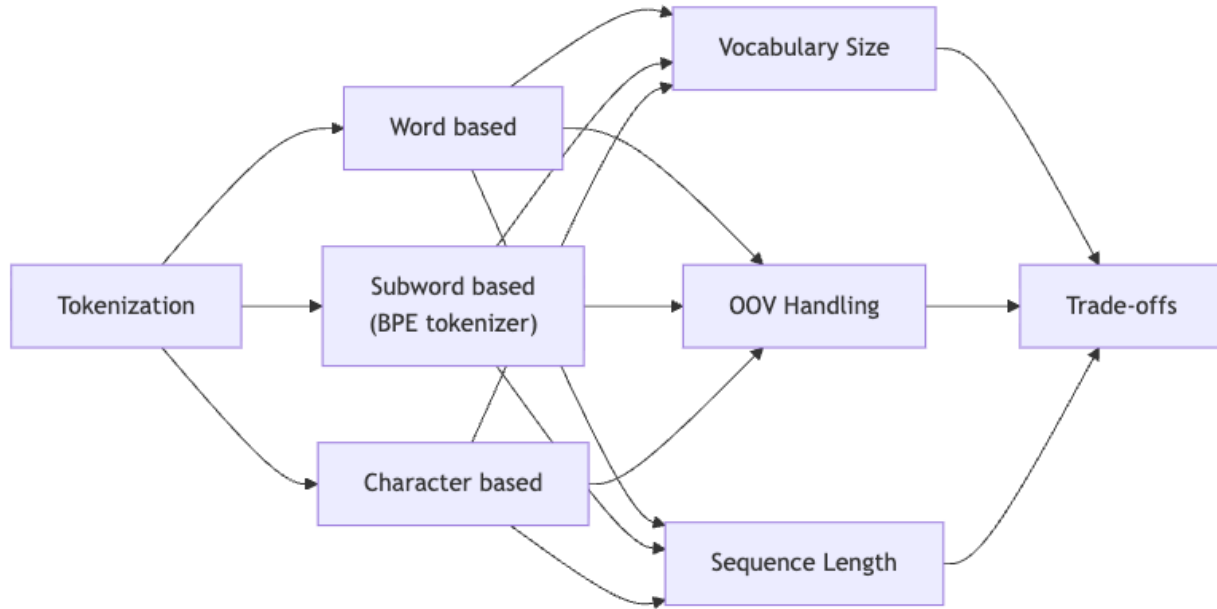


Figure 5.1: Comparison of three tokenization approaches (word-based, subword-based BPE, and character-based) showing their trade-offs across vocabulary size, out-of-vocabulary handling, and sequence length.

5.2.2 Splitting into Tokens

A naive whitespace split misses punctuation. A more robust approach splits on both punctuation and whitespace:

```
# Source:
tokens = re.split(r'[, \.;: \?!" \- \(\) /]|\s+', text)
tokens = [token for token in tokens if token.strip()]
```

This splits on commas, periods, semicolons, colons, question marks, exclamation marks, quotation marks, hyphens, parentheses, and forward slashes, as well as any whitespace. The list comprehension filters out empty strings that result from consecutive delimiters. An alternative regex pattern that achieves similar splitting is:

```
# Source:
result = re.split(r'[, .;: ?_!() "\\\- \s]', text)
```

followed by:

```
# Source:
[item for item in result if item.strip()]
```

Using `item.strip()` filters out whitespace tokens from the result. Applied to the book example, the raw text yields 4,690 tokens after the regex-based tokenization scheme is applied.

5.2.3 Building the Vocabulary

A vocabulary is a dictionary where every unique token is mapped to a unique integer called a token ID. To build one, collect all unique tokens, sort them for reproducibility, and enumerate them:

```
# Source:
vocab = sorted(set(preprocessed))
vocab_dict = {token: idx for idx, token in enumerate(vocab)}
```

You can also capture the vocabulary size at the same time:

```
# Source:
vocab = sorted(set(preprocessed))
vocab_size = len(vocab)
```

5.2.4 Encoding and Decoding

Encoding is the process of converting tokens into token IDs. Decoding is the reverse: mapping token IDs back to tokens, which is needed to convert the model's numerical output back into readable text.

A tokenizer class is a Python class containing an encode method to convert text into token IDs and a decode method to convert token IDs back into text. Its `__init__` method takes a vocabulary parameter—a mapping from tokens to token IDs—and stores two dictionaries internally: `str2int`, which maps tokens (strings) to token IDs (integers), and `int2str`, the reverse mapping from token IDs back to tokens.

```
# Source:
class SimpleTokenizer:
    def __init__(self, vocab):
        self.string_to_int = vocab
        self.int_to_string = {idx: token for token, idx in vocab.items()}

    def encode(self, text):
        tokens = re.split(r'[,.\;:\?!"\'-\(\)/]|s+', text)
        tokens = [token for token in tokens if token.strip()]
        return [self.string_to_int[token] for token in tokens]

    def decode(self, ids):
        text = ' '.join([self.int_to_string[idx] for idx in ids])
        return re.sub(r'\s+([,\.;:\?!"\'-\(\)/])', r'\1', text)
```

The tokenizer class uses `re.split` and `item.strip` to convert text into tokens. Here is what each method does:

- **encode:** Splits input text on punctuation and whitespace to produce individual tokens, then converts each token to its ID using the `str2int` dictionary. It takes text as input and outputs token IDs. In the context of a transformer architecture, the encode method corresponds to the encoder block, converting text to token IDs for training data.
- **decode:** Converts token IDs back to individual tokens using the `int2str` dictionary, then joins them into a single string. It takes token IDs as input and outputs text. A post-processing regex removes spurious spaces before punctuation (e.g., "Hello ," becomes "Hello,").

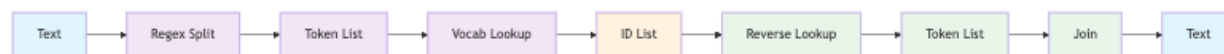


Figure 5.2: Tokenization encode and decode pipeline: the encode path converts text to token IDs via regex splitting and vocabulary lookup, while the decode path reverses the process to recover text.

5.3 Handling Unknown Words with Special Tokens

The SimpleTokenizer above will raise a `KeyError` if it encounters a word not in its vocabulary. The standard solution is to introduce **special tokens**—reserved tokens that handle unknown words and mark boundaries between text sources.

The **unknown token** (`<|unk|>`) is added to the vocabulary to represent any word absent from it. The **end-of-text token** (`<|endoftext|>`) is inserted between unrelated text sources to signal document boundaries and prevent the model from mixing separate documents. Together, these special tokens handle words not present in the vocabulary and separate multiple text sources.

5.3.1 Adding Special Tokens to the Vocabulary

```

# Source:
# Adding special tokens to vocabulary
pre_processed.extend(['<|endoftext|>', '<|unk|>'])
vocab = {token: idx for idx, token in enumerate(sorted(pre_processed))}

```

5.3.2 Updating the Encoder to Use the Unknown Token

```

# Source:
# In SimpleTokenizer v2 encode method: if a word is not in vocabulary, replace it
if item not in self.str2int:
    tokens.append(self.str2int['<|unk|>'])
else:
    tokens.append(self.str2int[item])

```

5.3.3 Other Special Tokens

Some researchers use additional special tokens beyond `<|unk|>` and `<|endoftext|>`:

- **BOS (Beginning of Sequence)**: marks the start of a text and signals to the LLM where a piece of context begins.
- **EOS (End of Sequence)**: positioned at the end of a text, useful for separating unrelated passages.
- **Padding token**: extends shorter texts in a batch to match the length of the longest text, enabling parallel processing of variable-length sequences.

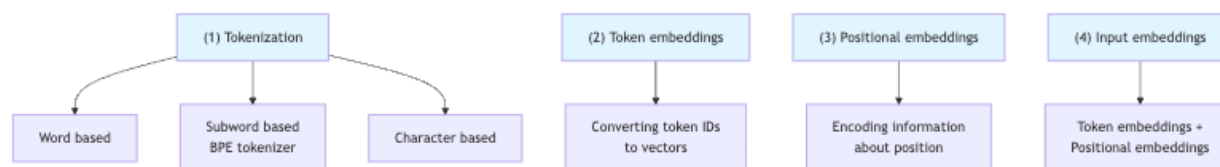


Figure 5.3: LLM Data Preprocessing: Four-step pipeline showing tokenization methods (word-based, subword-based with BPE, character-based), token embeddings, positional embeddings, and their combination into input embeddings.

5.4 The Limits of Word-Based Tokenization

Consider the words “old”, “older”, “finest”, and “lowest”. Word-based tokenization treats each as a distinct token, yielding four tokens for this small dataset—even though “old” and “older” clearly share a root. Subword tokenization solves this by retaining root words and breaking down rare words into subword units.

5.5 Byte Pair Encoding: The Algorithm

Byte pair encoding (BPE) is a subword-based tokenization method used by GPT models and distinct from simple regex-based splitting. Compared to word-level tokenization, BPE reduces vocabulary size while still handling rare and novel words gracefully.

5.5.1 Step 1: Initialize with Characters

The first step is to split all words into character-level tokens. A special end-of-word token (`</w>`) is appended to each word to mark word boundaries, enabling the algorithm to distinguish between subwords at word endings and the same subwords appearing within words. For example, “low” becomes `l o w </w>` and “lower” becomes `l o w e r </w>`.

5.5.2 Step 2: Count and Merge Pairs

The algorithm then identifies the most frequently occurring adjacent byte pair and replaces it with a new token not already present in the data. If “`l o`” appears very frequently, merging it into “`lo`” reduces the total number of tokens in the corpus. This process repeats iteratively, building up a merge table of learned subword units.

5.5.3 Step 3: Apply to New Text

Once the merge table is learned, the BPE tokenizer breaks down words not in its predefined vocabulary into smaller subword units or individual characters. It scans text from left to right, decomposing each word based on what is present in the vocabulary and assigning each piece a unique token ID. The version of BPE used for large language models is a tweaked variant of the standard algorithm, adapted to convert entire sentences into subwords.

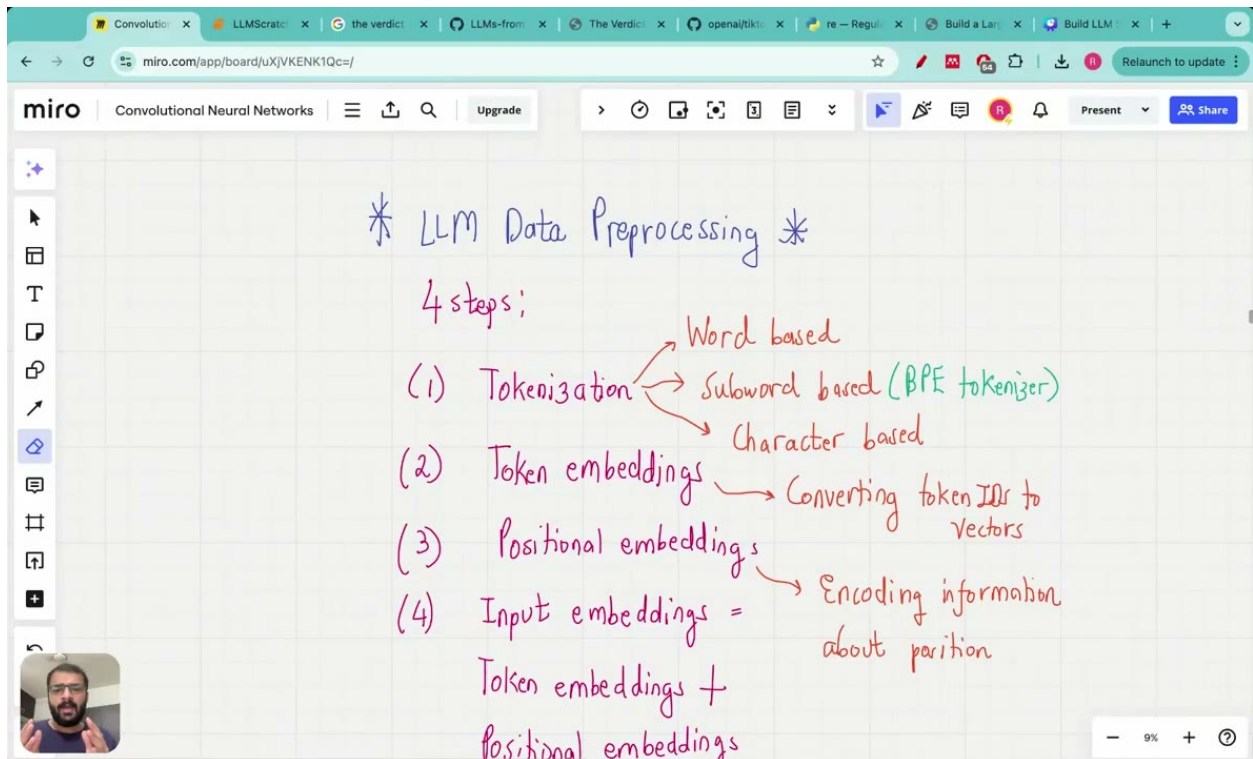


Figure 5.4: Step-by-step BPE merge example showing three rounds of merging the most frequent pair in a character-level corpus, with merge operations tracked in a table.

5.5.4 A Worked Example

After adding end-of-word markers and splitting into characters, the initial token inventory for “old”, “older”, “finest”, and “lowest” is: o, l, d, </w>, e, r, f, i, n, s, t, w. The algorithm counts all adjacent pairs across all occurrences, finds the most frequent, merges it into a new token, and repeats—ultimately retaining root words and decomposing rare words into subword units.

5.6 Using tiktoken for GPT-2 Compatible BPE

Implementing BPE from scratch is relatively complicated, so in practice we use the tiktoken Python library instead. The tiktoken library provides tokenizers using the same encodings as GPT-2.

5.6.1 Installation and Basic Usage

```
# Source:
import tiktoken
tokenizer = tiktoken.get_encoding('gpt2')
```

The tiktoken BPE tokenizer has an encode method that converts text into token IDs:

```
token_ids = tokenizer.encode(text)
print(token_ids)
```

e.g. `[15496, 11, 466, 345, 588, 8887, 30]`

5.6.2 How the BPE Tokenizer Handles Unknown Words

Rather than mapping unfamiliar words to a generic unknown token, the BPE tokenizer breaks them down into smaller subword units or individual characters. It scans text from left to right, decomposing each word based on what is present in the vocabulary and assigning each piece a unique token ID. This means that even a completely novel word—a proper noun or a technical term—will be represented as a sequence of recognizable subword pieces.

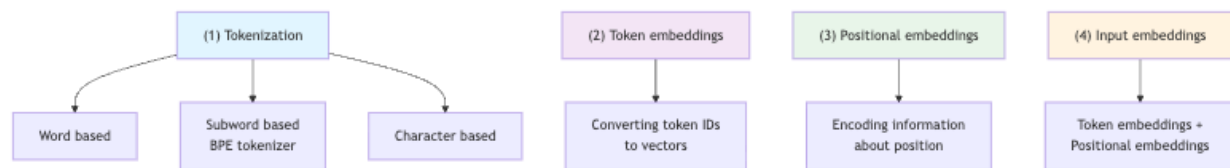


Figure 5.5: LLM Data Preprocessing: Four-step pipeline showing tokenization methods (word-based, subword-based BPE, character-based), token embeddings, positional embeddings, and final input embeddings.

5.7 Putting It All Together: The Full Tokenization Pipeline

Tokenization prepares input text for LLM training in two steps: splitting text into tokens and converting those tokens into token IDs. The encode method of a tokenizer class handles the forward direction—converting sample text into tokens and then into token IDs—while the decode method handles the reverse, converting token IDs back into text.

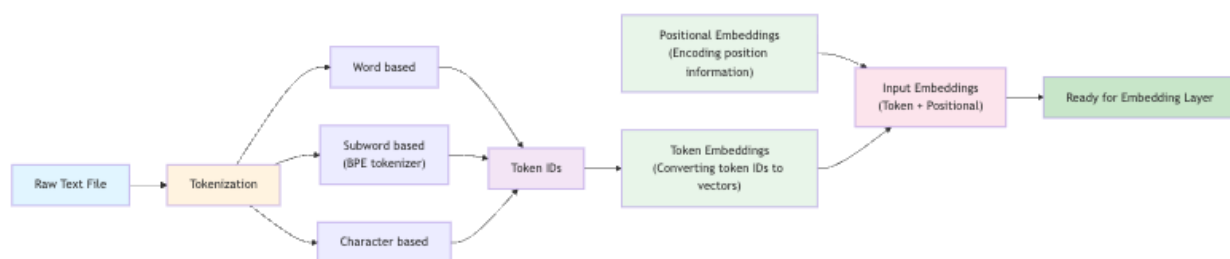


Figure 5.6: End-to-end LLM data preprocessing pipeline: from raw text through tokenization (word, subword, or character-based), token embeddings, positional embeddings, to final input embeddings ready for the embedding layer.

5.8 Summary

This chapter covered the full tokenization pipeline, from first principles to production tooling.

- **Three families of tokenization** exist—word-based, character-based, and subword-based—each making different trade-offs on vocabulary size, OOV handling, and sequence length.

- **Word-based tokenization** is easy to implement but suffers from large vocabularies and OOV words. We built a SimpleTokenizer class with encode and decode methods and extended it with special tokens (<|unk|> and <|endoftext|>).
- **Character-based tokenization** eliminates OOV with a tiny vocabulary but produces long sequences with little per-token semantics.
- **Subword-based tokenization**, and BPE in particular, offers the best of both worlds: it retains root words, handles rare words gracefully, and keeps vocabulary sizes manageable.
- **BPE** works by iteratively merging the most frequent adjacent byte pair, starting from a character-level initialization and using end-of-word markers to preserve word boundaries.
- The **tiktoken library** provides a production-ready BPE tokenizer compatible with GPT-2, accessible with just two lines of code.

Chapter 6

Data Preprocessing: Input-Target Pairs, Token Embeddings, and Positional Embeddings

Before a large language model can learn anything, raw text must be transformed into a form the model can actually consume. This chapter walks through the complete data preprocessing pipeline: creating input-target pairs from raw text, batching those pairs efficiently with a `DataLoader`, converting token IDs into dense vector embeddings, and finally injecting positional information so the model knows where each token sits in a sequence.

The pipeline has four distinct stages:

6.1 Creating Input-Target Pairs

6.1.1 Why LLMs Need a Special Pairing Strategy

Standard supervised learning tasks — classification, regression — come with explicit labels. Large language models, by contrast, use a specific technique for creating input-target pairs that differs from those conventional approaches.

Input-target pairs in LLMs are structured so that the input is a sequence of words up to a certain point and the target is the next word that follows. More precisely, there are two variables, x and y , where x contains input tokens and y contains targets that are the inputs shifted by one position. Tokens on the left of the arrow represent the input to the model, and the token on the right represents the target token ID the model is supposed to predict.

For example, given the sequence ["The", "cat", "sat"], the model should predict "on"; given ["The", "cat ", "sat ", "on"], it should predict "the". The same corpus of text generates thousands of such pairs automatically — no human labeling required.

Input-target pairs must be created before vector embeddings are fed to the LLM training process, and a dataset for language modeling must consist of these input-output pairs, not just raw tokens.

6.1.2 Context Size and the Sliding Window

A context size of 4 means the model is trained to look at a sequence of 4 tokens to predict the next one. Intuitively, context size represents how many words the model should pay attention to at one time.

Input-output pairs are constructed based on context size, where each pair contains multiple prediction tasks corresponding to predicting the next word at each position:

- Input $[t_0] \rightarrow$ target t_1
- Input $[t_0, t_1] \rightarrow$ target t_2
- Input $[t_0, t_1, t_2] \rightarrow$ target t_3
- Input $[t_0, t_1, t_2, t_3] \rightarrow$ target t_4

Input-output pairs are constructed by taking a sequence of tokens as input and the next token as the target output. The following code snippet illustrates this loop:

```
# Source:
context = encoded_data[:i]
desired = encoded_data[i]
```

A data loader fetches input-target pairs using a sliding window approach, advancing the window by one token position each time to produce the next pair.

Stride controls how far the window moves between consecutive pairs — specifically, how many tokens to skip when sliding the context window.

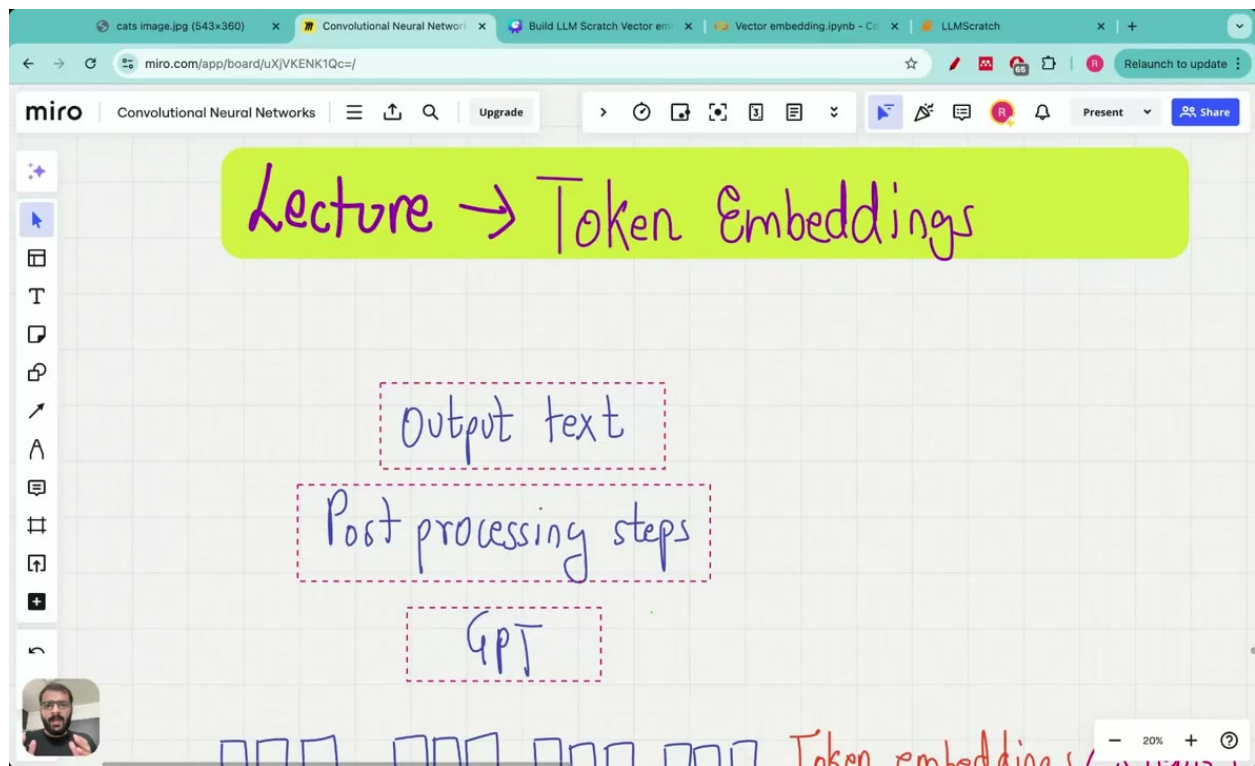


Figure 6.1: Token sequence with sliding window (`context_size=4`, `stride=1`) showing how successive input-target pairs are extracted from a token stream.

6.1.3 Loading the Raw Text

Before building pairs, we need encoded tokens. The following code reads a text file and encodes it with a tokenizer:

```
# Source:
raw_text = f.read()

enc_text = tokenizer.encode(raw_text)
print(len(enc_text))
```

This produces a flat list of integer token IDs ready for windowing.

6.2 The Dataset and DataLoader Abstraction

6.2.1 Why Use a DataLoader?

A data loader is a PyTorch tool that processes data by creating input-output target pairs using a sliding window approach, then iterating over those pairs and returning them as PyTorch tensors. A tensor can be thought of as a multi-dimensional array. Batch size specifies how many input-output pairs are processed before updating model parameters.

6.2.2 Dataset Structure

In a data loader implementation, the input tensor X contains rows representing input contexts, and the target tensor Y contains the corresponding prediction targets — that is, the inputs shifted by one word. The `__getitem__` method returns the input tensor row and target tensor row at a given index.

The `create_data_loader` function initializes a tokenizer, creates a dataset instance, and wraps it in a `DataLoader` for batch processing:

```
# Source:
data_loader = DataLoader(dataset, batch_size=8)
```

The `DataLoader` extracts input-output pairs in batches, which are then converted into vector embeddings before being fed into the model.

6.2.3 Workers and Parallel Loading

The `num_workers` parameter enables parallel computing by distributing data loading across multiple CPU threads, which can significantly speed up data ingestion for large datasets.

6.2.4 Verifying the Output

A `DataLoader` with batch size of 1 and context size of 4 produces input-output pairs where the output tensor is the input tensor shifted by one position — confirming that input-target pairs are sequences where the target is the input shifted by one position.

[Diagram: Batch of input-target pairs represented as tensors X and Y , where Y is X shifted right by one token position.]

6.3 Token Embeddings: From IDs to Vectors

6.3.1 What Is a Token Embedding?

Token embeddings map words into a vector space where each word is represented as a vector with a fixed number of dimensions, and they serve as the input to training large language models such as GPT.

The terminology can be confusing: token embeddings are also called vector embeddings or word embeddings, though “word embeddings” is not entirely accurate. The reason is that “token” is a broader term than “word” — a token might be a subword, a punctuation mark, or a special symbol — which is why “token embeddings” is the preferred terminology.

Token embeddings are created by converting token IDs into embedding vectors using an embedding weight matrix, making them the third step in the overall LLM workflow: (1) tokenize input text into tokens, (2) convert tokens into token IDs, and (3) convert token IDs into token embeddings.

6.3.2 The Embedding Weight Matrix

An embedding layer weight matrix is a lookup table that stores vector embeddings for each token ID in the vocabulary, with dimensions of (vocabulary_size, embedding_dimension). Constructing one requires two quantities: the vector dimension (the size each token is projected to) and the vocabulary size (the number of unique token IDs).

The weight matrix serves a dual role: it is both a matrix of trainable weights and a lookup table for retrieving vector representations by token ID, mapping vocabulary IDs to dense vectors of a specified embedding dimension.

6.3.3 Word2Vec: A Historical Perspective

The following code loads a classic Word2Vec model trained on Google News:

```
# Source:
import gensim.downloader as api
model = api.load("word2vec-google-news-300")
word_vectors = model
print(word_vectors['computer'])
```

Modern LLMs do not use pre-trained Word2Vec vectors; instead, they learn their own embeddings from scratch as part of end-to-end training. Creating token embeddings for large vocabularies is computationally expensive, which is one reason why training large language models like GPT takes a huge amount of time.

6.3.4 torch.nn.Embedding: The PyTorch Lookup Table

PyTorch provides a dedicated module for this purpose. `torch.nn.Embedding` is a lookup table that stores embeddings of a fixed dictionary and size, used to store word embeddings and retrieve them

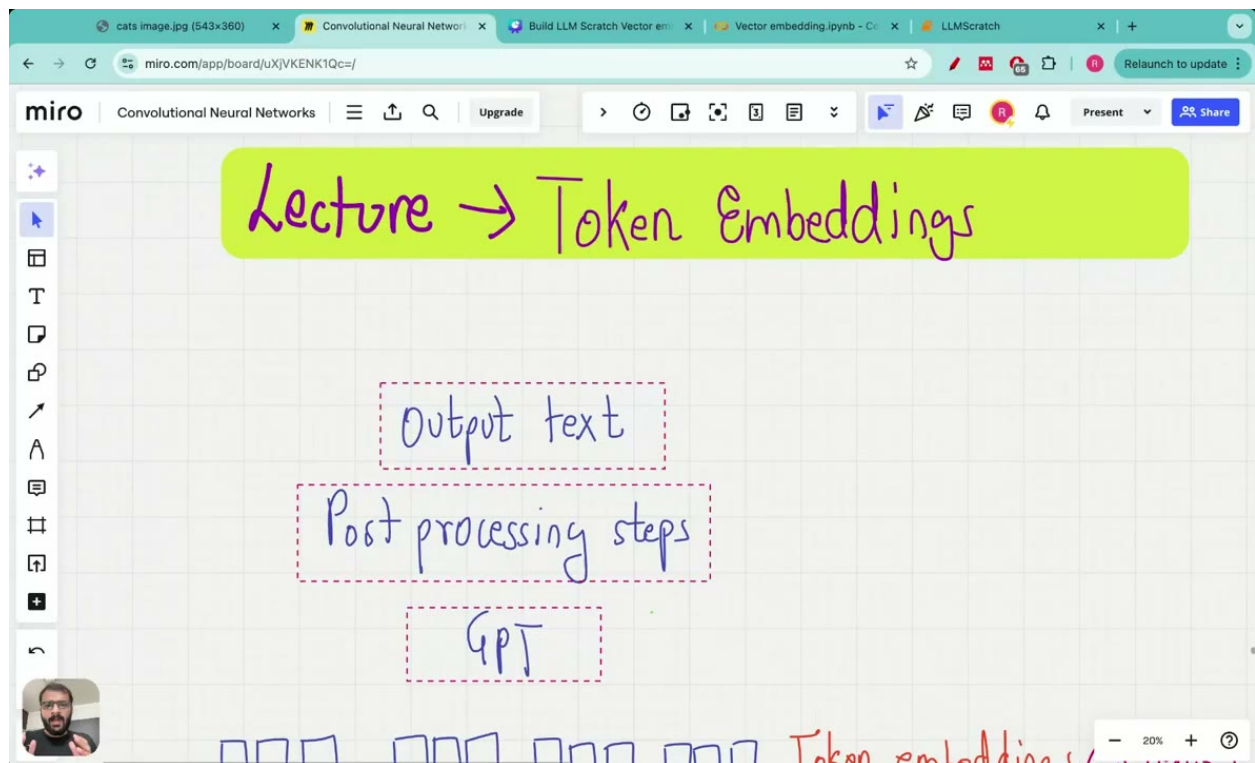


Figure 6.2: A rectangular embedding matrix of shape $(\text{vocab_size}, \text{embedding_dim})$ where each row corresponds to a token ID and contains its learned embedding vector; row lookup retrieves the embedding for a given token.

using indices:

```
# Source:
torch.nn.Embedding(num_embeddings, embedding_dim)
# Creates an embedding layer with num_embeddings rows and embedding_dim columns
# Stores embeddings of a fixed dictionary and size
```

```
# Source:
embedding_layer = torch.nn.Embedding(num_embeddings=6, embedding_dim=3)
```

`torch.nn.Embedding(num_embeddings, embedding_dim)` creates an embedding layer that functions as a lookup table, retrieving embeddings using token indices. To inspect the underlying weight matrix:

```
# Source:
embedding_layer.weight
```

The lookup operation retrieves only the embedding vectors corresponding to the input IDs, without computing embeddings for unused vocabulary IDs. Just as convolutional neural networks exploit spatial features of an image before training, token embeddings extract a structured representation of each token before it enters the model.

6.3.5 Building the Token Embedding Layer for GPT

```
# Source:
vocab_size = 50257
embedding_dim = 256
token_embedding_layer = torch.nn.Embedding(vocab_size, embedding_dim)
# Creates embedding matrix with 50257 rows and 256 columns
```

A simpler illustration of the same call:

```
# Source:
embedding = torch.nn.Embedding(vocab_size, embedding_dim)
# Example: torch.nn.Embedding(4, 5) creates a 4x5 embedding matrix
```

With a batch size of 8 and a context size of 4, the token embedding layer produces an output tensor of shape $8 \times 4 \times 256$.

6.4 The Problem with Token Embeddings Alone

Token embeddings capture semantic meaning — words with similar meanings end up close together in the vector space. However, token embedding does not take into account the position of words in a sequence, so identical words at different positions receive the same embedding vector. In other words, token embeddings alone carry no information about where a word appears in a sentence, and without that information the model cannot distinguish sentences that differ only in word order.

It is therefore important to inject additional position information into the model alongside the semantic content captured by token embeddings. Without token embeddings, the semantic relationship between related words like “dog” and “puppy” or “cat” and “kitten” is lost — but without positional embeddings, the *order* of those words is lost.

6.5 Positional Embeddings

6.5.1 What Is Positional Encoding?

Positional encoding is a technique that injects additional positional information into large language models to complement token embeddings. It is the final step in LLM data pre-processing, encoding information about the position of each token in a sequence. The terms “positional encoding” and “positional embedding” are used interchangeably to refer to vectors in a higher-dimensional space.

Both absolute and relative positional embeddings enable large language models to understand the order and relationship between tokens, leading to more accurate and context-aware predictions.

6.5.2 Absolute vs. Relative Positional Embeddings

There are two main approaches to encoding position information: absolute positional embeddings and relative positional embeddings.

In absolute positional embedding, the same token receives different final embeddings when it appears at different positions in the sequence, because a different positional embedding is added to the same token embedding at each position. This approach is well-suited for tasks where the fixed order of tokens is crucial, such as sequence generation. GPT models (GPT-3, GPT-4) use absolute positional embeddings that are optimized during training.

Relative positional encoding takes a different approach, emphasizing the relative distance between tokens rather than their absolute positions in the sequence. Intuitively, relative encodings let the model reason about “how far apart” two tokens are, rather than “what slot each token occupies.”

6.5.3 The Sinusoidal Formula (Original Transformer)

The original Transformer paper proposed a fixed, formula-based approach to absolute positional encoding using sinusoidal functions. For even dimensions:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

And for odd dimensions:

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

GPT-style models do not use this fixed formula; they use learned positional embeddings instead.

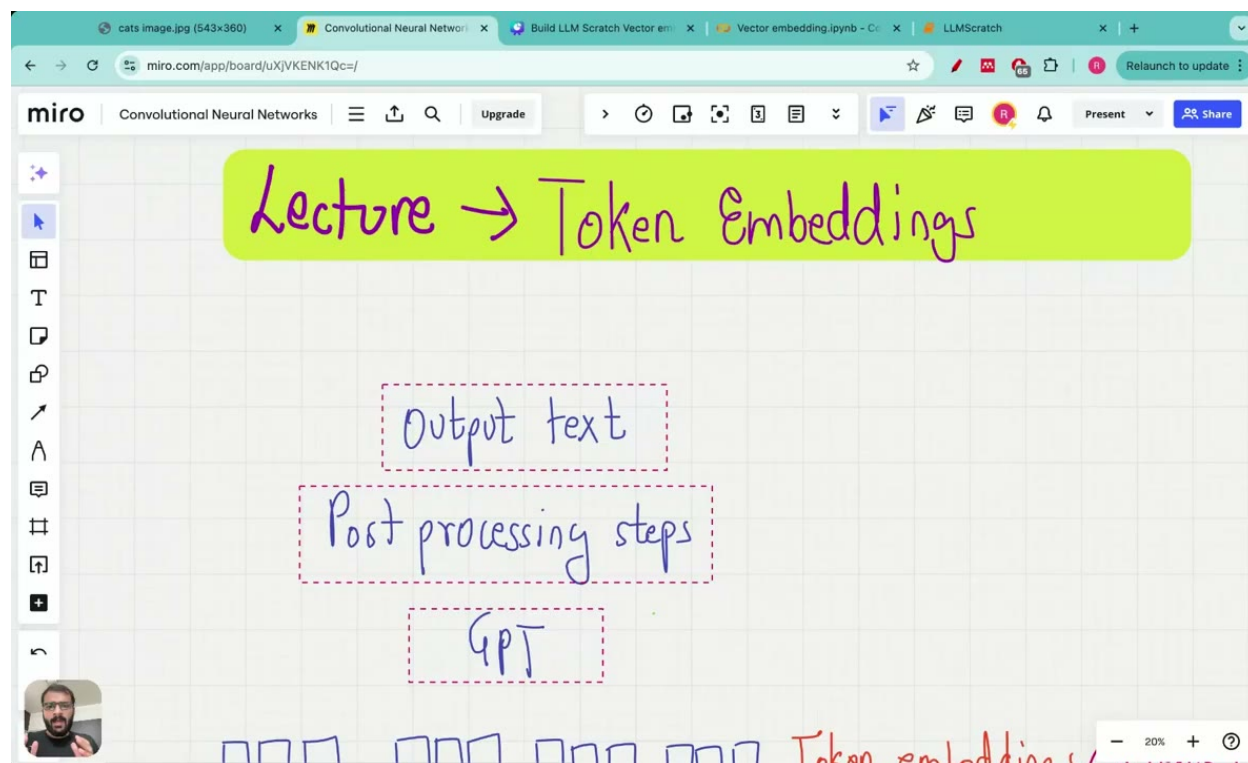


Figure 6.3: Side-by-side comparison of absolute positional embeddings (each position gets a unique fixed vector) versus relative positional embeddings (distances between positions are encoded).

6.5.4 Implementing Learned Positional Embeddings

Positional embedding matrix values are initialized randomly, similar to token embedding initialization, and both must be optimized during training.

To generate positional embeddings for a sequence of length `max_length`, we first create a sequence of position indices:

```
# Source:
torch.arange(max_length)
# creates a sequence of integers from 0 to max_length-1,
# used to index into the positional embedding lookup table
```

Passing positions 0, 1, 2, and 3 to the positional embedding lookup table generates four 256-dimensional positional embedding vectors. Because positions are identical across all sequences in a batch, the same four positional embedding vectors are reused for every input sequence.

6.6 Combining Token and Positional Embeddings

6.6.1 The Addition Operation

Positional embeddings are added to token embeddings using Python broadcasting, where the same positional embedding vectors are added to each row of the token embedding matrix. Concretely,

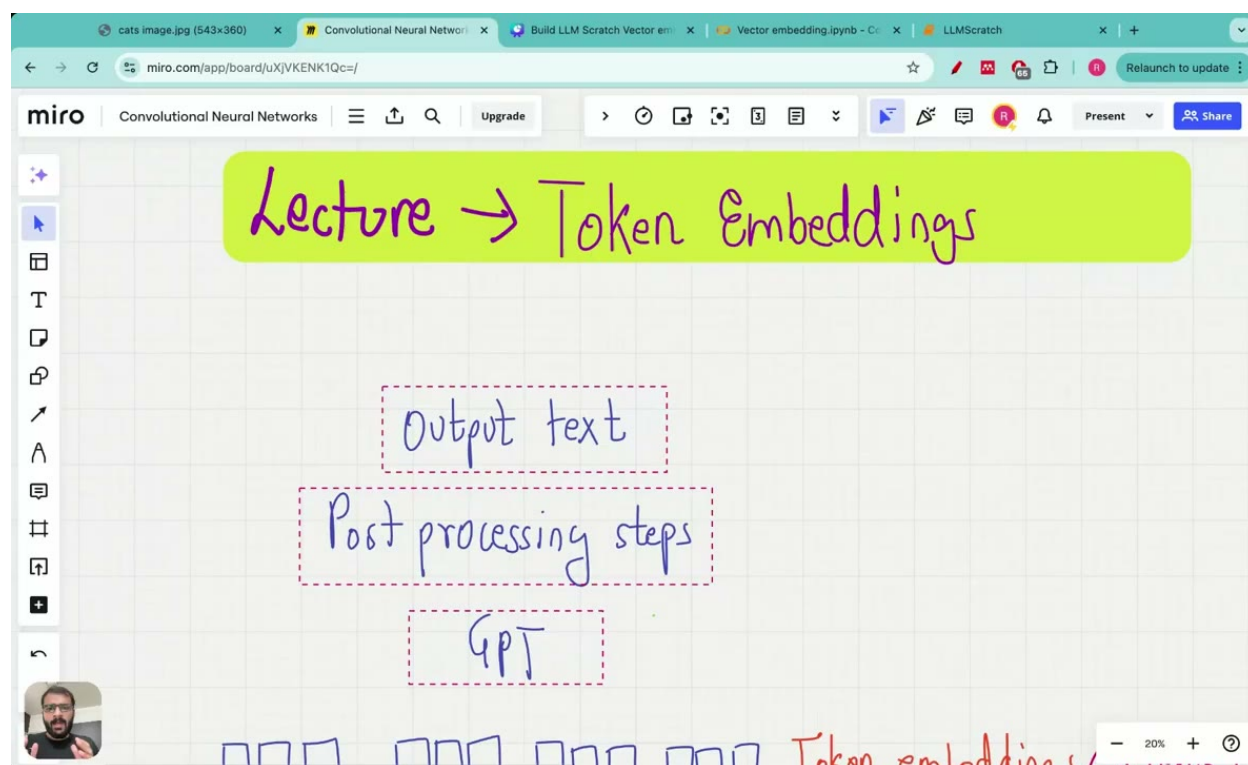


Figure 6.4: Positional embedding lookup table mapping position indices (0, 1, 2, 3) to distinct embedding vectors of shape (context_size, embedding_dim).

broadcasting converts the 4×256 positional embedding tensor into $8 \times 4 \times 256$ by duplicating values eight times — once for each item in the batch. PyTorch handles this expansion automatically, so no manual tiling is required.

6.6.2 The Complete Pipeline

With all four stages in place:

1. **Tokenization:** Raw text $\rightarrow$ token IDs (covered in earlier chapters)
2. **Token embeddings:** Token IDs $\rightarrow$ dense vectors via `torch.nn.Embedding(vocab_size, embedding_dim)`
3. **Positional embeddings:** Position indices $\rightarrow$ dense vectors via a second `torch.nn.Embedding(context_size, embedding_dim)`
4. **Input embeddings:** Token embeddings + positional embeddings $\rightarrow$ final input tensor

Token embeddings have shape $8 \times 4 \times 256$ and positional embeddings have shape 4×256 . The resulting input embeddings feed directly into the first layer of the transformer. During inference, the model's own predictions loop back as new inputs — making it an autoregressive model, where the output of one iteration becomes the input of the next.

6.7 Putting It All Together: A Worked Example

Let us trace a concrete example through the entire pipeline.

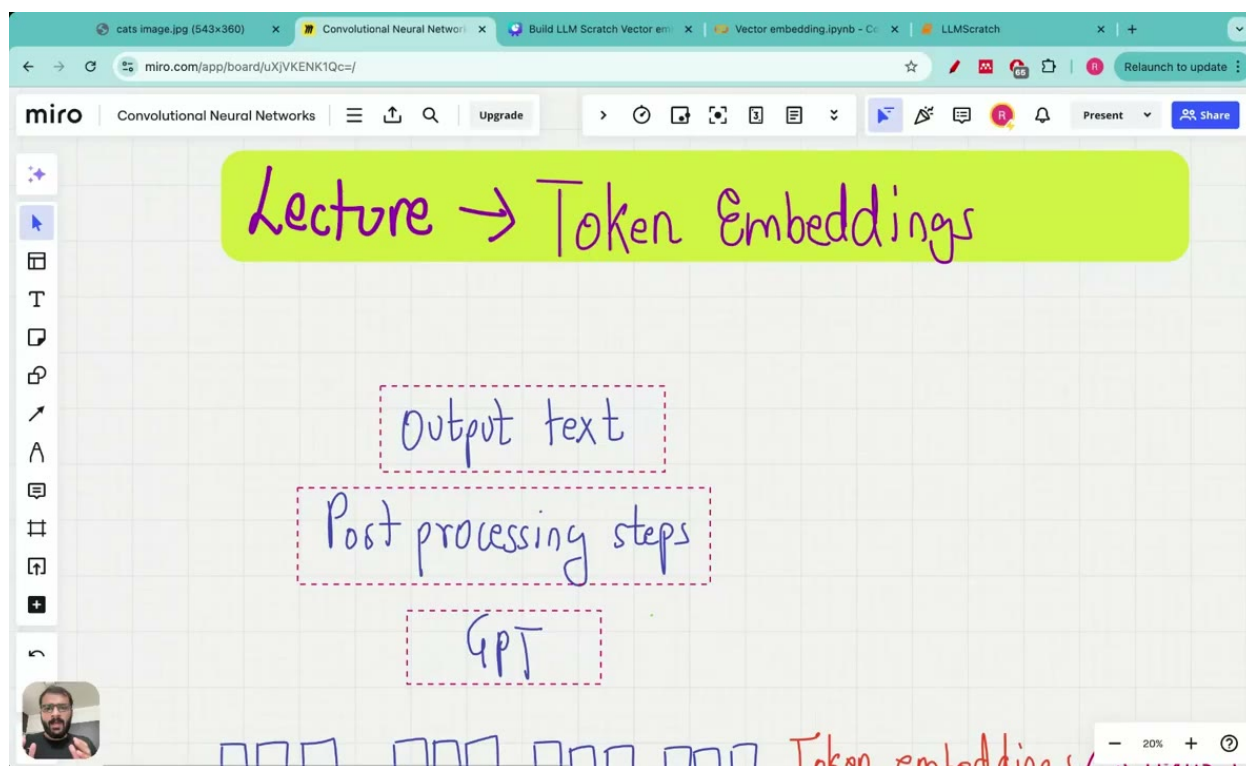


Figure 6.5: Token embedding tensor ($8 \times 4 \times 256$) plus positional embedding tensor (4×256) via broadcasting to produce input embedding tensor ($8 \times 4 \times 256$).

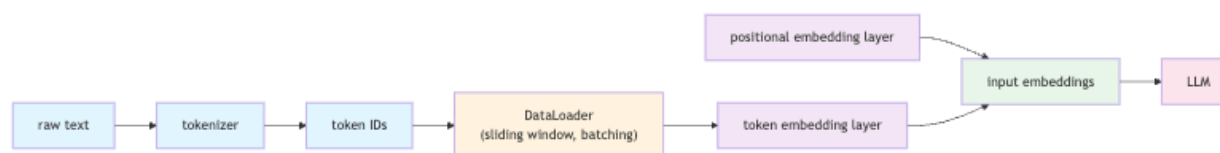


Figure 6.6: End-to-end LLM input pipeline: from raw text through tokenization, batching, and embedding layers to the language model.

Step 1 — Tokenize and encode. We read a text file and encode it into a list of integer token IDs.

Step 2 — Build input-target pairs. With a context size of 4 and a stride of 1, the sliding window produces pairs where each input is 4 tokens and each target is those same 4 tokens shifted right by one. The data loader uses this sliding window approach to fetch input-output target pairs.

Step 3 — Batch. A DataLoader with batch size 8 groups 8 such pairs into a single batch. The input tensor X has shape 8×4 and the target tensor Y has shape 8×4 .

Step 4 — Token embeddings. We pass X through `torch.nn.Embedding(50257, 256)`. The lookup retrieves one 256-dimensional vector per token ID, producing an output of shape $8 \times 4 \times 256$.

Step 5 — Positional embeddings. We create position indices `[0, 1, 2, 3]` with `torch.arange(4)` and pass them through a second `torch.nn.Embedding(4, 256)`, producing a tensor of shape 4×256 .

Step 6 — Combine. Broadcasting expands the positional embedding from 4×256 to $8 \times 4 \times 256$, and the element-wise sum yields the final input embedding tensor of shape $8 \times 4 \times 256$.

6.8 Summary

- **Input-target pairs** are constructed by pairing each sequence of tokens with the next token as the target. The sliding window with a configurable stride generates these pairs efficiently.
- **The DataLoader** wraps a dataset of input-target pairs into batches of PyTorch tensors, with optional parallel loading via multiple worker threads.
- **Token embeddings** are dense vector representations stored in a lookup table (`torch.nn.Embedding`). Each token ID maps to a row in the embedding weight matrix, and the values are learned during training.
- **Token embeddings alone are insufficient** because they carry no information about token order. Positional embeddings address this gap.
- **Positional embeddings** come in two flavors — absolute and relative. GPT-style models use learned absolute positional embeddings, initialized randomly and optimized during training.
- **Input embeddings** are the element-wise sum of token and positional embeddings, combined via broadcasting, and form the final input to the transformer’s first layer.

Chapter 7

The Attention Mechanism: From RNNs to Self-Attention

To understand self-attention, we need to understand *why* it exists—what problems it solves and what came before it. Along the way, we will build a working implementation of simplified self-attention from scratch in Python, step by step.

7.1 The Problem: Sequential Data and Long-Range Dependencies

The meaning of a word depends on the words around it—sometimes on words that appeared many sentences earlier.

7.1.1 Recurrent Neural Networks

The first serious attempt to handle sequential data with neural networks was the Recurrent Neural Network (RNN). RNNs process a sequence one token at a time, maintaining at each step a **hidden state**—a vector that accumulates information about everything the network has seen so far. This hidden state is the key innovation of RNNs: it is what allows them, in principle, to remember earlier parts of a sequence when processing later parts.

In sequence-to-sequence tasks like machine translation, an RNN encoder reads the entire source sentence and compresses it into a single vector—the **context vector**—which is the final hidden state of the encoder. This design works reasonably well for short sequences, but it runs into a fundamental wall as sequences grow longer.

7.1.2 The Vanishing Gradient and Loss of Context

RNNs struggle with **long-term dependencies**: complex sentence structures where multiple clauses or phrases are connected, making it difficult for the model to identify relationships between distant words.

Long Short-Term Memory networks (LSTMs) were designed to address this by maintaining both a long-term memory route and a short-term memory route. LSTMs are a meaningful improvement, but both RNNs and LSTMs still had problems with longer context. The fundamental bottleneck

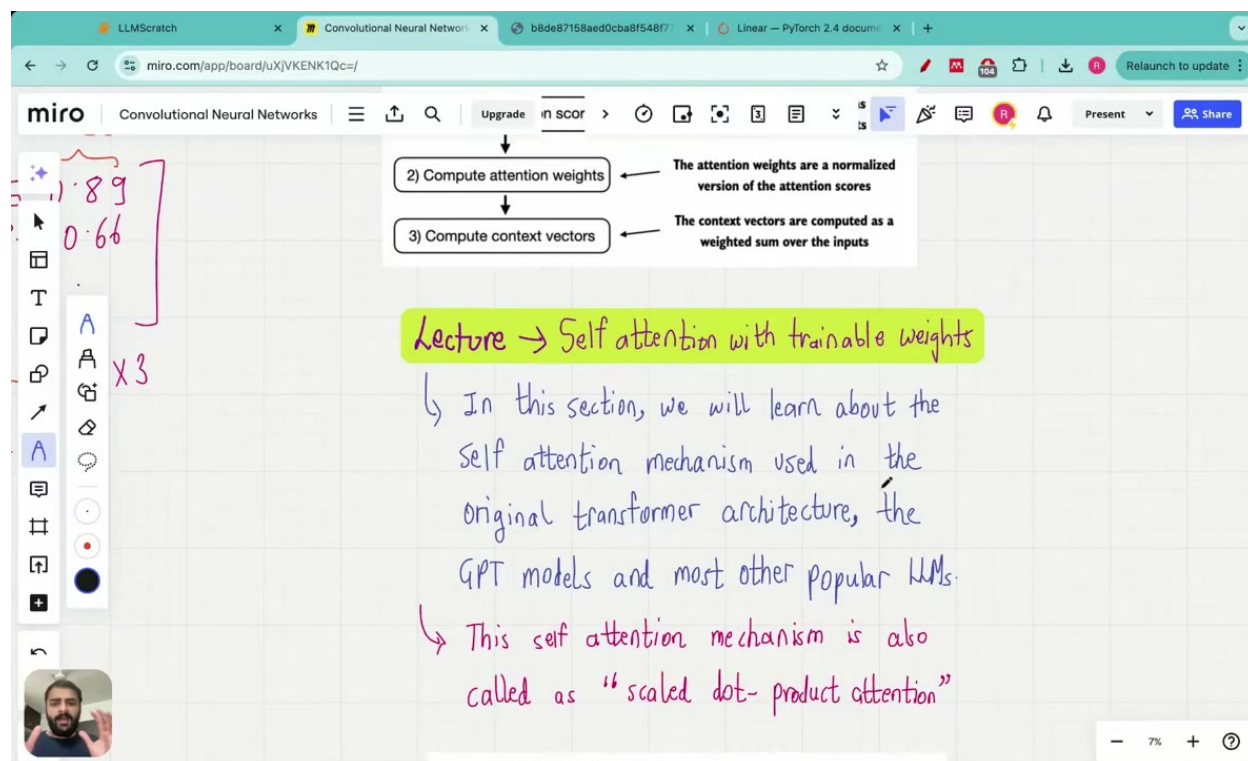


Figure 7.1: RNN unrolled over time with input tokens $x_1, x_2, \dots, x_n$ feeding into hidden states $h_1, h_2, \dots, h_n$ in sequence, with final hidden state passed to decoder.

remains: the entire source sequence must be compressed into a single fixed-size context vector. When the source sentence is long, that single vector simply cannot carry all the relevant information forward. The decoder, relying on only one final hidden state, struggles to capture longer dependencies and contextual information.

7.2 Bahdanau Attention: A First Solution

The breakthrough came in 2014, when Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio published “Neural Machine Translation by Jointly Learning to Align and Translate”. Their **Bahdanau attention mechanism** allows the decoder to selectively access different parts of the input sequence at each decoding step, rather than relying only on the final hidden state.

The key insight is **dynamic focus**: instead of a single context vector computed once at the end of encoding, the decoder computes a fresh, weighted combination of all encoder hidden states at every generation step. The attention mechanism allows the decoder to selectively choose which inputs to focus on and how much attention to give to each one at every decoding step. When decoding a particular part of the output, the decoder has access to all input tokens and decides how much attention to give to each. Traditional attention thus looks at one input sequence and one output sequence, determining which parts of the output are more related to which parts of the input.

7.3 From Attention to Self-Attention

The Transformer architecture, introduced in 2017, was built with the self-attention mechanism at its core, itself inspired by the Bahdanau attention mechanism.

Traditional attention operates *between* two sequences—for example, between a source sentence and a target sentence during translation. **Self-attention**, by contrast, looks at one sequence and examines how different parts of that same sequence relate to each other. The term “self” refers to the attention mechanism’s ability to compute attention weights by relating different positions within a single input sequence, learning the relationships between various parts of the input itself rather than between elements of two different sequences.

More formally, self-attention is a mechanism that allows each position of an input sequence to attend to all positions in the same sequence. It is used in the original Transformer architecture, GPT models, and most other popular large language models, and is also called **scaled dot-product attention**.

7.4 Simplified Self-Attention: The Core Idea

We will build up to the full self-attention mechanism in stages. The first stage is a **simplified self-attention** mechanism—the purest and most basic form of the attention technique—which does not include learnable parameters.

At a high level, self-attention transforms an input vector (a token embedding) into a **context vector** through attention weights computed from all input vectors. The context vector for a given token is an enriched embedding that combines contributions from all input embedding vectors, weighted by their corresponding attention weights. This chapter covers the mathematical foundations of this process and implements it from scratch in Python.

7.4.1 Notation

The input sequence consists of token embeddings $x_1, x_2, \dots, x_n$, where x_i denotes the vector representation of the i -th token. For a concrete example, consider the sentence “Your journey starts here.” The word “journey” is represented as x_2 . Vector embeddings capture semantic meaning, such that semantically related words like “journey” and “starts” are positioned closer to each other in vector space.

7.5 Step 1: Computing Attention Scores

The first step in self-attention is to compute **attention scores**—intermediate values that quantify how much importance or attention should be paid to each input word relative to a query word. A **query** is the element or token being examined; for example, “journey” (x_2) is the query when we are computing the context vector for that token.

Attention scores are calculated using the **dot product** between a query vector and each input vector. Recall that the dot product of two vectors a and b is:

$$a \cdot b = \sum_i (a_i \cdot b_i) = |a||b| \cos \theta$$

where θ is the angle between the vectors. In self-attention, the dot product determines the extent to which elements of a sequence attend to one another, and both the scores and the resulting attention weights encode information about how much the query vector and each input embedding are related.

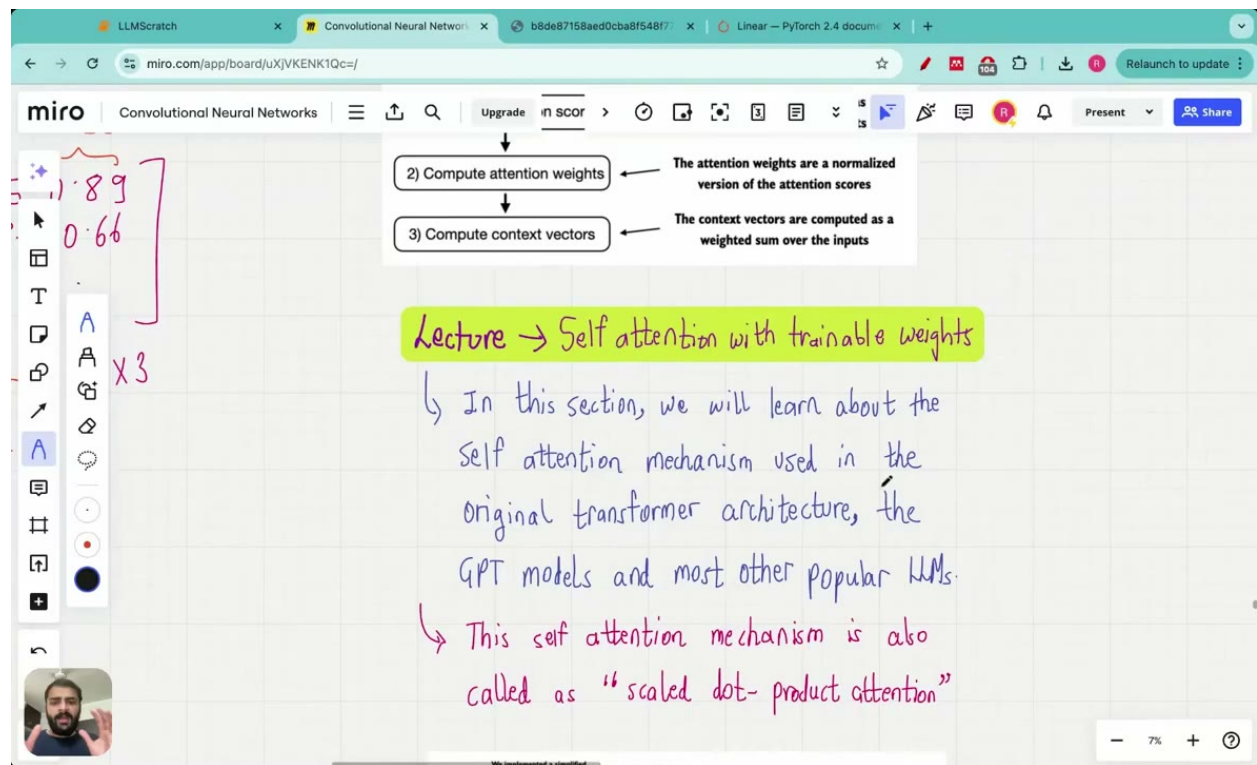


Figure 7.2: Self-attention computation pipeline showing the two main steps: computing normalized attention weights from attention scores, and computing context vectors as weighted sums over inputs.

7.6 Step 2: Normalizing to Attention Weights

Raw attention scores are useful, but what we really want is a distribution: how much attention, as a fraction, does the query pay to each token?

Normalization transforms attention scores so that they sum to one, enabling interpretable statements about attention allocation as percentages. The primary goal is to obtain attention weights that sum to one. These normalized scores are called **attention weights**—a normalized version of attention scores where each row sums to one, making them interpretable as probability distributions over tokens. Normalization also serves a second purpose: it helps backpropagation by keeping scales consistent between 0 and 1.

7.6.1 Softmax Normalization

The standard normalization function used in attention is softmax, which takes the exponent of every element and divides by the sum of all exponents:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Softmax attention weights are always positive because they result from the exponent operation. In PyTorch, softmax is available as `torch.nn.Softmax`, and the `dim` parameter specifies the dimension along which the normalization is computed.

In the simplified attention mechanism, attention weights are obtained through this normalization.

7.7 Step 3: Computing Context Vectors

With attention weights in hand, the context vector for a query token is simply a weighted sum of all input vectors. Formally:

$$z_i = \sum_j \alpha_{ij} \cdot x_j$$

The context vector, denoted z , is derived from attention weights and input vectors. For the query token “journey” (x_2), the context vector z_2 contains information about both the query token itself and all other input elements in the sequence. Attention weights determine how much attention to give to each input token when computing this vector: for instance, the attention weight between “journey” and “your” indicates how much the model should attend to “your” when processing “journey” as the query.

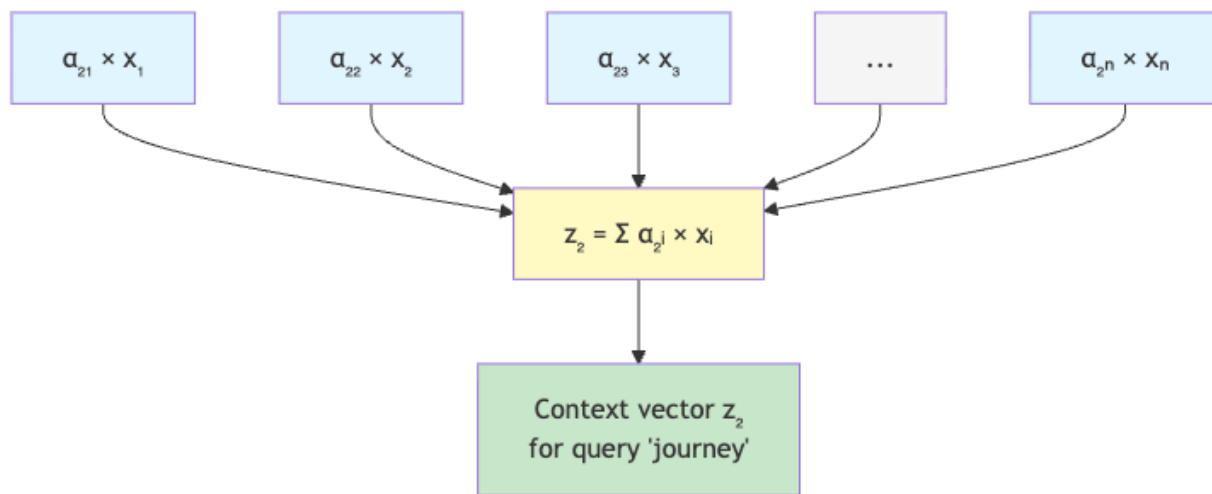


Figure 7.3: Attention mechanism computing context vector z as a weighted sum of input vectors x through x , where weights through are normalized attention scores.

7.8 Putting It Together: A Complete Walkthrough

When we compute context vectors for every token in the sequence simultaneously, we produce an **attention weight matrix**: a matrix where each row represents the attention weights for one particular query word, and each column represents the attention weight between that query and a key word.

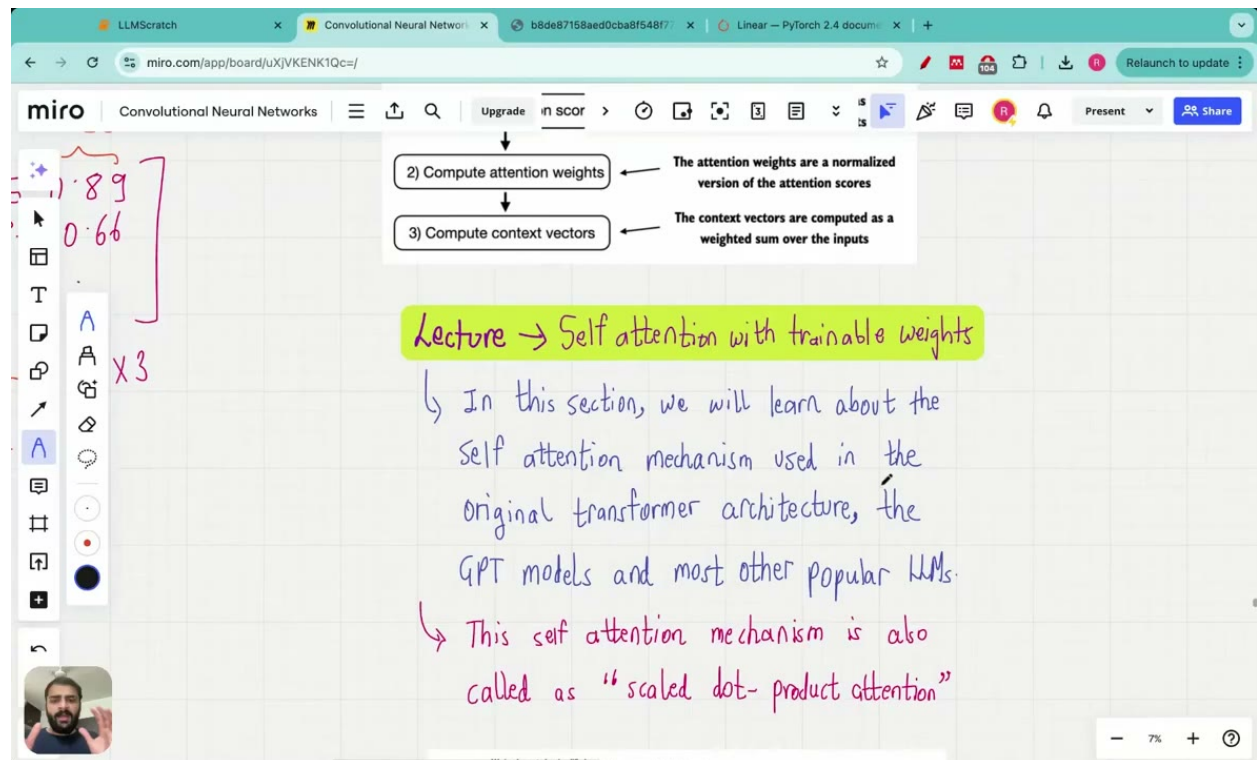


Figure 7.4: Attention weight matrix visualization showing normalized attention scores between query and key tokens, with row sums equal to 1.0.

7.9 Implementing Simplified Self-Attention in Python

The implementation follows directly from the three steps above: compute dot-product attention scores, normalize with softmax, and compute the weighted sum. In the simplified version, the input embeddings themselves serve directly as queries, keys, and values.

The following code shows how to compute scaled attention scores and weights:

```
# Source:
d_k = keys.shape[-1] # Extract key embedding dimension from last axis
attention_scores_scaled = attention_scores / (d_k ** 0.5) # Scale by sqrt(d_k)
attention_weights = softmax(attention_scores_scaled, dim=-1) # Apply softmax over
```

```
# Source:
context_vector = attention_weights @ values
```

We will revisit the scaling factor $\sqrt{d_k}$ shortly. For now, note that attention weights are computed by scaling attention scores by the square root of the key embedding dimension, then applying softmax.

7.10 Introducing Trainable Weights

The simplified mechanism has a fundamental limitation: using only the dot product to compute attention weights means attention is based solely on semantic similarity, which fails to capture contextual importance when semantically unrelated words are critical in context. Consider a sentence where a grammatically important function word like “not” needs to strongly influence the interpretation of a semantically distant content word—raw dot-product similarity would miss this relationship entirely.

Trainable weights solve this problem by allowing the model to learn attention patterns beyond semantic similarity, enabling it to assign high attention to contextually important words even when they are not semantically related to the query. More broadly, trainable weights allow the attention mechanism to capture long-range dependencies by learning contextual patterns beyond the semantic embedding space. These weight matrices are optimized during training of the large language model.

7.10.1 The Three Weight Matrices

The self-attention mechanism is implemented using three trainable weight matrices: **Query** (W_Q), **Key** (W_K), and **Value** (W_V). Their roles map onto a familiar analogy:

- The **Query** is analogous to a search query in a database and represents the current token the model is focusing on.
- The **Key** represents items in the input sequence and is used to match with the query.
- The **Value** represents the actual content or representation of the input items themselves.

7.10.2 Projecting Inputs Through Weight Matrices

With trainable weight matrices, instead of using raw input embeddings as queries, keys, and values, we project the inputs through the learned matrices:

```
# Source:
keys = inputs @ W_k
values = inputs @ W_v
queries = inputs @ W_q
```

Or equivalently, in a class-based implementation:

```
# Source:
keys = x @ W_key
queries = x @ W_query
values = x @ W_value
```

```
# Source:
attention_scores = attention_scores / (keys.shape[-1] ** 0.5)
attention_weights = softmax(attention_scores, dim=-1)

# Source:
context_vector = attention_weights @ values
```

7.10.3 Initializing Weight Matrices in PyTorch

In PyTorch, `torch.nn.Parameter` is a Tensor subclass used to mark tensors as module parameters; they are automatically added to the module’s parameter list and appear in the `parameters()` iterator. The Query, Key, and Value weight matrices are initialized as `torch.nn.Parameter` objects with random values and shape (`D_in`, `D_out`):

```
# Source:
torch.nn.Parameter(torch.rand(D_in, D_out))
```

Self-attention is organized as a Python class (`SelfAttentionVersion1`) derived from `nn.Module` to enable reusable instantiation and integration into larger LLM implementations.

7.11 Why Scale by $\sqrt{d_k}$?

In both the simplified and trainable versions, attention scores are divided by the square root of the key embedding dimension before applying softmax—this is the “scaled” part of “scaled dot-product attention”. Attention weights sum to one after softmax normalization, and the scaling helps maintain numerical stability across different embedding dimensions.

7.12 The Road Ahead

To summarize the conceptual journey covered in this chapter:

1. **RNNs** process sequences with a hidden state but struggle with long-range dependencies.
2. **LSTMs** improve on RNNs with dual memory routes but still bottleneck at a single context vector.
3. **Bahdanau attention** (2014) allows the decoder to attend to all encoder states at each step, introducing dynamic focus.
4. **Self-attention** generalizes this to a single sequence, allowing every position to attend to every other position.
5. **Simplified self-attention** (no trainable weights) demonstrates the core dot-product $\rightarrow$ normalize $\rightarrow$ aggregate pattern.
6. **Full self-attention** (with W_Q , W_K , W_V) adds learnable projections that allow the model to go beyond raw semantic similarity.

The lecture series will go on to cover causal attention and multi-head attention, with mathematical formulations and code implementations in Python. **Causal attention** modifies the self-attention mechanism to prevent the model from accessing future information in the sequence—a critical

property for autoregressive language models like GPT, which must predict the next token without looking ahead. In the next chapter, we will implement the full self-attention mechanism with trainable weights, add the causal mask, and stack multiple attention heads to build the complete Transformer building block.

Chapter 8

Self-Attention with Trainable Weights: Keys, Queries, and Values

The previous chapter introduced a simplified attention mechanism that computes context vectors directly from raw input embeddings. It worked, but it had a fundamental limitation: there were no trainable weights, so the model had no way to *learn* what to attend to. This chapter fixes that by introducing the full self-attention mechanism—the one actually used in transformers, GPT models, and most other popular large language models.

You will see this mechanism called both **self-attention** and **scaled dot-product attention**. The core idea is to introduce three trainable weight matrices—a Query matrix W_Q , a Key matrix W_K , and a Value matrix W_V —that transform raw input embeddings into richer representations before any attention computation happens. These matrices are initialized randomly and optimized during training, allowing the model to learn, through gradient descent, how to produce context vectors that are genuinely useful for the task at hand.

8.1 Why Trainable Weights?

In the simplified mechanism, attention scores were calculated using a direct dot product between input vectors, and attention weights were obtained by normalizing those scores. The model had no parameters to adjust, so it could not improve its attention patterns through training.

The trainable weight matrices solve this by projecting each input embedding into three distinct roles:

- **Query:** what the current token is looking for
- **Key:** what each token in the sequence offers as a match
- **Value:** the actual content that gets aggregated into the output

The attention scores measure how well each key matches the query, and the context vector is a weighted blend of the values, weighted by those match scores. Because the weight matrices are trainable parameters optimized during LLM training, the model can learn to ask the right questions (queries), expose the right features for matching (keys), and encode the right information for aggregation (values).

8.2 The Three Weight Matrices

Self-attention requires exactly three trainable weight matrices: W_Q , W_K , and W_V . Each input embedding vector is multiplied with all three matrices separately to produce the corresponding query, key, and value vectors.

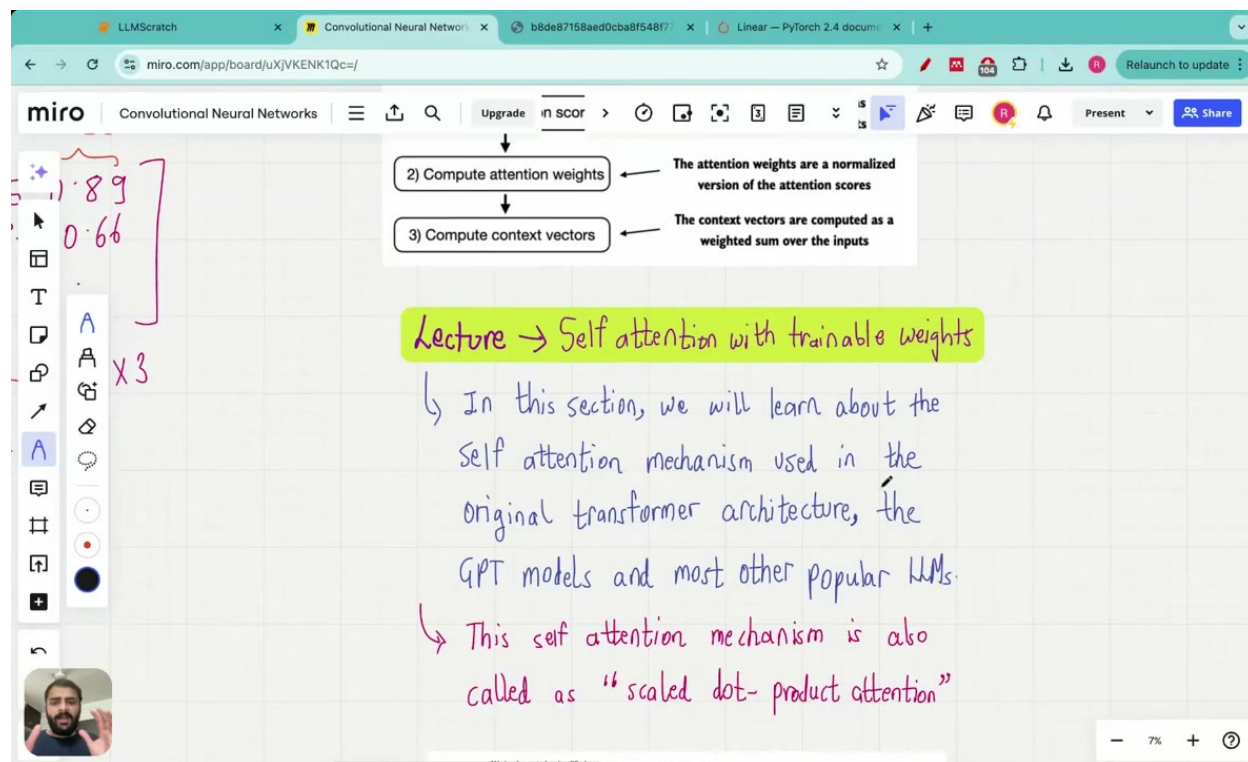


Figure 8.1: Self-attention mechanism with trainable weight matrices: a single input embedding is projected through three separate weight matrices (W_Q , W_K , W_V) to produce query, key, and value vectors.

In a concrete example, the input embeddings live in a 3-dimensional space, and the weight matrices project them into a 2-dimensional space for keys, queries, and values. The Key weight matrix W_K , for instance, has dimensions 3×2 , transforming each 3-dimensional input embedding into a 2-dimensional key vector. For the word “journey” (the second input vector), multiplying the 3-dimensional input vector by the 3×2 Query weight matrix produces a 2-dimensional query vector with values approximately $[0.4306, 1.4551]$.

When all six input tokens are projected through the key, query, and value weight matrices, the resulting matrices each have shape $(6, 2)$: six rows for the six input tokens and two columns for the embedding dimension. The input dimension of these weight matrices must match the embedding vector dimension, because a dot product is computed between them.

The keys, queries, and values are obtained by multiplying input embeddings with these separate trainable weight matrices:

Source:

```

keys = inputs @ W_k
values = inputs @ W_v
queries = inputs @ W_q

```

8.3 Computing Attention Scores

Each query vector has an attention score with every key vector in the sequence. For a query matrix of shape 6×2 , computing attention scores for the second query (the token “journey”) involves taking the dot product of that query row with all key rows to produce 6 attention scores.

The attention scores encode how much the query vector and each key vector are related to each other. Geometrically, when two vectors are aligned (small angle between them), their dot product is maximum, indicating high attention. The model learns to align query and key vectors for tokens that should attend strongly to each other. However, raw attention scores do not sum to one and are not directly interpretable as attention percentages.

8.4 Scaling by $\sqrt{d_k}$: Why It Matters

Before applying softmax, the attention scores are divided by the square root of the key embedding dimension—which is one of the reasons the mechanism is called *scaled* dot-product attention. The scaling factor is $\sqrt{d_k}$, where d_k is the embedding dimension of the keys. In the running example, the key embedding dimension is 2, so attention scores are divided by $\sqrt{2}$.

To understand why this matters, consider what happens to the dot product as the dimension grows. For a d -dimensional query and key vector sampled from a normal distribution, the variance of their dot product before scaling is approximately equal to d ; without scaling, this variance grows proportionally with the dimension. When softmax receives large input values, its output becomes *peaky*—the highest value receives disproportionately high probability compared to all others—making the model overly confident in a single key and destabilizing learning. Dividing by $\sqrt{d_k}$ keeps the variance of the attention scores close to 1 regardless of the query and key dimension, which is essential for stable learning and proper backpropagation.

8.5 From Scores to Attention Weights

Attention weights are computed by scaling the attention scores by $\sqrt{d_k}$ and then applying softmax:

```

# Source:
d_k = keys.shape[-1] # Extract key embedding dimension
attention_scores_scaled = attention_scores / (d_k ** 0.5) # Scale by sqrt(d_k)
attention_weights = softmax(attention_scores_scaled, dim=-1) # Apply softmax over d

```

The result is a matrix of **attention weights**: a normalized version of the attention scores where each row sums to one, making the weights interpretable as probability distributions over tokens.

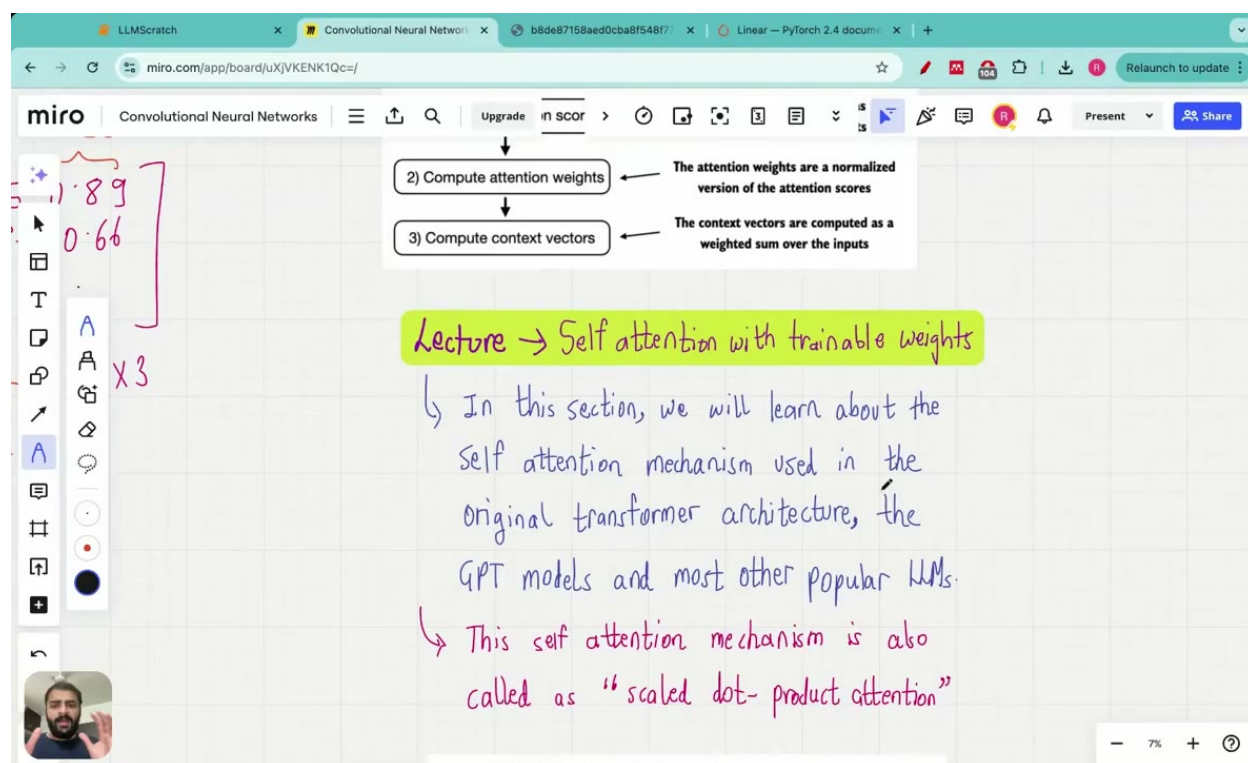


Figure 8.2: Attention mechanism computation steps: (2) computing attention weights as normalized attention scores, and (3) computing context vectors as weighted sums over inputs.

All values lie between zero and one, and this normalization serves two purposes: making attention interpretable and helping backpropagation by keeping scales consistent.

For the full sequence of six tokens, the attention weights matrix has dimensions 6×6 , computed by applying the scaling and softmax operation to the full 6×6 attention scores matrix. One important caveat: attention scores are meaningless until the weight matrices are trained; their values only become semantically significant after training.

8.6 Computing Context Vectors

The final step is to compute context vectors as a weighted sum over the value vectors. The attention weight between two tokens indicates how much the model should attend to the second token when processing the first as the query, and the attention scores and value representations are used together to produce the final context vector.

Concretely, to compute the context vector for “journey,” each value vector is scaled by its corresponding attention weight and the results are summed. The attention weights for “journey” in the example are: 0.15 for “your,” 0.2264 for “journey,” 0.2199 for “begins,” 0.13 for “width,” 0.09 for “one,” and 0.18 for “step”. Accordingly, the “your” vector is multiplied by 0.15, the “journey” vector by 0.22, the “begins” vector by 0.2199, the “width” vector by 0.13, the “one” vector by 0.09, and the “step” vector by 0.18, and these scaled vectors are then added together. The resulting

context vector for “journey” equals $[0.3061, 0.8210]$.

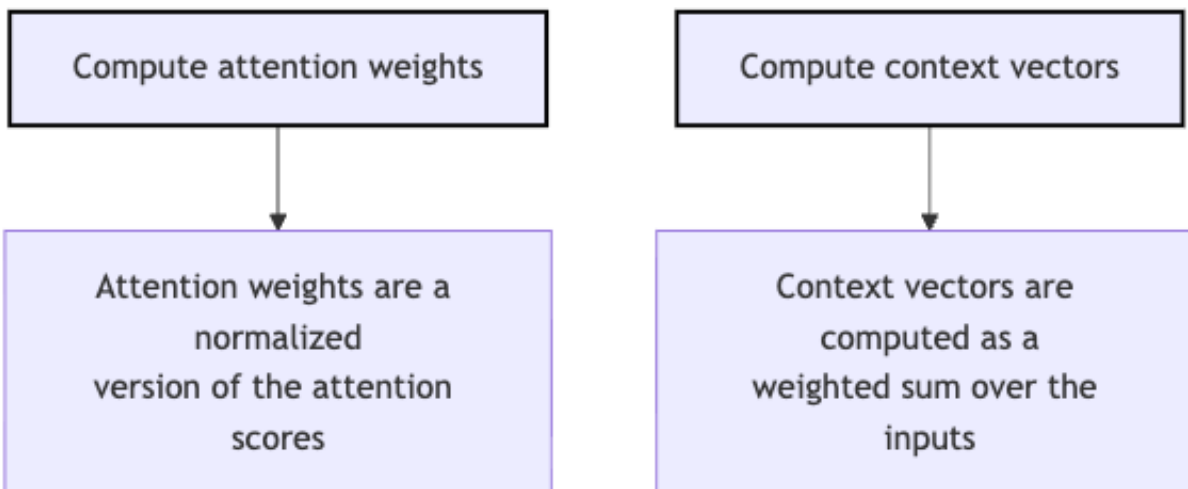


Figure 8.3: Two-step process for computing attention in self-attention mechanisms: first normalizing attention scores into weights, then computing context vectors as weighted sums of inputs.

In matrix form, the context vector for a specific token is obtained by taking that token’s row from the attention weights matrix (1×6) and multiplying it by the values matrix (6×2) to produce a 1×2 vector. Whether computed graphically—by scaling individual value vectors and summing—or via matrix multiplication, both approaches yield identical results:

```
# Source:
context_vector = attention_weights @ values
```

8.7 Putting It All Together: The Forward Pass

The complete forward pass of scaled dot-product attention proceeds as follows. First, project the inputs into queries, keys, and values:

```
# Source:
keys = x @ W_key
queries = x @ W_query
values = x @ W_value
```

Then scale the attention scores and apply softmax:

```
# Source:
attention_scores = attention_scores / (keys.shape[-1] ** 0.5)
attention_weights = softmax(attention_scores, dim=-1)
```

Finally, compute the context vectors:

```
# Source:
context_vector = attention_weights @ values
```

The objective throughout is to compute a context vector for every input token using these trainable weight matrices.

8.8 Implementing SelfAttentionV1 in PyTorch

With the forward pass understood, we can package this logic into a proper PyTorch class. Self-attention is organized as a class derived from `nn.Module` to enable reusable instantiation and integration into larger LLM implementations; `nn.Module` is a fundamental building block of PyTorch models, providing the necessary infrastructure for layer creation and parameter management.

8.8.1 Initializing the Weight Matrices

The `__init__` constructor initializes query, key, and value weight matrices randomly with dimensions `d_in` rows by `d_out` columns. `torch.nn.Parameter` is a Tensor subclass that marks tensors as module parameters, automatically adding them to the module's parameter list and exposing them through the `parameters()` iterator. The three weight matrices are therefore initialized as `torch.nn.Parameter` objects with random values and shape (D_{in}, D_{out}) , and they are optimized during training.

8.8.2 The Forward Method

The forward method takes an input embedding matrix `x` and returns a context vector matrix where each row corresponds to the context vector for one input token. When instantiating `SelfAttentionVersion1` with `input_dimension=3` and `output_dimension=2` and passing six input embedding vectors, the output is a context vector matrix with dimensions 6×2 .

8.9 Implementing SelfAttentionV2: Using `nn.Linear`

A cleaner design is to replace `nn.Parameter` with `nn.Linear` layers (with `bias=False`) to initialize the query, key, and value weight matrices. Using `nn.Linear` is more common practice when implementing self-attention for LLMs, as it handles weight initialization in a principled way and integrates cleanly with PyTorch's optimizer and parameter tracking infrastructure.

8.10 Interpreting the Mechanism

It is worth pausing to build intuition for what the three matrices actually do. The **query** represents the current token the model is focusing on—analogous to a search query in a database. The **key** represents items in the input sequence and is used to match against the query. The **value** represents the actual content or representation of those input items, which gets blended into the output. Importantly, the key and value for the same token can be very different vectors, because they serve entirely different purposes.

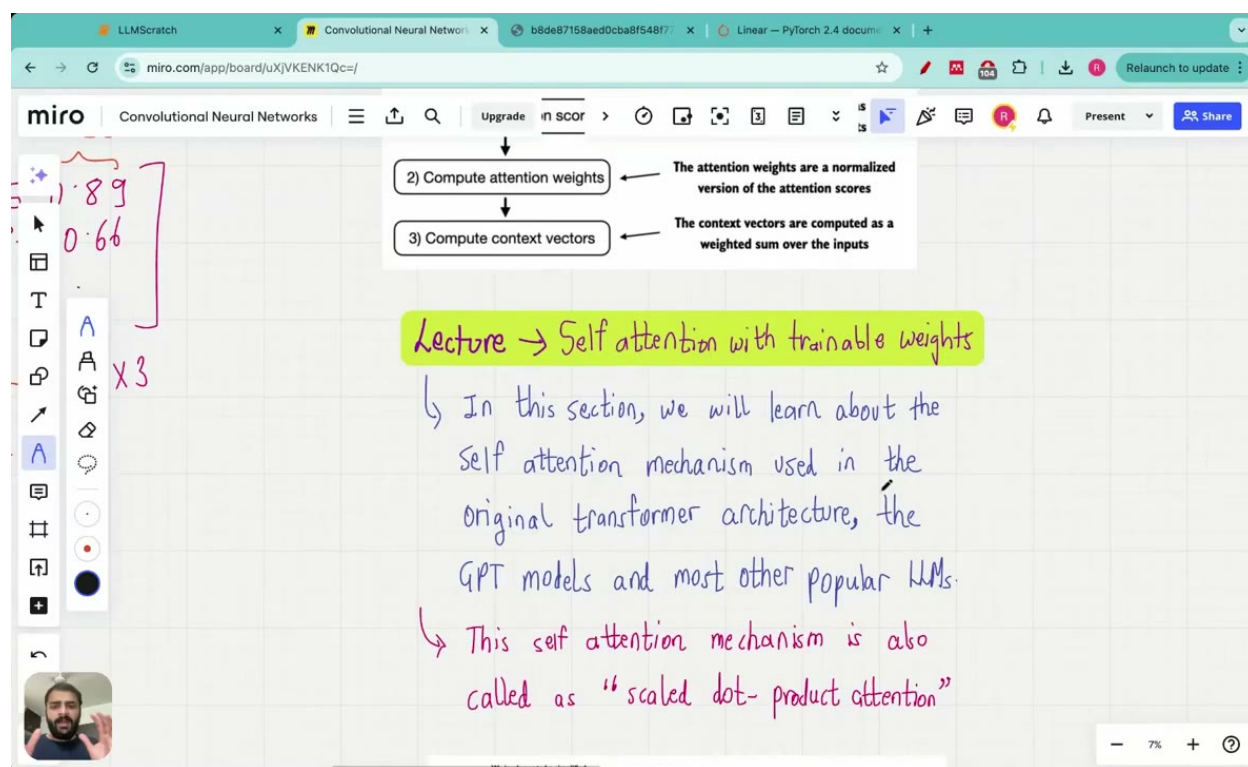


Figure 8.4: Flow diagram showing the two main steps of self-attention computation: computing normalized attention weights from attention scores, and computing context vectors as weighted sums over inputs.

In contrast to the earlier simplified mechanism, where the query was simply the token’s own embedding, the learned query, key, and value projections allow the model to develop specialized representations for each role.

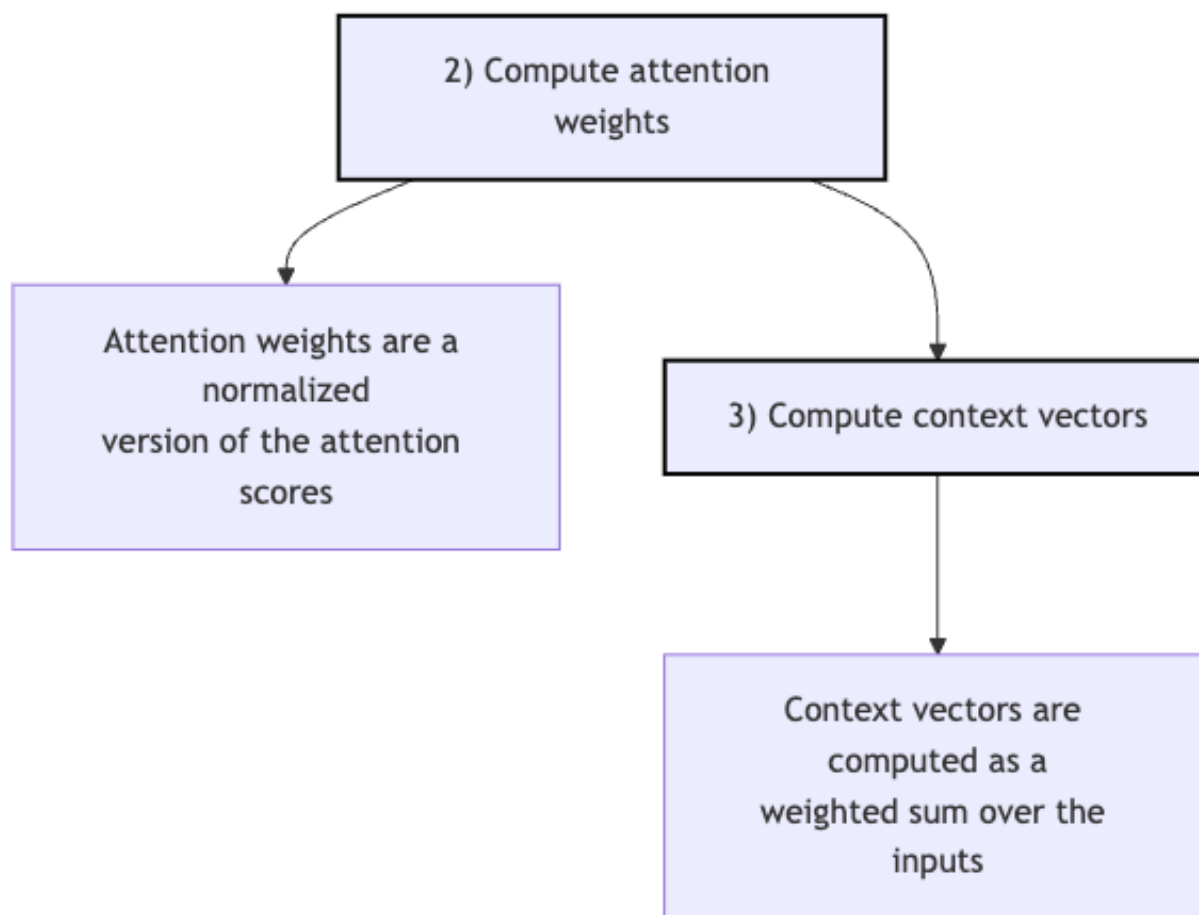


Figure 8.5: Two-step attention computation process: computing normalized attention weights from attention scores, then computing context vectors as weighted sums over inputs.

8.11 The Full Picture

To summarize, scaled dot-product attention proceeds in five steps:

1. **Linear projections:** Multiply input embeddings by W_Q , W_K , W_V to get queries Q , keys K , and values V .
2. **Attention scores:** Compute $QK^\top$ via dot products.
3. **Scaling:** Divide by $\sqrt{d_k}$ to keep variance close to 1.
4. **Softmax:** Apply softmax row-wise to obtain attention weights that sum to one.
5. **Context vectors:** Multiply attention weights by V to produce the output.

This mechanism is used in the original transformer architecture, GPT models, and most other

popular large language models. Understanding it at this level of detail—from the matrix shapes to the scaling rationale—is essential for everything that follows.

8.12 What Comes Next

The self-attention mechanism developed here processes all tokens symmetrically: every token can attend to every other token, including those that appear later in the sequence. For language modeling, however, the model should not be allowed to peek at future tokens when predicting the next word. Causal attention modifies the self-attention mechanism to prevent exactly that, by masking out future positions—and that is the subject of the next chapter.

Chapter 9

Causal Self-Attention and Multi-Head Attention

Self-attention, as introduced in earlier chapters, lets every token attend to every other token in the sequence—exactly what you want for tasks like sentiment classification, where the full context is available at inference time. Language modeling is different: when a model generates the next word, it has not yet seen the words that follow the current position. Allowing the model to peek at future tokens during training would be cheating, because the model would learn to rely on information it cannot possibly have at generation time.

The solution is **causal attention**, sometimes called **masked attention**. This chapter builds causal attention from the ground up, explains why masking must happen *before* the softmax, adds dropout for regularization, and then scales the whole mechanism up to **multi-head attention**—the version that actually runs inside GPT.

9.1 The Problem with Unrestricted Self-Attention

Recall the standard self-attention pipeline: compute queries, keys, and values from the input; form an attention score matrix; apply softmax to get attention weights; and finally compute context vectors as a weighted sum of the values. In this unrestricted form, every token can attend to every other token, including tokens that appear later in the sequence.

Causal attention is a special form of self-attention that restricts the model to only consider the previous and current inputs when processing any given token. Concretely, it is designed to prevent future tokens from influencing past tokens. In GPT-like large language models, this is achieved by masking out future tokens when computing attention scores.

9.2 Building the Causal Mask

The causal mask is the set of attention weights above the diagonal that are masked out. To understand the geometry: an upper triangular matrix has all elements below the diagonal set to

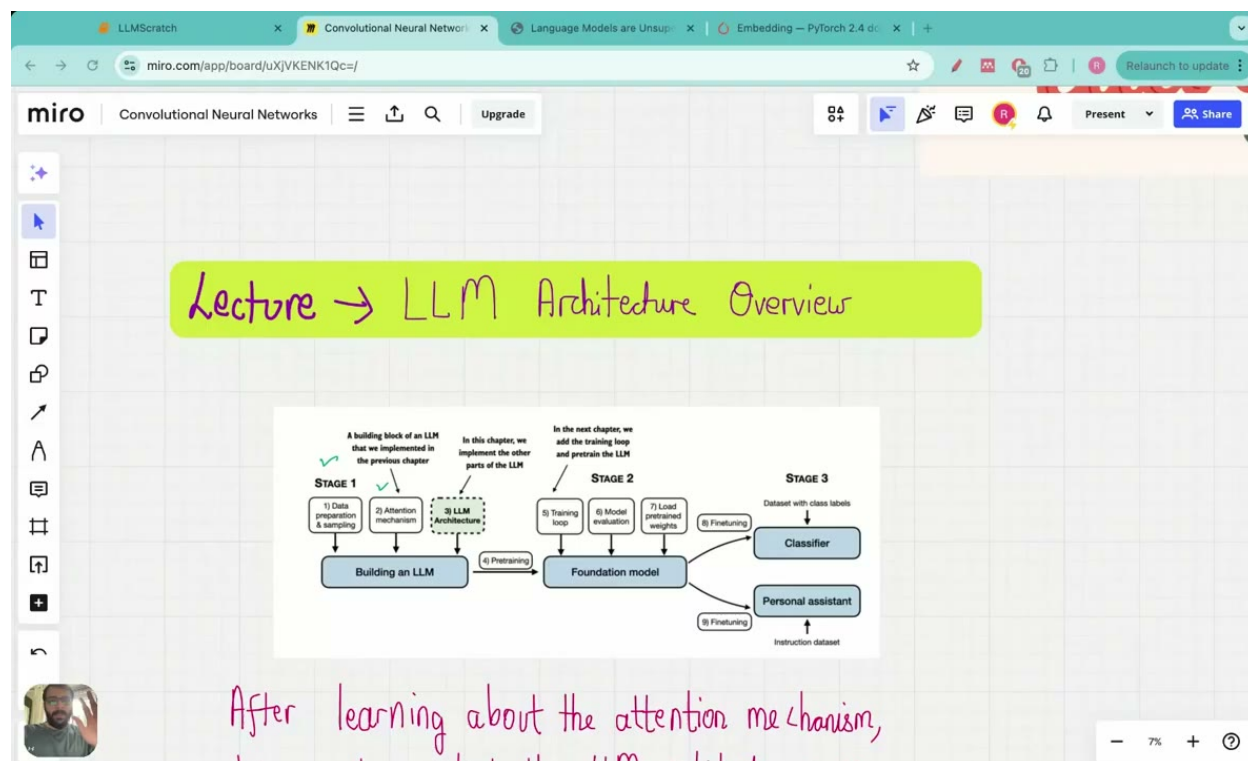


Figure 9.1: A 4×4 attention score matrix with the upper triangle (above the diagonal) shaded in grey to indicate masked positions in causal attention.

zero, while a lower triangular matrix has all elements above the diagonal set to zero. The causal mask is upper-triangular, covering exactly the positions where token $j > i$ —that is, where token j lies in the future relative to token i .

9.2.1 A Simple Lower-Triangular Mask

The most direct approach is to create a lower-triangular matrix of ones and multiply it element-wise with the attention weights:

```
mask_simple = torch.tril(torch.ones(context_length, context_length))
```

After computing raw attention weights (before any normalization), apply the mask:

```
masked_attn_weights = attn_weights * mask_simple
```

This zeros out all upper-triangular positions, but each row no longer sums to one, so renormalization is required:

```
mask_simple_normalized = masked_attn_weights / masked_attn_weights.sum(dim=1, keepdim=True)
```

Intuitively this works, but it has a subtle flaw.

9.2.2 Why You Must Mask Before Softmax

Applying softmax first and then zeroing out future positions creates a data leakage problem. The softmax has already “seen” the future token scores when computing the exponentials and the

normalization denominator; zeroing those positions afterward and renormalizing does not produce the same result as if the future tokens had never participated in the softmax at all.

The correct approach is to apply an upper-triangular mask of negative infinity to the raw attention *scores* before softmax. Because $e^{-\infty} = 0$, those positions contribute nothing to the softmax denominator, and the remaining weights automatically sum to one without any manual renormalization.

The preferred implementation creates an upper-triangular mask (ones above the diagonal) and fills those positions with $-\infty$:

```
mask = torch.triu(torch.ones(seq_len, seq_len), diagonal=1)

attention_scores = attention_scores.masked_fill(mask == 1, float('-inf'))
```

Softmax is then applied in the usual way:

```
attention_weights = torch.softmax(attention_scores / sqrt(d_k), dim=-1)
```

This causal masking mechanism prevents information leakage from future tokens while ensuring that every row of attention weights sums to one.

9.3 Adding Dropout to Attention Weights

Regularization is important in large models. Dropout is a deep learning technique where neurons in different layers are randomly switched to zero during training. In causal attention specifically, dropout randomly zeroes out attention weights while preserving the causal masking of future tokens.

For example, `torch.nn.Dropout(p=0.5)` creates a dropout layer that randomly zeroes elements with probability 0.5 and scales the remaining elements by $1/(1 - 0.5) = 2$. This scaling ensures the expected value of the output is unchanged, so the same model can be used at inference time without adjustment.

9.4 The CausalSelfAttention Class

With masking and dropout in hand, we can write a complete causal self-attention class. The `CausalSelfAttention` class incorporates causal attention and dropout into the self-attention mechanism, with the same structure as the base self-attention class except for the masking and dropout additions.

The input has shape (batch_size, num_tokens, input_dimensions), and the weight matrices W_Q , W_K , and W_V are trainable parameters learned during LLM training. The core of the forward pass—computing scores, masking, softmax, dropout, and context vectors—looks like this:

```
attn_scores = queries @ keys.transpose(1, 2)
attn_scores.masked_fill_(self.mask.bool()[ :num_tokens, :num_tokens], -torch.inf)
attn_weights = torch.softmax(attn_scores / keys.shape[-1]**0.5, dim=-1)
attn_weights = self.dropout(attn_weights)
context_vec = attn_weights @ values
```

Notice the mask slicing `[:num_tokens, :num_tokens]`: it ensures the causal mask is applied only up to the number of tokens in the batch, handling cases where the batch is shorter than the maximum supported context size.

An input embedding is a vector representation of a token that encodes semantic meaning but does not carry information about how other words in the sentence relate to it. The context vectors produced by causal attention remedy this—each one aggregates information from all *previous* tokens (and the token itself), weighted by learned attention scores.

9.5 From Single-Head to Multi-Head Attention

A single causal attention head produces one set of context vectors. Multi-head attention divides the attention mechanism into multiple heads, each operating independently, and is called “multi-head” because it aggregates the outputs of those independent heads.

The motivation is representational richness: different heads can learn to attend to different aspects of the input, and their outputs are combined to form a richer context representation.

9.5.1 Naive Wrapper Implementation

The straightforward approach is to create multiple instances of the causal self-attention mechanism, each with its own weights, and concatenate their outputs. In the forward method, `torch.cat` with `dimension=-1` concatenates outputs along the column dimension. A causal attention module is applied to compute context vectors for each head separately, and the results are then concatenated, with the output dimension D_{out} of each head chosen so that the concatenated result has the desired total dimensionality.

While conceptually clean, this approach is less efficient because it requires a separate matrix multiplication for each head at the start of the forward pass.

9.6 The Efficient Weight-Split Implementation

The production-grade approach—used in GPT—performs a single large matrix multiplication and then *splits* the result across heads. This is more computationally efficient than maintaining separate classes for a multi-head attention wrapper and causal attention.

The key insight is that instead of h separate small weight matrices, a single weight matrix of dimension $d_{in} \times d_{out}$ is used, where:

$$D_{out} = \text{head_dimension} \times \text{number_of_heads}$$

Equivalently:

$$\text{head_dim} = \frac{d_{out}}{\text{num_heads}}$$

The head dimension is the dimension of the representation within each individual attention head. For a concrete example, with $d_{out} = 6$ and $num_heads = 2$:

$$head_dim = d_{out}/num_heads = 6/2 = 3$$

The number of attention heads is the count of parallel attention mechanisms that operate on the output dimension.

9.6.1 Initializing the Layers

The `__init__` method creates three linear projections—one each for queries, keys, and values—plus an output projection, a dropout layer, and a causal mask buffer:

```
self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)

self.out_proj = nn.Linear(d_out, d_out)
self.dropout = nn.Dropout(dropout)
self.register_buffer('mask', torch.triu(torch.ones(context_length, context_length))
```

The mask is registered as a buffer rather than a parameter so that it moves to the correct device automatically but is not updated by the optimizer.

9.6.2 The Forward Method: Step by Step

The forward method takes an input tensor with three dimensions: batch size, number of tokens, and input dimension D_{in} . D_{in} represents the dimensionality of the vector embedding for each token.

Step 1 — Project to queries, keys, and values.

```
keys = self.W_key(x)  # Shape: (b, num_tokens, d_out)
queries = self.W_query(x)
values = self.W_value(x)
```

In the resulting matrices of shape $(batch_size, num_tokens, d_out)$, each row corresponds to one token represented as a d_{out} -dimensional vector.

Step 2 — Reshape to expose the head dimension.

The last dimension (d_{out}) is unrolled into the number of heads and the head dimension, using the relationship $d_{out} = head_dimension \times num_heads$:

```
keys = keys.view(b, num_tokens, self.num_heads, self.head_dim)
queries = queries.view(b, num_tokens, self.num_heads, self.head_dim)
values = values.view(b, num_tokens, self.num_heads, self.head_dim)
```

Each tensor now has shape $(batch_size, num_tokens, num_heads, head_dim)$.

Step 3 — Transpose to bring the head dimension before the token dimension.

```
keys = keys.transpose(1, 2)
queries = queries.transpose(1, 2)
values = values.transpose(1, 2)
```

After transposing, the shape becomes (batch_size, num_heads, num_tokens, head_dim). This layout lets PyTorch’s batch matrix multiplication operate independently on each head.

Step 4 — Compute attention scores.

Attention scores are computed by multiplying queries with the transpose of keys along dimensions 2 and 3:

```
attn_scores = queries @ keys.transpose(2, 3)  # Compute scaled dot-product attention
```

Each element in an attention score matrix row encodes how much a particular token relates to the query token—that is, how much importance should be paid to it when processing the query.

Step 5 — Apply the causal mask.

```
mask_bool = self.mask.bool()[ :num_tokens, :num_tokens]
attn_scores.masked_fill_(mask_bool, -torch.inf)
```

The causal attention mechanism ensures that only attention scores with tokens that come before (and including) the current token survive; all scores involving future tokens are zeroed out.

Step 6 — Scale, softmax, and dropout.

```
attn_weights = torch.softmax(attn_scores / keys.shape[-1]**0.5, dim=-1)
attn_weights = self.dropout(attn_weights)
```

Attention weights differ from attention scores in that each row of attention weights sums to one, whereas attention scores do not have this property.

Step 7 — Compute context vectors and project.

The context vector matrix is computed by multiplying attention weights with the values matrix. Each row of the result represents the context vector for a particular token, with dimensions equal to head_dim:

```
context_vec = (attn_weights @ values).transpose(1, 2)
context_vec = context_vec.contiguous().view(b, num_tokens, self.d_out)
context_vec = self.out_proj(context_vec)
```

The `.transpose(1, 2)` swaps the head and token dimensions back, and `.view(b, num_tokens, self.d_out)` concatenates the head outputs by merging the last two dimensions. The output projection `self.out_proj` then mixes information across heads with a learned linear transformation. The resulting d_{out} is the dimension of the context embedding vector produced for each token.

9.7 A Working Example

To verify the implementation, consider a batch of two sequences, each with three tokens of embedding dimension 6:

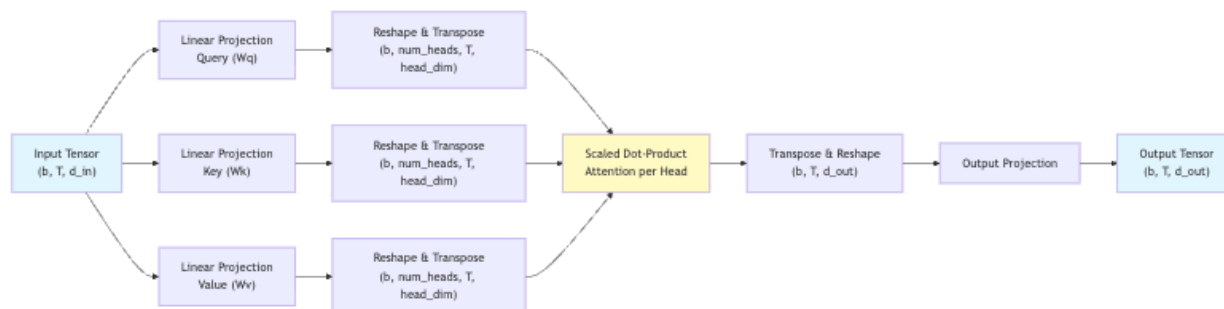


Figure 9.2: Multi-head attention architecture: input tensor flows through three parallel linear projections (query, key, value), reshaped and transposed for per-head scaled dot-product attention, then recombined and projected to output.

```
x = torch.tensor([[[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0],
                    [6.0, 5.0, 4.0, 3.0, 2.0, 1.0],
                    [1.0, 1.0, 1.0, 1.0, 1.0, 1.0]]]])
```

We then instantiate the multi-head attention module and run a forward pass:

```
batch_size, context_length, d_in = batch.shape
d_out = 6
mha = MultiHeadAttention(d_in, d_out, context_length, 0.0, num_heads=2)
context_vecs = mha(batch)
print(context_vecs.shape)
```

With $d_{\text{out}} = 6$ and $\text{num_heads} = 2$, the head dimension is $6/2 = 3$. The output shape is $(\text{batch_size}, \text{num_tokens}, 6)$, matching the input embedding dimension—a common design choice that makes it straightforward to stack transformer blocks.

9.8 Comparing the Two Implementations

Property	Wrapper (separate heads)	Weight-split (efficient)
Number of QKV projections	One per head	One shared projection
Matrix multiplications at start	h separate	1 large
Conceptual clarity	High	Moderate
Computational efficiency	Lower	Higher

The weight-split implementation is what you will find in production GPT code. The wrapper is useful for building intuition, but the efficient version is what we carry forward into the full model.

9.9 Summary

This chapter traced the full path from unrestricted self-attention to the efficient multi-head causal attention used in GPT:

1. **Causal masking:** Future token positions are filled with $-\infty$ before softmax, preventing data leakage and ensuring attention weight rows sum to one.
2. **Mask geometry:** An upper-triangular mask created with `torch.triu (... , diagonal=1)` identifies future positions.
3. **Dropout:** Applied to attention weights after softmax to regularize training.
4. **Multi-head attention:** Multiple independent attention heads are implemented efficiently by projecting to a large d_{out} , reshaping to expose the head dimension, running batched attention, and reshaping back.
5. **Output projection:** A final linear layer mixes information across heads.

The attention series covered simplified self-attention, self-attention, causal attention, and multi-head attention. With multi-head causal attention complete, we have the core computational primitive of the transformer. The next step is to embed this module inside a full transformer block, add feed-forward layers and layer normalization, and stack those blocks into a complete GPT model.

Chapter 10

Building the Transformer Block: LayerNorm, GELU, Feed-Forward, and Shortcut Connections

The transformer block is the fundamental building block of GPT and other large language model architectures. Before we can assemble it, we need to understand and implement each of its sub-components individually. Let's start from the bottom up.

10.1 Layer Normalization

10.1.1 The Problem: Internal Covariate Shift

At each training step, the weights of every layer change, which means the input distribution seen by each subsequent layer also shifts. This phenomenon is called **internal covariate shift**: the problem where the input distribution to each layer changes across training iterations, making weight updates difficult and delaying convergence.

10.1.2 The Math

Layer normalization normalizes each sample in a batch independently by subtracting the mean and dividing by the square root of variance computed across the features for that sample. This is the key distinction from batch normalization, which performs normalization for an entire batch. Layer normalization normalizes along the feature dimension (columns), while batch normalization normalizes along the batch dimension (rows).

The mean is computed as:

$$\mu = (X_1 + X_2 + X_3 + X_4)/4$$

The variance is:

$$\sigma^2 = \frac{1}{4} [(X_1 - \mu)^2 + (X_2 - \mu)^2 + (X_3 - \mu)^2 + (X_4 - \mu)^2]$$

And the normalized value for each feature is:

$$\hat{X} = \frac{X - \mu}{\sqrt{\sigma^2}}$$

The result is that layer normalization normalizes the output of every layer based on the mean and standard deviation, so that the mean becomes 0 and the variance becomes 1.

10.1.3 A Quick Empirical Check

The following code creates a two-sample batch, passes it through a linear layer and ReLU, and prints the output so we can inspect its statistics:

```
torch.manual_seed(123)
batch_example = torch.randn(2, 5)
layer = nn.Sequential(nn.Linear(5, 6), nn.ReLU())
out = layer(batch_example)
print(out)
```

With the output tensor in hand, we can compute the mean and variance manually to mirror the formulas above. The mean across the feature dimension is:

```
mean = output.mean(dim=-1, keepdim=True)
```

The variance is:

```
variance = output.var(dim=-1, keepdim=True)
```

And the normalized output is:

```
normalized_output = (output - mean) / torch.sqrt(variance)
```

Notice that `dim=-1` operates along the last dimension—the feature dimension—which is exactly what layer normalization requires.

10.1.4 Scale and Shift: Trainable Parameters

To give the model the ability to undo normalization when useful, layer normalization introduces two trainable parameters: **scale** and **shift**. Both have the same dimension as the input and are applied to the normalized output. These parameters are learned during training, so the network can decide for itself how much normalization to apply.

10.1.5 A Note on Bessel's Correction

When computing variance in PyTorch, you will notice the flag `unbiased=False` in the implementation below. This relates to **Bessel's correction**, which is the use of $n - 1$ instead of n in the formula for sample variance. For layer normalization we normalize over the full set of features for a given sample—not a statistical sample drawn from a larger population—so we use n in the denominator, hence `unbiased=False`.

10.1.6 The LayerNorm Class

Layer normalization is a class that takes the output of a layer and applies normalization to it. Its forward method takes input with a certain number of rows and embedding-dimension columns:

Source:

```
class LayerNorm(nn.Module):
    def __init__(self, emb_dim):
        super().__init__()
        self.eps = 1e-5
        self.scale = nn.Parameter(torch.ones(emb_dim))
        self.shift = nn.Parameter(torch.zeros(emb_dim))

    def forward(self, x):
        mean = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)
        norm_x = (x - mean) / torch.sqrt(var + self.eps)
        return self.scale * norm_x + self.shift
```

A few things to note:

- `self.eps = 1e-5` is a small epsilon value added inside the square root to prevent division by zero.
- `self.scale` is initialized to ones and `self.shift` to zeros, so at initialization the layer acts as an identity transformation; the network learns to deviate from this as needed.
- The normalization formula $y_{\text{normalized}} = (y - \mu) / \sqrt{\sigma^2}$ is applied independently to each sample in the batch.

10.2 GELU: The Activation Function of Choice

10.2.1 Why Not ReLU?

The ReLU activation function is simple and effective: it returns the input value for $x > 0$ and returns zero for $x < 0$. Its main weakness is the **dead neuron problem**, which occurs when a neuron's output becomes negative, ReLU sets it to zero, and the neuron stops contributing to the learning process entirely. In a large model with millions of neurons, a significant fraction can die during training, wasting capacity.

10.2.2 GELU: A Smooth Alternative

GELU (Gaussian Error Linear Unit) is a smooth activation function that is differentiable at $x = 0$, unlike ReLU. Its exact definition is:

$$\text{GELU}(x) = x \cdot \Phi(x)$$

where $\Phi(x)$ is the cumulative distribution function of the standard Gaussian distribution. Because $\Phi(x)$ involves the error function, which has no closed form, a practical approximation is used in GPT-2:

$$\text{GELU}(x) \approx 0.5 \cdot x \cdot \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right)\right)$$

The screenshot shows a Miro board with a diagram of a neural network. The diagram includes an input layer, two hidden layers (labeled 'layer 2' and 'hidden layer 3'), and an output layer. A handwritten note 'gradient 4' is next to the input layer. A yellow sticky note at the top center reads 'Lecture → Shortcut Connections'. Below it, handwritten text says 'Also known as skip or residual connections'. To the right, a blue sticky note contains a numbered list: '① Shortcut connections were first proposed in the field of Computer Vision to solve the problem of vanishing gradients'. Below this, more handwritten text says 'Gradients become progressively smaller as they propagate backward → making'. In the bottom left corner, there is a small video feed of a man with a beard and glasses. The Miro interface includes a toolbar on the left and a top navigation bar with tabs for 'LLMScratch', 'Convolutional Neural Networks', and '1712.09913'.

, highlighting GELU's smooth, non-zero gradient for negative x values and the hard zero of ReLU for $x < 0$]

10.2.3 The GELU Class

The PyTorch implementation wraps this approximation in a module:

```
class GELU(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, x):
        return 0.5 * x * (1 + torch.tanh(
            torch.sqrt(torch.tensor(2.0 / torch.pi)) *
            (x + 0.044715 * torch.pow(x, 3))
        ))
```

This matches the GPT-2 approximation formula exactly.

10.3 The Feed-Forward Network

10.3.1 Purpose and Architecture

The feed-forward neural network sub-module is a core component of the LLM transformer block. Within the block, it processes each element of the sequence separately, without considering relationships with other elements.

The network follows an **expansion–contraction** design: an expansion layer followed by a compression layer, with input and output dimensions preserved. Specifically, the architecture consists of three layers:

1. An **expansion layer** that increases the dimension to $4\times$ the embedding dimension.
2. A **GELU activation function**.
3. A **contraction layer** that reduces back to the original embedding dimension.

For GPT-2’s embedding dimension of 768, the first linear layer projects from 768 to 3072 dimensions and the second projects from 3072 back to 768. The GELU activation sits between these two linear layers.

10.3.2 Why Expand and Then Contract?

The expansion-contraction architecture preserves input dimensionality while allowing richer exploration of the feature space. By temporarily lifting the representation into a higher-dimensional space, the network can learn more complex, non-linear relationships between features; the expansion and contraction together allow exploration of a richer parameter space and help LLMs learn better relationships between parameters. The contraction then brings the representation back to the original dimension so it can be combined with the residual connection and passed to the next block.

10.3.3 Implementation

`nn.Sequential` is a PyTorch module that chains neural network layers together by forwarding outputs sequentially to inputs of subsequent modules. The feed-forward network processes input through expansion, GELU activation, and contraction, with output dimensions matching the input dimensions. The `FF` object is an instance of the feed-forward class that takes the embedding dimension from configuration and creates layers with GELU activation, initializing weights randomly.

10.4 Shortcut Connections and the Vanishing Gradient Problem

10.4.1 The Vanishing Gradient Problem

The **vanishing gradient problem** occurs when gradients become extremely small or approach zero during backpropagation through deep layers, preventing weight updates and causing training stagnancy. Shortcut connections are the primary mechanism by which transformers solve this problem.

10.4.2 What Are Shortcut Connections?

Shortcut connections—also known as skip connections or residual connections—are achieved by adding the output of one layer to the output of a later layer. In a diagram of the transformer block, they are represented by plus symbols with associated arrows that bypass the linear flow of data.

By creating an alternative path for gradient flow, shortcut connections ensure that gradients no longer have to travel exclusively through the sequence of transformations—they can bypass one or more layers entirely. Shortcut connections can be added between any layers of the transformer block, and they solve the vanishing gradient problem by providing these alternative paths.

10.4.3 The Math Behind Shortcut Connections

Consider a residual block where the output of layer $l + 1$ is:

$$y_{l+1} = f(y_l) + y_l$$

Using the chain rule, the partial derivative of the loss with respect to y_l is:

$$\frac{\partial L}{\partial y_l} = \frac{\partial L}{\partial y_{l+1}} \cdot \frac{\partial y_{l+1}}{\partial y_l}$$

Because $y_{l+1} = f(y_l) + y_l$, the inner derivative becomes:

$$\frac{\partial y_{l+1}}{\partial y_l} = \frac{\partial f(y_l)}{\partial y_l} + 1$$

The guaranteed $+1$ term means the gradient can never vanish completely through this path. As a result, gradient magnitude remains stable across layers, preventing the vanishing gradient problem.

10.4.4 Implementing Shortcut Connections

After computing a layer's output, the original input is added back:

```
x = x + layer_output
```

To verify that shortcut connections actually stabilize gradients, we can set up a small experiment. The loss function is defined as the squared difference between the model output and the ground truth target:

```
loss = (model(x) - target) ** 2
loss.backward()
```

A deep neural network class for this experiment takes `layer_sizes` as an argument, which specifies the number of neurons in each layer—for example, `[3, 3, 3, 3, 1]` means 5 layers with 3 neurons each and a final layer with 1 neuron.

10.5 Assembling the Transformer Block

10.5.1 Components

The five components of a transformer block are: masked multi-head attention, layer normalization, dropout, feed-forward neural network, and GELU activation function. Here is the role each plays:

- **Multi-head attention** takes an input matrix X and multiplies it with trainable queries, keys, and values matrices to produce context vectors. The self-attention mechanism analyzes the relationship between input elements and assigns attention scores based on how each element relates to the others.
- **Layer normalization** appears twice within the transformer block: before the multi-head attention and before the feed-forward neural network. The first instance (`norm_one`) precedes multi-head attention and the second (`norm_two`) precedes the feed-forward network.
- **Dropout** randomly turns off some layer outputs during training to improve generalization and prevent overfitting. The `drop_shortcut` object is a dropout layer from PyTorch (`torch.nn.Dropout`).
- **The feed-forward neural network** processes each element separately without considering relationships with other elements, implemented as the FF object described above.
- **Shortcut connections** are applied after both the attention sub-layer and the feed-forward sub-layer.

In a transformer block, each element of an input sequence is represented by a fixed-size vector equal to the embedding dimension. Both multi-head attention and the feed-forward layers are designed to preserve this dimensionality, which is what makes the residual addition possible—you can only add tensors of the same shape.

10.5.2 Pre-Layer Norm vs. Post-Layer Norm

The original transformer model applied layer normalization *after* the self-attention and feed-forward sub-layers, a pattern called **post-layer norm**. The arrangement implemented here applies `norm_one` before attention and `norm_two` before the feed-forward network.

10.5.3 The Stacking Order

The full transformer block stacking order is:

1. Layer normalization → multi-head attention → dropout → shortcut connection
2. Layer normalization → feed-forward neural network (with GELU) → dropout → shortcut connection

A transformer block class in PyTorch includes a multi-head attention mechanism and a feed-forward neural network, with shortcut connections that add the input of the block to the output of each component. A dropout layer and residual shortcut connection are applied after the feed-forward network to prevent vanishing gradients. The `ATT` object is an instance of the multi-head attention class that takes embedding vectors and converts them into context vectors, with input and output dimensions equal to the embedding dimension.

10.5.4 Putting It All Together

The complete transformer block forward pass follows the stacking order above. For the attention sub-layer:

```
shortcut = x
x = norm_one(x)
x = attention(x)
x = drop_shortcut(x)
x = x + shortcut  # residual connection
```

For the feed-forward sub-layer:

```
shortcut = x
x = norm_two(x)
x = feedforward(x)
x = drop_shortcut(x)
x = x + shortcut  # residual connection
```

10.6 The GPT Architecture in Context

The four sub-components built in this chapter—layer normalization, GELU activation, feed-forward neural network, and shortcut connections—are exactly the four key components of the GPT architecture. Together with the multi-head attention mechanism covered in earlier chapters, they form a complete transformer block. Layer normalization, GELU activation, and feed-forward networks serve as foundational building blocks for transformers, and shortcut connections are essential for making deep transformer stacks trainable.

10.7 Summary

This chapter built four essential sub-components of the transformer block from scratch:

- **Layer normalization** normalizes each sample independently along the feature dimension, unlike batch normalization which normalizes along the batch dimension. Trainable scale and shift parameters let the network recover expressive power after normalization.
- **GELU activation** is a smooth, differentiable alternative to ReLU that avoids the dead neuron problem. Its approximation formula— $0.5 \cdot x \cdot (1 + \tanh(\sqrt{2/\pi} \cdot (x + 0.044715 \cdot x^3)))$ —is the version used in GPT-2.
- **The feed-forward network** expands the representation to $4\times$ the embedding dimension, applies GELU, then contracts back to the original dimension. This allows the model to explore a richer feature space while preserving dimensionality for the residual connection.
- **Shortcut connections** add the input of a sub-layer directly to its output, creating an alternative gradient path. The guaranteed +1 term in the gradient ensures that magnitude remains stable across many layers.

Chapter 11

Assembling and Running the Full GPT-2 Model

11.1 The Big Picture: What the GPT Pipeline Looks Like

The complete pipeline runs as follows: input sentence → token IDs → token embeddings → positional embeddings → input embeddings → dropout → transformer blocks → layer normalization → output head → output tensor. Each stage has a well-defined tensor shape, and the only stage where the embedding dimension changes is the very last one.

The fundamental task the model is solving is next-token prediction. Everything else—the attention heads, the feed-forward layers, the normalization—exists to make that prediction as accurate as possible. The model generates tokens given a sequence of input tokens, and because each newly generated token is fed back in as input, the process is inherently autoregressive.

11.2 The GPT Configuration Object

Throughout this chapter we refer to a configuration dictionary that holds all hyperparameters in one place:

- `vocab_size`: 50257 (byte-pair encoding vocabulary)
- `context_length`: 1024
- `emb_dim`: 768
- `n_heads`: 12
- `n_layers`: 12
- `drop_rate`: 0.1
- `qkv_bias`: False

Passing this dictionary into the model constructor makes it easy to swap configurations without touching the model code itself.

11.3 Building the DummyGPTModel

Source:

```
class DummyGPTModel(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
        self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
        self.drop_emb = nn.Dropout(cfg["drop_rate"])
        self.trf_blocks = nn.Sequential(
            *[DummyTransformerBlock(cfg) for _ in range(cfg["n_layers"])]
        )
        self.final_norm = DummyLayerNorm(cfg["emb_dim"])
        self.out_head = nn.Linear(
            cfg["emb_dim"], cfg["vocab_size"], bias=False
        )

    def forward(self, in_idx):
        batch_size, seq_len = in_idx.shape
        tok_embeds = self.tok_emb(in_idx)
        pos_embeds = self.pos_emb(torch.arange(seq_len, device=in_idx.device))
        x = tok_embeds + pos_embeds
        x = self.drop_emb(x)
        x = self.trf_blocks(x)
        x = self.final_norm(x)
        logits = self.out_head(x)
        return logits
```

Let's walk through each component in turn.

11.4 Component Deep-Dive

11.4.1 Token Embeddings

Token embeddings have dimension 768, so the embedding table has shape (vocab_size, emb_dim) = (50257, 768).

11.4.2 Positional Embeddings

Raw token embeddings carry no information about *where* in the sequence a token appears. A positional embedding is a vector representation of the position of a token within a sequence, and it fills exactly that gap.

11.4.3 Combining the Two

The input embedding for each token is the sum of its token embedding and its positional embedding. Token embeddings are added to positional embeddings in the GPT model forward pass, so every vector entering the transformer simultaneously encodes what the token *is* and where it *sits* in the sequence.

11.4.4 Dropout

Immediately after the embeddings are combined, a dropout layer is applied. Dropout randomly sets some elements of every input embedding to zero, acting as a regularizer during training and preventing the model from becoming overly reliant on any single embedding dimension.

11.4.5 The Transformer Blocks

The transformer block is the key component of the entire GPT model. Twelve such blocks are chained together using `nn.Sequential`, and two sub-components do most of the heavy lifting inside each one:

- **Masked multi-head attention** converts embedding vectors into context vectors that capture both semantic meaning and attention relationships between tokens. A context vector encodes both the semantic meaning of a token and how much attention should be given to every other token in the sequence. Multi-head attention is a key architectural component that enables coherent and meaningful outputs in modern GPT models like GPT-4.
- **Shortcut (residual) connections** add the output of a layer back to its input, providing an alternative gradient-flow path and preventing the vanishing gradient problem.

The preprocessing pipeline feeding into the transformer blocks consists of tokenization, token embedding, positional embedding, input-embedding computation, and dropout. Everything after that pipeline lives inside the stacked transformer blocks.

11.4.6 Layer Normalization

Layer normalization normalizes token embeddings by setting the mean to zero and the variance to one for each token independently. Concretely, it subtracts the mean and divides by the square root of the variance, then applies trainable scale and shift parameters. This normalization is applied at multiple points in the network—including after each transformer block—always operating token-by-token.

11.4.7 The Output Head

The output head is a neural network at the final stage of the GPT model that transforms token embeddings into logits for vocabulary prediction. It is the only stage where the embedding dimension changes—from `(batch_size, num_tokens, 768)` to `(batch_size, num_tokens, 50257)`—and its output, the logits matrix, is produced by passing those token embeddings through this linear layer.

11.5 The Full GPT Model Forward Pass

With all components understood individually, the forward pass reads almost like pseudocode. The forward method takes a batch of input tokens, computes their embeddings, adds positional embeddings, passes the sequence through the transformer blocks, normalizes the final output, and returns logits representing the next token's unnormalized probabilities. The model input is a tensor of shape `(batch_size, sequence_length)` containing token IDs, and the output is logits with shape `(batch_size, sequence_length, vocabulary_size)`, which can be decoded to predict the next token.

Remarkably, the core GPT model implementation can be expressed in approximately 8 lines of code. The complexity lives in the sub-modules, not in the top-level assembly.

11.6 Counting Parameters: 124 M vs. 163 M

You can count the total number of parameters in a model's parameter tensors using `p.numel()`:

```
print(f "Total parameters: {total_params}")
```

A naïve count yields roughly 163 million parameters. Where does the discrepancy with the well-known 124 million figure come from?

Weight tying is a technique where the original GPT-2 architecture reuses the weights from the token embedding layer in its output layer. Both layers have shape (50257, 768), so each holds approximately 38.6 million parameters. When the output layer's parameters are removed from the total—as weight tying effectively does—the count drops to exactly 124 million, matching the original GPT-2 model size.

Weight tying reduces the memory footprint from 163 million to 124 million parameters and also reduces computational complexity. That said, using separate token embedding and output layers yields better training and model performance than weight tying. The assembled model requires 600 megabytes of space on a laptop—well within the reach of modern consumer hardware—and can run inference through the full 124-million-parameter architecture on that same machine.

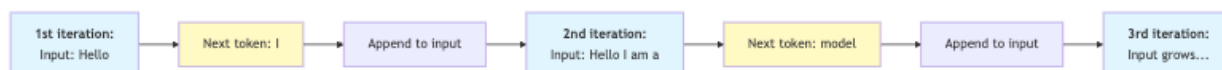
11.7 Generating Text: The Autoregressive Loop

To produce a sequence of tokens, we need a function that repeatedly queries the model, appending each new prediction to the growing input.

11.7.1 The Five-Step Algorithm

Text generation follows five sequential steps: examine the output tensor, extract the last vector, convert logits to probabilities, identify the index of the largest value, and append the resulting token ID to the previous inputs—then repeat until the maximum number of new tokens is reached.

LLM text generation is autoregressive: the input context grows with each iteration as newly generated tokens are appended, and generation continues until the maximum token count is reached. Concretely, the token generated in one iteration becomes part of the input for the next, and the loop runs until the user-specified limit is hit.



→ GPT model → logits → softmax → argmax → new token t_4 → append → $[t_1, t_2, t_3, t_4]$ → repeat]

11.7.2 The `generate_text_simple` Function

`generate_text_simple` takes a model (an instance of the GPT model class), input token indices (`idx`), a maximum number of new tokens to generate, and a context size. Here is how each piece is built up.

If the accumulated sequence grows longer than the model’s context window, it must be cropped before being passed in:

```
# Source:
idx_cond = idx[:, -context_size:]
```

During inference we do not need gradients, so the forward call is wrapped in `torch.no_grad()` to save memory and computation:

```
# Source:
with torch.no_grad():
    logits = model(idx_cond)
```

The model outputs logits for every position in the sequence, but only the *last* position matters for the next-token prediction:

```
# Source:
logits = logits[:, -1, :]
```

This reduces the shape from `(batch_size, seq_len, vocab_size)` to `(batch_size, vocab_size)`. Softmax is then applied to convert those logits into a probability distribution—included here to make the full transformation explicit, even though it is technically redundant for argmax selection:

```
# Source:
probas = torch.softmax(logits, dim=-1)
```

The token with the highest probability is selected greedily:

```
# Source:
idx_next = torch.argmax(probas, dim=-1, keepdim=True)
```

Finally, the new token is appended to the running sequence for the next iteration:

```
# Source:
idx = torch.cat((idx, idx_next), dim=1)
```

11.7.3 Putting It All Together

Assembling the fragments above into a complete function:

```
for _ in range(max_new_tokens):
    # Crop context if needed
    idx_cond = idx[:, -context_size:]

    # Forward pass
    with torch.no_grad():
        logits = model(idx_cond)
```

```

# Focus on the last time step
logits = logits[:, -1, :]

# Convert to probabilities
probas = torch.softmax(logits, dim=-1)

# Greedy selection
idx_next = torch.argmax(probas, dim=-1, keepdim=True)

# Append to running sequence
idx = torch.cat((idx, idx_next), dim=1)

return idx

```

The loop iterates exactly `max_new_tokens` times, producing one new token per iteration.

11.8 Running the Model: A Concrete Example

With the model instantiated and the generation function defined, we can run a quick end-to-end test. A GPT model instance is created by passing in a model configuration object, initialized with random weights. Calling `generate_text_simple` with `max_new_tokens=6` and `context_size=1024` on the input 'hello i am' produces the complete output 'hello i am a model ready to help'.

Of course, the goal is ultimately to convert the output tensor into meaningful next-word predictions. The model's output logits will be converted into tokens and text in subsequent work, and GPT model output has the same number of rows as input tokens but a number of columns equal to the vocabulary size (50257).

11.9 Why the Output Is Nonsense (For Now)

The next-token predictions from an untrained model are essentially random, because the parameters have not yet been optimized. Once training begins, sampling techniques will modify the softmax output so the model does not always select the most likely token, introducing variability and creativity into the generated text. The next phase of work focuses on training the GPT-2 model with 124 million parameters.

11.10 What Was Built from Scratch

The GPT architecture was built entirely from scratch, without relying on external libraries like LangChain. Every sub-module—layer normalization, GELU activation, feed-forward networks, shortcut connections, and transformer blocks—was implemented and coded from the ground up

across six to seven lectures. The result is a working implementation that, given a text input, can predict output tokens.

11.11 Summary

This chapter assembled the complete GPT-2 architecture from its individual components: token and positional embeddings, dropout, 12 stacked transformer blocks, a final layer normalization, and a linear output head. The model has approximately 163 million raw parameters, which reduces to 124 million when weight tying is applied. It fits in 600 MB of RAM and can run inference on a laptop.

The `generate_text_simple` function implements greedy autoregressive decoding: crop the context, run a forward pass, extract the last-position logits, apply softmax, take argmax, append the new token, and repeat. The output logits will be converted into tokens and text in subsequent work, and training the model to produce meaningful outputs is the natural next step.

Chapter 12

Pre-Training: Loss Functions, Training Loop, and Evaluation

This chapter builds each of those three pieces from scratch. We start by understanding exactly what cross-entropy loss means in the context of next-token prediction, then implement helper functions that compute loss over batches and entire data loaders, and finally assemble the full pre-training loop—complete with backpropagation, parameter updates, and periodic evaluation.

12.1 The Loss Landscape: Why Cross-Entropy?

12.1.1 A Taxonomy of Loss Functions

For regression—predicting a continuous value—common choices include:

Mean Bias Error (MBE) simply averages the signed errors:

$$\mathcal{L}_{MBE} = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))$$

Mean Absolute Error (MAE) removes the sign by taking absolute values:

$$\mathcal{L}_{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - f(x_i)|$$

Mean Squared Error (MSE) squares the residuals, penalizing large errors more heavily:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2$$

Root Mean Squared Error (RMSE) restores the original units by taking the square root:

$$\mathcal{L}_{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2}$$

Huber Loss blends MSE and MAE: it behaves quadratically for small errors and linearly for large ones, controlled by a threshold δ :

$$\mathcal{L}_{Huber} = \begin{cases} \frac{(y_i - f(x_i))^2}{2} & \text{if } |y_i - f(x_i)| \leq \delta \\ \delta |y_i - f(x_i)| - \frac{\delta^2}{2} & \text{otherwise} \end{cases}$$

Log-Cosh Loss is a smooth approximation to MAE:

$$\mathcal{L}_{LogCosh} = \frac{1}{N} \sum_{i=1}^N \log(\cosh(f(x_i) - y_i))$$

For classification tasks, the model no longer predicts a scalar; it predicts a probability distribution over discrete classes. The standard choices are:

Binary Cross-Entropy (BCE) for two-class problems:

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Hinge Loss, used in support-vector-style classifiers:

$$\mathcal{L}_{Hinge} = \max(0, 1 - (f(x) \cdot y))$$

Cross-Entropy Loss for multi-class problems:

$$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^M y_{i,c} \log(f(x_i)_c)$$

Kullback-Leibler Divergence measures how one probability distribution diverges from another:

$$\mathcal{L}_{KL} = -\sum_{i=1}^N y_i \cdot \log\left(\frac{y_i}{f(x_i)}\right)$$

12.1.2 Why Language Modeling Uses Cross-Entropy

Cross-entropy loss measures the difference between two probability distributions—specifically, between the predicted logits and the target token indices. The goal of LLM training is to minimize this loss so that the predicted probabilities for target tokens approach one.

Concretely, cross-entropy loss is computed by taking the logarithm of the probability the model assigns to the correct token at each position, averaging those log-probabilities, and negating the result:

$$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N \log(p_i)$$

where p_i are the probabilities at the target indices. This quantity is also called **negative log likelihood (NLL)**—the negative of the logarithm of the target probabilities. As the model becomes less certain or more wrong, the loss grows; the goal of training is to drive it toward zero.

12.2 From Logits to Loss: The Forward Pass in Detail

12.2.1 The Model’s Output: Logits

Large language models are autoregressive models in which input-output pairs are constructed from the text itself without pre-labeling. The transformer block is the main engine of the GPT architecture, transforming input embedding vectors into context vectors—richer representations that encode both the semantic meaning of a token and its relationship to every other token in the sequence. After the transformer blocks, the model produces **logits**: the raw output values before normalization into probabilities. A logit tensor is then converted into a softmax tensor, yielding a probability tensor.

12.2.2 Constructing Input-Output Pairs

Before computing any loss, we need properly structured training data. Targets are the input token sequence shifted by one position, creating input-output pairs for training. More precisely, X represents the input tensor and Y represents the target tensor (the actual desired output values). Input-target pairs are created by pairing each input with its corresponding target output, where the target is the expected output sequence the LLM should learn to predict. In the context of language model training, “target” refers to the true values that the model should predict.

12.2.3 The Dataloader

A dataloader loops over the entire dataset and creates input-output pairs based on a specified context size and stride. Two key parameters govern this process:

- **max_length**: the context size—the number of tokens to consider at once. Context size is the maximum number of tokens an LLM can see before predicting the next token.
- **stride**: how many steps to advance before creating the next input-output pair. Stride is the step size used to move through the dataset when constructing consecutive pairs.

The PyTorch DataLoader makes batch processing convenient, and two additional parameters control its behavior:

- **shuffle**: shuffles the dataset order when batches are created, which is sometimes useful for generalization.
- **drop_last**: drops the last batch if its size is smaller than the specified batch size.

Tiktoken provides a byte pair encoder that operates at the character and sub-word level, making it the natural tokenizer for this pipeline. The `text_to_token_ids` function converts text input into token IDs using the tiktoken tokenizer, and `token_ids_to_text` performs the reverse conversion.

12.3 Computing Cross-Entropy Loss in PyTorch

12.3.1 The Flat-Tensor Pattern

PyTorch's `cross_entropy` function expects a 2-D logit tensor and a 1-D target tensor, so we must flatten both before passing them to the loss function. For a batch of shape (2, 3, 50257):

```
logits_flat = logits.view(-1, 50257)  # Flatten first two dimensions from (2, 3, 50257)
targets_flat = targets.view(-1)       # Flatten from (2, 3) to (6,)
loss = torch.nn.functional.cross_entropy(logits_flat, targets_flat)
```

The same pattern applies in any loss-computation context:

```
torch.nn.functional.cross_entropy(logits_flat, targets_flat)
```

12.3.2 Selecting the Predicted Token

During evaluation, we often want to know which token the model actually predicts. `torch.argmax(input, dim=-1)` returns the indices of the maximum value along the last dimension, effectively picking the highest-logit token at each position.

12.3.3 The `calculate_loss_batch` Function

The `calculate_loss_batch` function computes cross-entropy loss for a single input-target batch pair by flattening both the logits and target tensors before passing them to `nn.functional.cross_entropy`:

```
nn.functional.cross_entropy(flattened_logits, flattened_target_batch)
```

Cross-entropy loss is used to measure the discrepancy between target tokens and output tokens.

12.4 Perplexity: An Interpretable Loss Metric

Raw cross-entropy values can be difficult to interpret in isolation. Perplexity offers a more intuitive alternative: it measures how well the probability distribution predicted by the model matches the actual distribution of words in the dataset. It is calculated as e raised to the loss value:

$$\text{Perplexity} = \exp(\text{loss})$$

As training progresses and the loss decreases, perplexity falls correspondingly.

12.5 The Pre-Training Loop

With loss computation in hand, we can now build the full training loop. The training pipeline for LLMs consists of multiple steps, starting with text generation and text evaluation, and this chapter focuses on training and validation loss computation, building on the cross-entropy loss calculation covered previously.

12.5.1 High-Level Structure

The pre-training loop has two nested loops: an outer loop iterating over epochs and an inner loop iterating over batches in the training dataset. One epoch is a single pass through the entire training set. The training set is divided into batches, and the loop processes one batch per iteration; after all batches are exhausted, the outer loop repeats for the next epoch.

12.5.2 The Core Update Rule

At each iteration, parameters are updated according to:

$$p_{\text{new}} = p_{\text{old}} - \text{step_size} \times \text{loss_gradient}$$

LLM pre-training involves minimizing the loss function to make model outputs as close as possible to the target value tensors, and back propagation is used to calculate the required loss gradients. The algorithm consists of three steps:

1. **Forward pass** — run the model and compute cross-entropy loss.
2. **Backward pass** — compute gradients of the loss with respect to all parameters.
3. **Parameter update** — apply the optimizer step using those gradients.

The backward pass is the most important step because it calculates the gradients of the loss:

```
loss.backward()
```

The pre-training workflow is fully differentiable, allowing back propagation to compute partial derivatives of the loss with respect to all model parameters. In practice, the entire loop is approximately 15 to 20 lines of Python, with `loss.backward()` and `optimizer.step()` forming the core gradient update mechanism.

12.5.3 Iterating Over Batches

The pretraining loop iterates through batches from the training data loader, dividing each batch into an input batch and a target batch. Cross-entropy loss is calculated between these two components for the entire batch at each iteration.

12.5.4 Evaluation During Training

An evaluation step is performed periodically to monitor both training loss and validation loss as the model trains. Computing these losses on an actual dataset also enables backpropagation in the subsequent step.

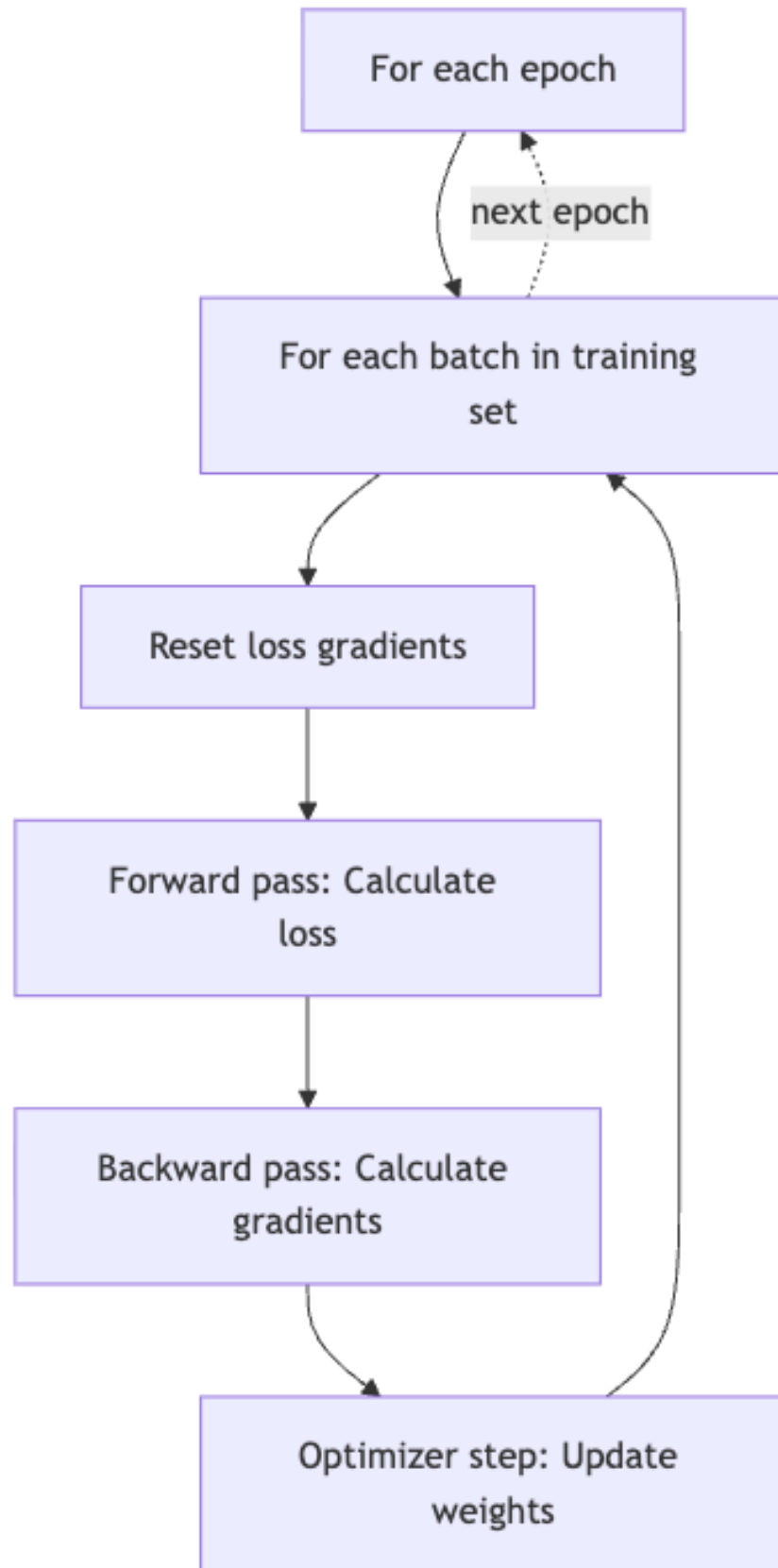


Figure 12.1: Training loop structure showing nested epoch and batch loops with forward pass, loss calculation, backward pass, and optimizer step.

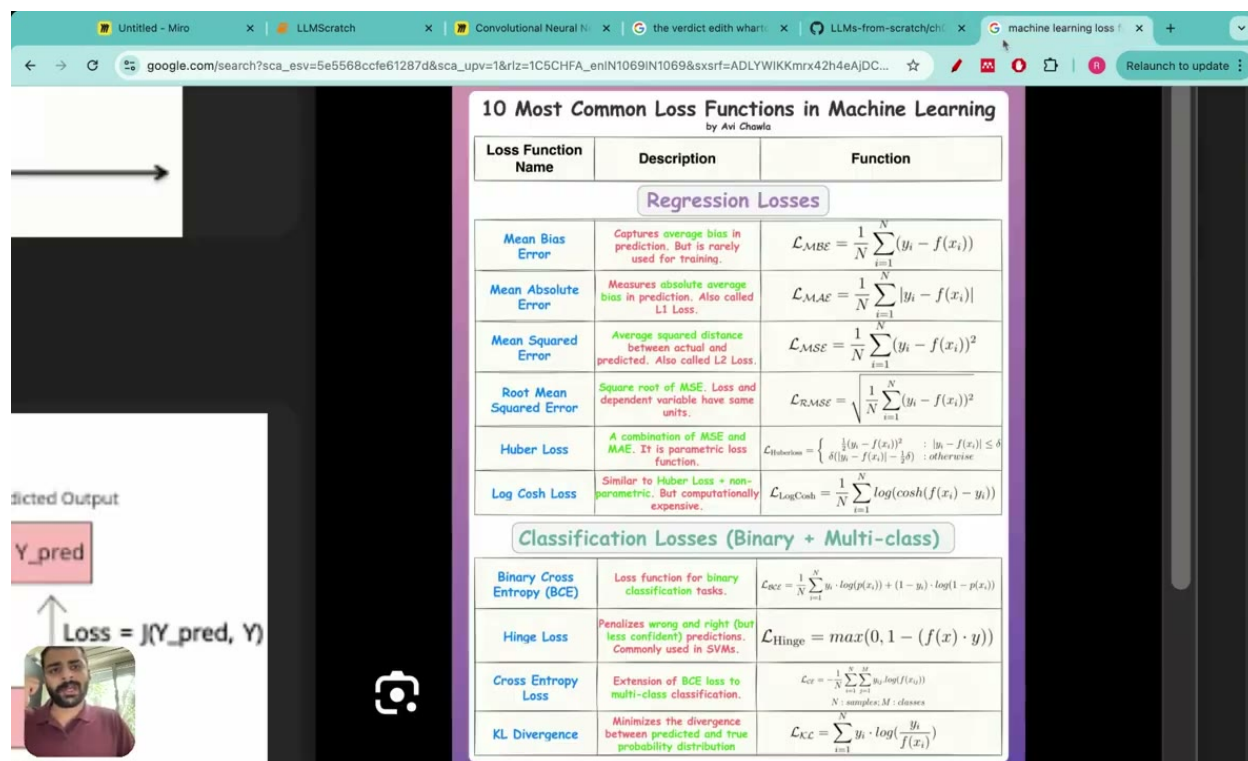


Figure 12.2: Training curve showing training and validation loss decreasing over epochs with evaluation checkpoints marked at regular intervals.

12.5.5 Model Parameters: A Quick Accounting

The key parameter counts for the major components are:

- **Token embedding parameters** = vocabulary size $\times$ embedding dimension
- **Positional embedding parameters** = context size $\times$ embedding dimension
- **Final layer parameters** = embedding dimension $\times$ vocabulary size

Embedding dimension is the size of the vector space into which token embeddings are projected to capture semantic meaning. The number of attention heads specifies how many self-attention mechanism blocks exist within one transformer block.

12.6 Putting It All Together: Training on “The Verdict”

With all the pieces defined, we can now describe a full training run on a real text corpus.

12.6.1 Data Preparation

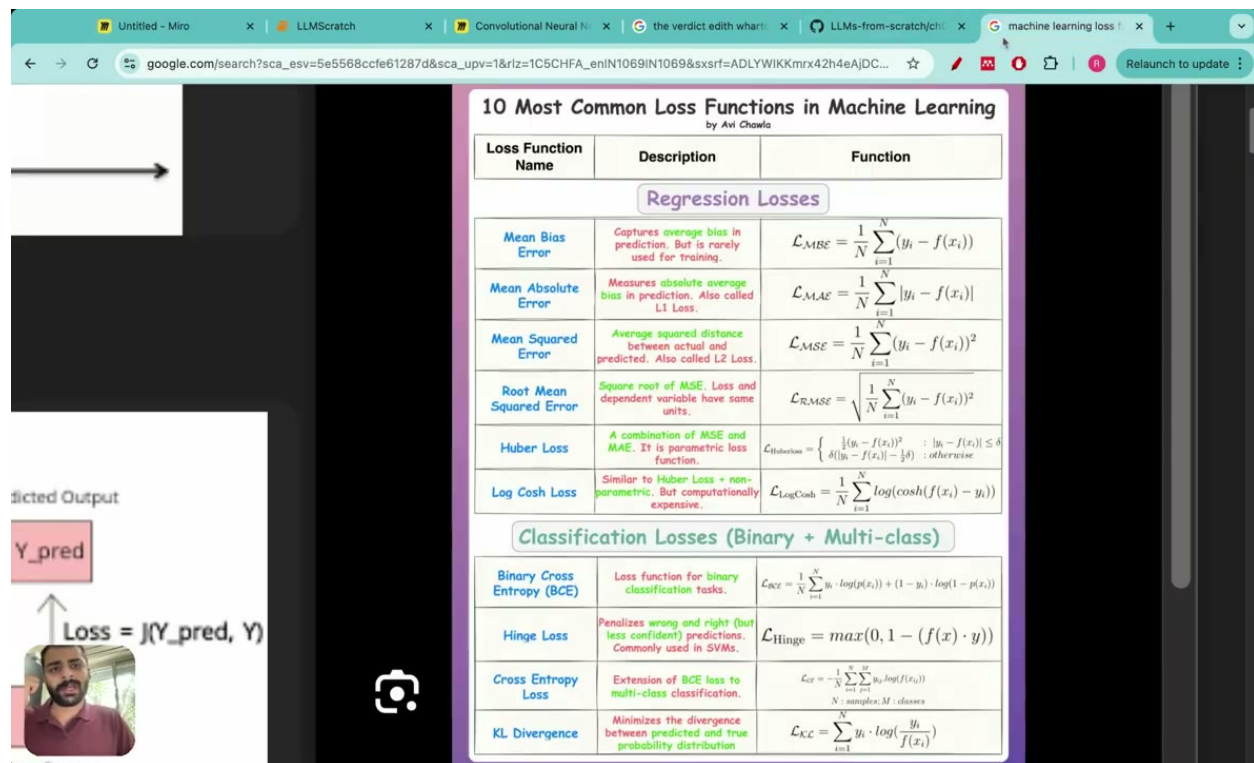
The developed code is generalizable and can be applied to custom datasets for pre-training. With a small corpus like “The Verdict,” setting the stride equal to the context size avoids redundant samples; with larger corpora, overlapping strides can increase the effective dataset size.

12.6.2 The Training Function

An LLM training function is defined to implement backpropagation and minimize training and validation loss. It wraps the nested epoch-batch loop described above, calling `calculate_loss_batch` on each mini-batch, invoking `loss.backward()`, and then stepping the optimizer. A training loop has been implemented where the loss function is minimized and the large language model learns. The next lecture will cover LLM pre-training in full detail.

12.6.3 Decoding Predictions During Training

Temperature scaling is a decoding strategy used to control randomness in model predictions.



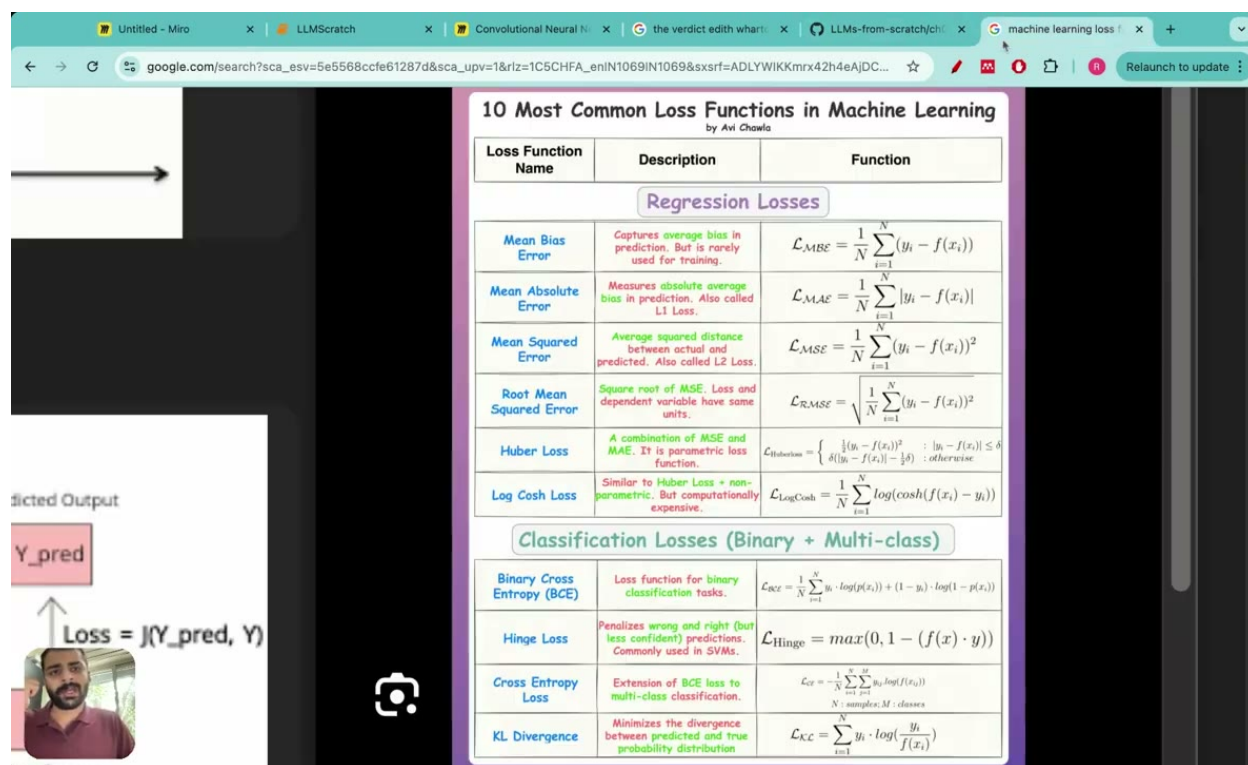
The screenshot shows a web browser with a search result for "10 Most Common Loss Functions in Machine Learning" by Arif Chowdhury. The page is divided into two main sections: "Regression Losses" and "Classification Losses (Binary + Multi-class)". Each section contains a table of loss functions with their names, descriptions, and mathematical formulas.

Loss Function Name	Description	Function
Regression Losses		
Mean Bias Error	Captures average bias in prediction. But is rarely used for training.	$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))$
Mean Absolute Error	Measures absolute average bias in prediction. Also called L1 Loss.	$\mathcal{L}_{MAE} = \frac{1}{N} \sum_{i=1}^N y_i - f(x_i) $
Mean Squared Error	Average squared distance between actual and predicted. Also called L2 Loss.	$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2$
Root Mean Squared Error	Square root of MSE. Loss and dependent variable have same units.	$\mathcal{L}_{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2}$
Huber Loss	A combination of MSE and MAE. It is parametric loss function.	$\mathcal{L}_{Huber} = \begin{cases} \frac{1}{2}(y_i - f(x_i))^2 & : y_i - f(x_i) \leq \delta \\ \delta(y_i - f(x_i) - \frac{1}{2}\delta) & : \text{otherwise} \end{cases}$
Log Cosh Loss	Similar to Huber Loss + non-parametric. But computationally expensive.	$\mathcal{L}_{LogCosh} = \frac{1}{N} \sum_{i=1}^N \log(\cosh(f(x_i) - y_i))$
Classification Losses (Binary + Multi-class)		
Binary Cross Entropy (BCE)	Loss function for binary classification tasks.	$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \log(p(x_i)) + (1 - y_i) \cdot \log(1 - p(x_i))]$
Hinge Loss	Penalizes wrong and right (but less confident) predictions. Commonly used in SVMs.	$\mathcal{L}_{Hinge} = \max(0, 1 - (f(x) \cdot y))$
Cross Entropy Loss	Extension of BCE loss to multi-class classification.	$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M y_{ij} \cdot \log(f(x_{ij}))$ N : samples, M : classes
KL Divergence	Minimizes the divergence between predicted and true probability distribution	$\mathcal{L}_{KL} = \sum_{i=1}^N y_i \cdot \log\left(\frac{y_i}{f(x_i)}\right)$

Figure 12.3: Overview of 10 common loss functions in machine learning, categorized into regression losses (Mean Bias Error, MAE, MSE, RMSE, Huber Loss, Log Cosh Loss) and classification losses (Binary Cross Entropy, Hinge Loss, Cross Entropy Loss, KL Divergence), with descriptions and mathematical formulations.

12.6.4 Observing Overfitting

A well-regularized model trained on a large, diverse corpus will show both curves tracking closely together.



dicted Output

Y_{pred}

Loss = $J(Y_{\text{pred}}, Y)$

10 Most Common Loss Functions in Machine Learning

by Avi Chavla

Loss Function Name	Description	Function
Regression Losses		
Mean Bias Error	Captures average bias in prediction. But is rarely used for training.	$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))$
Mean Absolute Error	Measures absolute average bias in prediction. Also called L1 Loss.	$\mathcal{L}_{MAE} = \frac{1}{N} \sum_{i=1}^N y_i - f(x_i) $
Mean Squared Error	Average squared distance between actual and predicted. Also called L2 Loss.	$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2$
Root Mean Squared Error	Square root of MSE. Loss and dependent variable have same units.	$\mathcal{L}_{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2}$
Huber Loss	A combination of MSE and MAE. It is parametric loss function.	$\mathcal{L}_{Huber} = \begin{cases} \frac{1}{2}(y_i - f(x_i))^2 & : y_i - f(x_i) \leq \delta \\ \delta(y_i - f(x_i) - \frac{1}{2}\delta) & : \text{otherwise} \end{cases}$
Log Cosh Loss	Similar to Huber Loss + non-parametric. But computationally expensive.	$\mathcal{L}_{LogCosh} = \frac{1}{N} \sum_{i=1}^N \log(\cosh(f(x_i) - y_i))$
Classification Losses (Binary + Multi-class)		
Binary Cross Entropy (BCE)	Loss function for binary classification tasks.	$\mathcal{L}_{BCE} = \frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(x_i)) + (1 - y_i) \cdot \log(1 - p(x_i))$
Hinge Loss	Penalizes wrong and right (but less confident) predictions. Commonly used in SVMs.	$\mathcal{L}_{Hinge} = \max(0, 1 - (f(x) \cdot y))$
Cross Entropy Loss	Extension of BCE loss to multi-class classification.	$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M y_{ij} \log(f(x_{ij}))$ $N = \text{samples}, M = \text{classes}$
KL Divergence	Minimizes the divergence between predicted and true probability distribution	$\mathcal{L}_{KL} = \sum_{i=1}^N y_i \cdot \log\left(\frac{y_i}{f(x_i)}\right)$

Figure 12.4: Overview of 10 common loss functions in machine learning, categorized into regression losses (Mean Bias Error, MAE, MSE, RMSE, Huber Loss, Log Cosh Loss) and classification losses (Binary Cross Entropy, Hinge Loss, Cross Entropy Loss, KL Divergence), with mathematical formulas and descriptions.

12.7 Summary

This chapter developed the full machinery needed to pre-train a GPT-style language model from scratch:

1. **Loss functions:** We surveyed the family of regression and classification losses and established why cross-entropy—equivalently, negative log likelihood—is the right choice for next-token prediction. The formula is $-\frac{1}{N} \sum_i \log(p_i)$, and the training goal is to drive this value toward zero.
2. **Perplexity:** Defined as $\exp(\text{loss})$, perplexity provides a more interpretable metric that reflects how well the model’s predicted distribution matches the true word distribution.
3. **Batch loss computation:** The `calculate_loss_batch` function flattens logits from `(batch, seq, vocab)` to `(batch×seq, vocab)` and targets from `(batch, seq)` to `(batch×seq,)` before calling `nn.functional.cross_entropy`.
4. **The training loop:** The loop is nested—outer over epochs, inner over batches—and each iteration performs a forward pass, `loss.backward()`, and an optimizer step. The entire workflow is fully differentiable.
5. **Evaluation:** Training and validation loss are monitored throughout training, and perplexity provides a human-readable summary of model quality at each checkpoint.

Chapter 13

Decoding Strategies, Model Checkpointing, and Loading Pre-Trained GPT-2 Weights

Every time a language model produces a word, it must follow some rule for choosing which token comes next. The choice of that rule — the *decoding strategy* — has an enormous effect on the quality and diversity of the generated text. This chapter covers three complementary topics: the decoding strategies that control text generation, the checkpointing machinery that saves and restores training state, and the process of loading OpenAI’s publicly released GPT-2 weights into a custom PyTorch model.

13.1 Decoding Strategies

13.1.1 Greedy Decoding

The simplest possible strategy is *greedy decoding*: always pick the token with the highest probability. In PyTorch terms, this is just `torch.argmax` applied to the probability vector. While straightforward, greedy decoding causes the model to effectively memorize passages rather than generate fresh text.

13.1.2 Multinomial Sampling

A more flexible approach draws the next token *probabilistically*, selecting it in proportion to its probability score rather than always taking the maximum — a technique called *multinomial sampling*.

The multinomial distribution models k mutually exclusive outcomes, each with a corresponding probability, across n independent trials. Sampling from such a distribution means drawing values where the outcome is not predetermined; each draw follows the probability scores of the outcomes.

PyTorch’s `torch.multinomial` function implements this directly. The following examples illustrate its behavior:

```
# Source:
weights = torch.tensor([0, 10, 3, 0], dtype=torch.float)
torch.multinomial(weights, 2)
# Returns: tensor([1, 2])
```

The weights act as unnormalized probabilities. Attempting to draw more samples than there are non-zero entries without replacement raises an error:

```
# Source:
torch.multinomial(weights, 5) # ERROR without replacement
# RuntimeError: cannot sample n_sample > prob_dist.size(-1) samples without replace
```

Enabling replacement allows repeated draws:

```
# Source:
torch.multinomial(weights, 4, replacement=True)
# Returns: tensor([2, 1, 1, 1])
```

In practice, the `generate` function samples *with replacement* across the vocabulary at each step, drawing exactly one token per step, with each token selected proportional to its probability score.

13.1.3 Temperature Scaling

Multinomial sampling introduces diversity, but you often want to control *how much* diversity. Temperature scaling is a technique used in large language models for precisely this purpose: controlling randomness and diversity in generated text. It works by dividing the logits by a temperature value $T > 0$ before applying softmax to convert them into probabilities. Temperature is therefore a parameter that controls the entropy of the probability distribution over tokens — lower values sharpen the distribution toward the most likely tokens, while higher values flatten it.

Temperature scaling adjusts this sharpness to either focus or spread probability mass, helping prevent overfitting through multinomial sampling. The effect is visible in practice:

```
# Source:
next_token_logits_2 = next_token_logits / 0.1
probabilities = softmax(next_token_logits_2)
```

```
# Source:
next_token_logits_3 = next_token_logits / 5
probabilities = softmax(next_token_logits_3)
```

A low temperature (e.g., 0.1) concentrates probability on the top tokens; a high temperature (e.g., 5) spreads it more evenly, making the model more creative — and more random.

13.1.4 Top-K Sampling

Even with temperature scaling, multinomial sampling can occasionally select a very low-probability token that makes no linguistic sense in context. *Top-k sampling* addresses this by restricting the candidate set before sampling.

Top-k sampling selects only the k tokens with the highest logits for the next token prediction, excluding all others from consideration entirely. This prevents random or low-probability tokens from ever becoming the next token.

The implementation relies on two PyTorch operations. First, find the top- k logit values and their indices:

Source:

```
torch.topk(logits, k=3) # returns the top 3 values and their indices
```

Then replace every logit that did *not* make the top- k cut with negative infinity, so that softmax assigns those positions zero probability:

Source:

```
torch.where(condition, logits, -inf) # replaces non-top-k logits with -inf
```

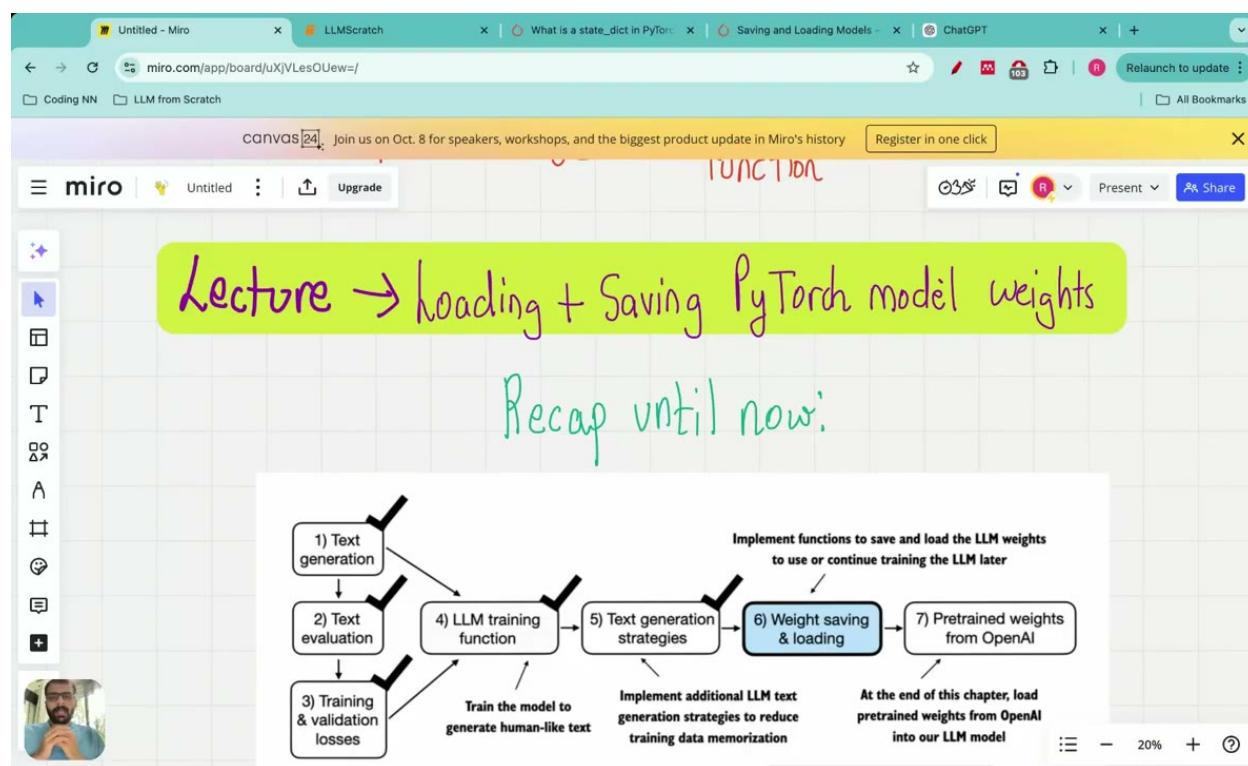


Figure 13.1: Top-K filtering step: logits for all vocabulary tokens are ranked, the top-K values are retained, and all remaining values are replaced with $-\infty$ before softmax normalization.

13.1.5 Combining Top-K Sampling and Temperature Scaling

Top-k sampling and temperature scaling are most powerful when used together. Temperature scaling alone can still occasionally produce nonsense tokens; top-k sampling alone does not control the sharpness of the distribution within the candidate set. Integrating the two ensures both creativity in token selection and prevention of random low-probability tokens from being selected.

The complete decoding workflow is:

1. Obtain the logit tensor from the GPT model.

2. Apply top-k filtering.
3. Replace non-top-k logits with negative infinity.
4. Divide the remaining logits by the temperature T .
5. Apply softmax to obtain probabilities.
6. Sample from the multinomial distribution to select the next token.



Figure 13.2: Six-step decoding pipeline for sampling the next token from GPT logits, including top-k filtering, masking, temperature scaling, softmax normalization, and multinomial sampling.

Combining these strategies reduces overfitting in text generation and prevents the model from memorizing passages when predicting next words. This is precisely why ChatGPT produces different output each time you ask the same question — it relies on decoding strategies like top-k sampling and temperature scaling rather than memorizing user input.

13.1.6 The `generate` Function

The `generate` function encapsulates this entire pipeline. It takes as parameters a GPT model instance, the maximum number of new tokens to generate, the context size, a temperature value, a top-k value, and an end-of-sequence token ID. Internally, it applies operations in sequence: top-k filtering, temperature scaling, softmax, and multinomial sampling. This replaces the naive maximum-probability decoding used in earlier versions, improving both quality and diversity.

A representative call looks like this:

```
# Source:
generate(model, input_token_ids=['every ', 'effort ', 'moves ', 'you '],
         max_new_tokens=25, temperature=1.5, top_k=50)
```

13.2 Model Checkpointing: Saving and Loading Weights

Training a large language model takes significant time and compute, making it essential to save progress periodically. Loading and saving model weights is especially important when dealing with large models, and doing so correctly saves both memory and time. Establishing this checkpointing foundation is also a prerequisite for working with pre-trained weights from OpenAI.

13.2.1 PyTorch Model State

Every `torch.nn.Module` exposes its learnable parameters through `model.parameters()`. For saving and restoring, the more useful interface is `model.state_dict()`, which returns a dictionary mapping each layer to its parameter tensors. The two main PyTorch functions for saving and loading these parameters are `torch.save()` and `model.load_state_dict()`.

13.2.2 Saving the Optimizer State

Saving model parameters alone is not sufficient if you intend to resume training, because the optimizer must also be restored to its previous state. The optimizer state dictionary stores both

hyperparameters (such as learning rate and weight decay) and historical data used by the optimizer (such as past gradient values and squared gradient values). `optimizer.state_dict()` returns this state as a dictionary with two entries: 'state' (holding the current optimization state per parameter) and 'param_groups' (a list of all parameter groups).

It is therefore recommended to save the optimizer state alongside the model parameters when checkpointing. For large language models in particular, losing this state means having to restart training from scratch — a costly outcome.

13.2.3 Restoring from a Checkpoint

When loading a saved checkpoint, `optimizer.load_state_dict()` restores both the optimizer's hyperparameters and its history of gradients and squared gradients. The complete restoration workflow is:

```
# Source:
checkpoint = torch.load('model_and_optimizer.pth')
model = GPTModel()
model.load_state_dict(checkpoint['model'])
optimizer = torch.optim.AdamW(model.parameters())
optimizer.load_state_dict(checkpoint['optimizer'])
model.train()
```

13.3 Loading Pre-Trained GPT-2 Weights

Rather than training a model from scratch, you can load OpenAI's publicly released GPT-2 weights directly into the custom GPT model class and use it for text generation immediately. There is one complication: OpenAI originally saved GPT-2 weights using TensorFlow, while this series uses PyTorch. Pre-processing steps are therefore required after downloading the weights before they can be integrated with the GPT architecture.

13.3.1 Downloading the GPT-2 Files

The `download_and_load_gpt2` function handles the download:

```
# Source:
def download_and_load_gpt2(model_size, models_dir):
    allowed_sizes = ("124M", "355M", "774M", "1558M")
    if model_size not in allowed_sizes:
        raise ValueError(f"Model size not in {allowed_sizes}")
    model_dir = os.path.join(models_dir, model_size)
    base_url = "https://openaipublic.blob.core.windows.net/gpt-2/models"
    filenames = ["checkpoint", "encoder.json", "params.json",
                 "model.ckpt.data-00000-of-00001", "model.ckpt.index",
                 "model.ckpt.meta", "vocab.bpe"]
    os.makedirs(model_dir, exist_ok=True)
    for filename in filenames:
        file_url = os.path.join(base_url, model_size, filename)
```



```
file_path = os.path.join(model_dir, filename)
download_file(file_url, file_path)
```

To download and load the 124M-parameter model:

```
# Source:
```

```
from gpt_download3 import download_and_load_gpt2
```

```
settings, params = download_and_load_gpt2(model_size="124M", models_dir="gpt2")
```

13.3.2 What Gets Downloaded

- **checkpoint** — contains the path where all current parameters and weights of the GPT-2 model are stored.
- **encoder.json** — a vocabulary mapping tokens to their corresponding token IDs.
- **vocab.bpe** — a list of byte-pair encoded token merges, ordered by merge frequency with the highest-probability merges at the top. Byte-pair encoding is a sub-word tokenization scheme that merges the most frequently occurring pairs of tokens into single tokens.
- **hparams.json** — contains all hyperparameter settings and configuration values for the GPT-2 model.
- **model.ckpt.*** — the TensorFlow checkpoint files containing the actual weight tensors.

13.3.3 Understanding the **settings** and **params** Dictionaries

`download_and_load_gpt2` returns two dictionaries. The `settings` dictionary contains the same hyperparameters as `hparams.json`: vocabulary size, context length, embedding dimension, number of attention heads, and number of transformer blocks. Key fields include:

- **Embedding dimension** — the size of the vector space into which each token ID is converted; for GPT-2 small, each token is represented as a 768-dimensional vector.
- **N_heads** — the number of attention heads in each transformer block.
- **N_layers** — the number of transformer blocks in the model.

The `params` dictionary has five top-level keys:

Key	Contents
WTE	Token embedding parameters — converts input token IDs into token embeddings
WPE	Positional embedding parameters — adds positional information to token embeddings
blocks	All trainable weights within the transformer blocks, including attention layers, feed-forward networks, output projections, and layer normalization parameters
G	Final layer normalization scale parameters
B	Final layer normalization shift parameters

One important structural difference from the custom model: in GPT-2, the query, key, and value weight matrices in the attention layer are fused into a single large matrix called `C_ATTN`, whereas

the custom model stores them separately. The weight-loading code must therefore split C_ATTN appropriately. Layer normalization in GPT-2 also includes trainable scale (G) and shift (B) parameters applied after mean subtraction and variance normalization.

13.3.4 GPT-2 Model Variants

OpenAI released GPT-2 in four sizes. The following configuration covers all of them:

```
# Source:
model_configs = {
    "gpt2-small_124M": {"emb_dim": 768, "n_layers": 12, "n_heads": 12},
    "gpt2-medium_355M": {"emb_dim": 1024, "n_layers": 24, "n_heads": 16},
    "gpt2-large_774M": {"emb_dim": 1280, "n_layers": 36, "n_heads": 20},
    "gpt2-xl_1558M": {"emb_dim": 1600, "n_layers": 48, "n_heads": 25},
}

model_name = "gpt2-small_124M"
NEW_CONFIG = GPT_CONFIG_124M.copy()
NEW_CONFIG.update(model_configs[model_name])
```

The goal is to replace the random weight initializations in the GPT model with the downloaded GPT-2 parameters from the params dictionary.

13.3.5 Loading TensorFlow Weights into PyTorch

load_gpt2_params_from_tf_ckpt iterates over every variable in the TensorFlow checkpoint, extracts its array, and places it into the correct position in a nested Python dictionary that mirrors the structure of the params dict:

```
# Source:
def load_gpt2_params_from_tf_ckpt(ckpt_path, settings):
    # Initialize parameters dictionary with empty blocks for each layer
    params = {"blocks": [{for _ in range(settings["n_layer"])]}]

    # Iterate over each variable in the checkpoint
    for name, _ in tf.train.list_variables(ckpt_path):
        # Load the variable and remove singleton dimensions
        variable_array = np.squeeze(tf.train.load_variable(ckpt_path, name))

        # Process the variable name to extract relevant parts
        variable_name_parts = name.split("/")[1:] # Skip the 'model/' prefix

        # Identify the target dictionary for the variable
        target_dict = params
        if variable_name_parts[0].startswith("h"):
            layer_number = int(variable_name_parts[0][1:])
            target_dict = params["blocks"][layer_number]

        # Recursively access or create nested dictionaries
        for key in variable_name_parts[1:-1]:
```

```

target_dict = target_dict.setdefault(key, {})

# Assign the variable array to the last key
last_key = variable_name_parts[-1]
target_dict[last_key] = variable_array

```

13.3.6 Shape Validation During Assignment

To catch mismatches between the downloaded weights and the custom model's parameter shapes, a small helper function validates shapes before assignment:

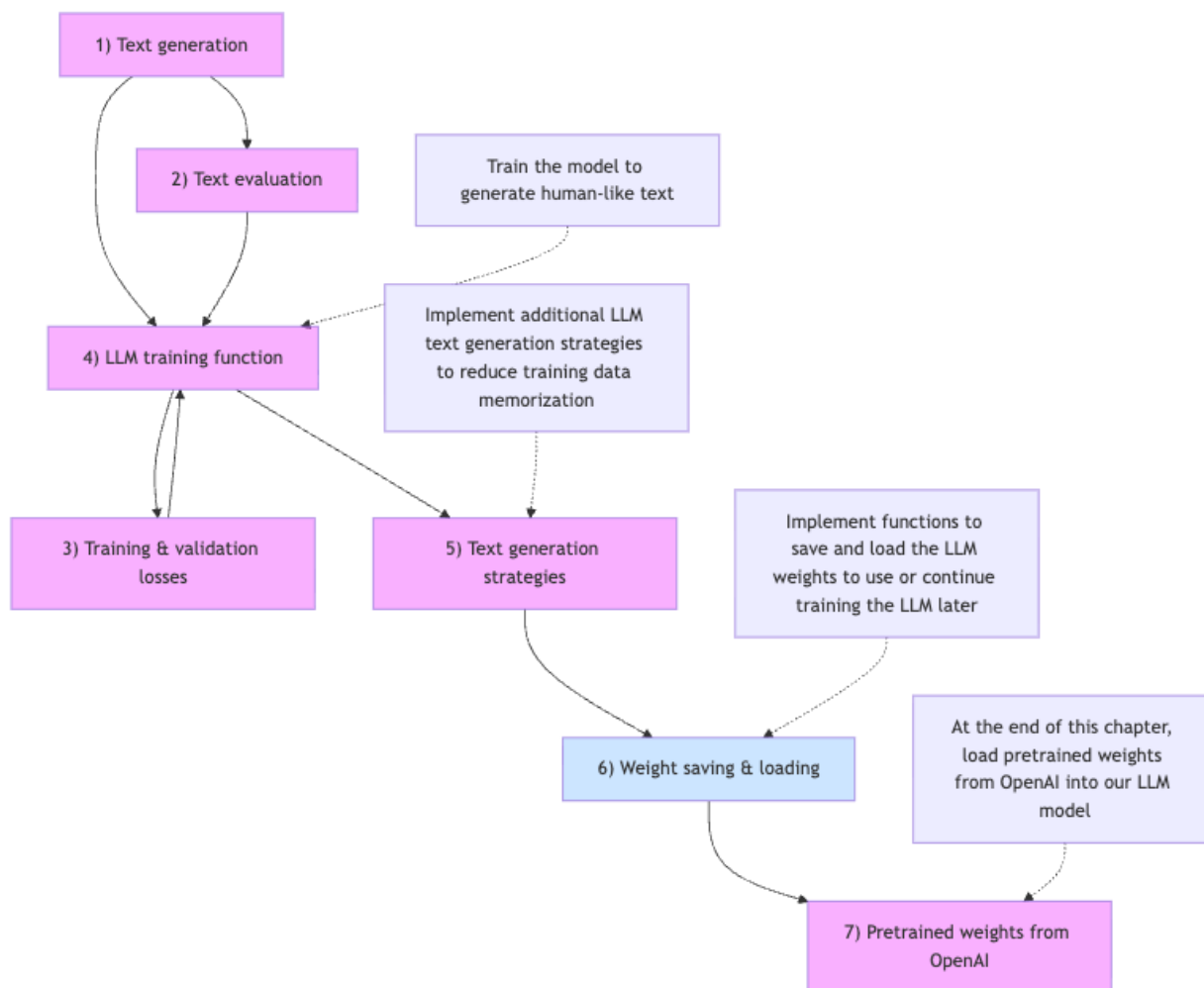
```

# Source:
def assign(left, right):
    if left.shape != right.shape:
        raise ValueError(f"Shape mismatch: {left.shape} vs {right.shape}")
    return right

```

13.3.7 Assigning Feed-Forward Weights

Feed-forward neural network weights and biases for both the fully connected layer and the projection layer are assigned from the downloaded GPT-2 values.



, G, B) to the corresponding layers in the custom PyTorch GPTModel class]

13.3.8 Running Inference with Pre-Trained Weights

The full download-and-load sequence is:

Source:

```
from gpt_download3 import download_and_load_gpt2
```

```
settings, params = download_and_load_gpt2(model_size="124M", models_dir="gpt2")
```

Without GPT-2 weights, the model generates incoherent output. Once the pre-trained weights are loaded, inference is fast because no training is required, and the output is immediately more coherent than that of the untrained model. The next step is to use these pre-trained weights directly rather than training from scratch.

13.4 Why Build Your Own Instead of Using the API?

Owning the model code and the weights opens up possibilities that a hosted API cannot offer. The custom-built GPT model allows exploration of hyperparameter tuning and architectural modifications — changing the number of layers, the embedding dimension, the attention mechanism, the decoding strategy, or any other component — that are simply not possible with ChatGPT.

13.5 Summary

Decoding strategies control how the model selects the next token from its probability distribution. Greedy decoding is simple but leads to repetitive, memorized output. Multinomial sampling introduces diversity by drawing from the full probability distribution. Temperature scaling sharpens or flattens that distribution by dividing logits by a temperature parameter before softmax. Top-k sampling restricts the candidate pool to the k most likely tokens, preventing low-probability nonsense from being selected. Combining the two gives creative yet coherent generation.

Model checkpointing preserves training state so that work is never lost. PyTorch's `model.state_dict()` captures all learnable parameters, and `optimizer.state_dict()` captures the optimizer's hyperparameters and gradient history. Saving both together and restoring them with `model.load_state_dict()` and `optimizer.load_state_dict()` allows training to resume seamlessly.

Loading pre-trained GPT-2 weights bridges the gap between a randomly initialized model and a capable text generator. The weights are stored in TensorFlow format and must be parsed, restructured, and shape-validated before being assigned to the PyTorch model. Once loaded, the model generates coherent text immediately, without any additional training.

Chapter 14

Classification Fine-Tuning: Building a Spam Detector

Fine-tuning is the process of adapting a pre-trained model to a specific task by training it on additional data. Rather than training a language model from scratch—which requires enormous compute and data—we start from a foundation model that has already learned rich representations of language, then nudge its parameters toward a narrower objective.

We will build the entire classification fine-tuning pipeline from scratch. The journey follows three main stages: dataset preparation, model initialization and modification, and fine-tuning with evaluation. Along the way we will implement accuracy and loss utilities, a training loop, and finally run inference on text the model has never seen.

14.1 What Is Classification Fine-Tuning?

LLM fine-tuning has two broad categories: instruction fine-tuning and classification fine-tuning.

Instruction fine-tuning trains the model to follow natural-language instructions. An example prompt might be: *“Is the following text spam? Answer with a yes or no.”*. Because the model must search the entire vocabulary and perform non-specific actions, instruction fine-tuning requires greater computational power than its classification counterpart.

Classification fine-tuning, by contrast, trains the model to classify inputs into predefined categories without being given explicit instructions. In our spam detector, no instruction is given to the LLM; only the input text is provided, and the model outputs a class label—spam or not spam. The trade-off is that classification fine-tuning can only handle a narrow set of prompts because the model is constrained to predefined categories. Beyond spam detection, the same approach applies to tasks like sentiment classification into categories such as angry, sad, or happy, so the pattern generalises well.

14.1.1 12.1.1 The Role of the Foundation Model

A foundation model is a pre-trained large language model trained on large-scale data that can be fine-tuned for downstream tasks. During fine-tuning, the weights, biases, and parameters of this

pre-trained model are updated through training on additional data. More precisely, LLM fine-tuning involves the additional training of a pre-existing model that has already acquired patterns and features from an extensive dataset, now applied to a smaller domain-specific dataset. The result is a fine-tuned large language model tailored to the target task.

In this project we use GPT-2 pre-trained weights and then run the training procedure again on the SMS Spam Collection dataset. Without fine-tuning, the pre-trained GPT model cannot correctly answer spam classification prompts even when given explicit instructions.

14.2 The SMS Spam Collection Dataset

The classification fine-tuning project uses the SMS Spam Collection dataset from the UCI Machine Learning Repository. The task is to predict whether a text message is spam or not spam; the label used to denote non-spam messages in this dataset is **HAM**.

14.2.1 12.2.1 Class Imbalance and Balancing

The original dataset is imbalanced, containing approximately 4,800 non-spam messages and only 747 spam messages. Training on such a skewed distribution can cause a model to simply predict the majority class for every input and still achieve high nominal accuracy, so we balance the dataset before splitting.

14.2.2 12.2.2 Downloading the Dataset

The following code downloads the dataset, unzips it, and renames the file with a .csv extension so it can be read by pandas:

```
# Create an unverified SSL context
ssl_context = ssl.__create_unverified_context()

# Downloading the file
with urllib.request.urlopen(url, context=ssl_context) as response:
    with open(zip_path, "wb") as out_file:
        out_file.write(response.read())

# Unzipping the file
with zipfile.ZipFile(zip_path, "r") as zip_ref:
    zip_ref.extractall(extracted_path)

# Add .csv file extension
original_file_path = Path(extracted_path) / "SMSSpamCollection"
os.rename(original_file_path, data_file_path)
print(f"File downloaded and saved as {data_file_path}")
```

14.2.3 12.2.3 Splitting into Train, Validation, and Test Sets

With the balanced dataset in hand, we split it into three subsets:

```
def random_split(df, train_frac=0.7, validation_frac=0.2):
    train_df = df.sample(frac=train_frac)
    remaining = df.drop(train_df.index)
    validation_df = remaining.sample(frac=validation_frac/(1-train_frac))
    test_df = remaining.drop(validation_df.index)
    return train_df, validation_df, test_df
```

Calling this function on the balanced dataset yields:

```
train_df, validation_df, test_df = random_split(balanced_df)
print(len(train_df))          # 1045
print(len(validation_df))     # 149
print(len(test_df))           # 300
```

The training set contains 1,045 examples, the validation set 149, and the test set 300. The test set is held out entirely until the final stage of the project, when we evaluate the fine-tuned model on data it has never seen before.

14.3 Building PyTorch Datasets and Data Loaders

With the data split in hand, we wrap it in PyTorch abstractions that the training loop can consume. A **Dataset** stores the samples and their corresponding labels; a **DataLoader** wraps an iterable around the Dataset to enable easy access to those samples. More specifically, a PyTorch dataset specifies how data is loaded and processed before a data loader is instantiated.

14.3.1 12.3.1 Tokenization

To feed text into the model we must first convert it to token IDs. We use **TikToken**, a tokenizer library developed by OpenAI that converts sentences into token IDs. TikToken employs a **byte pair encoder**—a sub-word tokenizer that breaks words into smaller units rather than mapping each word to a single token. The resulting **vocabulary** is a dictionary mapping tokens to their corresponding token IDs.

Because text messages in the dataset vary in length, we pad all sequences to the same length so they can be batched efficiently. The **end-of-text token** is a special token used to mark the end of a document or sentence; it also serves as a natural padding token because the model has already encountered it during pre-training and treats it as neutral filler.

14.3.2 12.3.2 The Spam Dataset Class

The spam dataset class performs three steps: identifying the longest sequence in the training dataset, converting each text message into token IDs, and padding all sequences to match that longest length.

Padding to the longest sequence in the *training* set—rather than the longest sequence in each individual batch—ensures that all data loaders produce tensors of the same shape, which simplifies the training loop. The large language model architecture then takes this fixed-length input and outputs a classification of whether the message is spam or not spam.

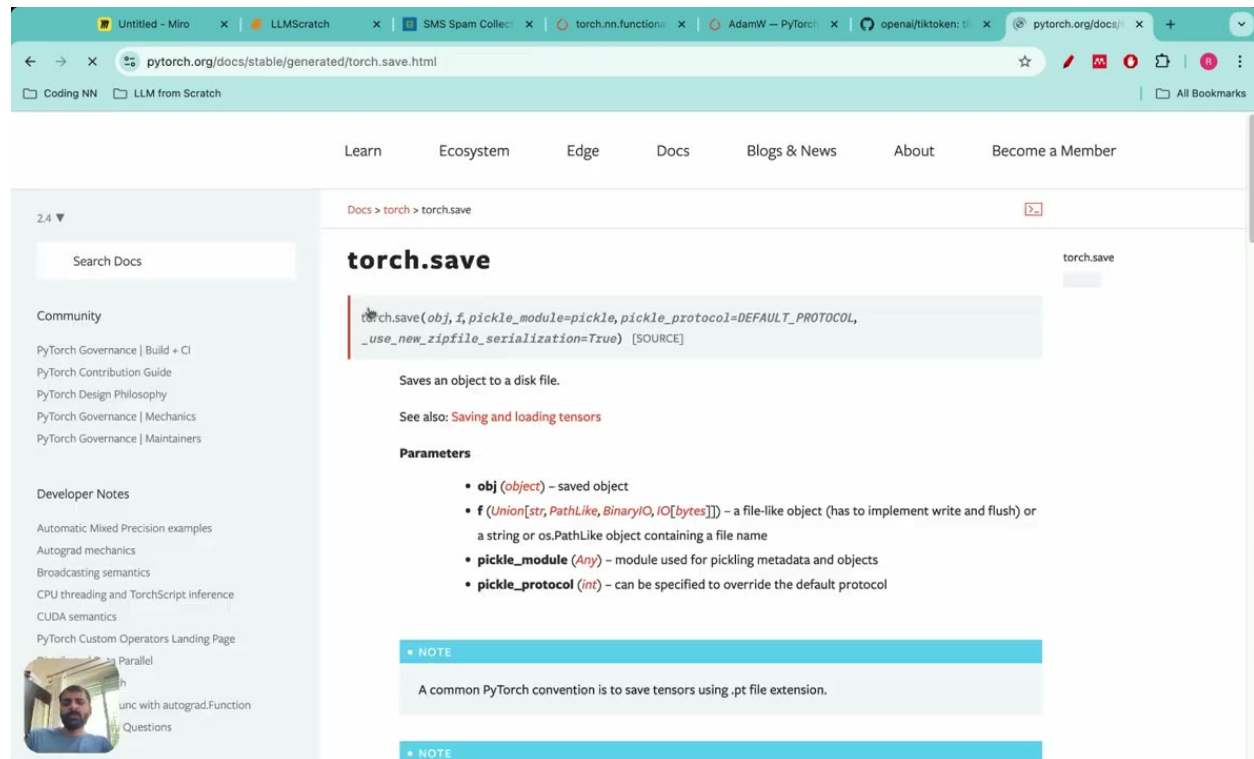


Figure 14.1: Three text messages of varying lengths are tokenized and right-padded with end-of-text tokens to match the longest sequence length.

14.4 Loading Pre-Trained GPT-2 Weights

The project uses a GPT-based LLM architecture that will be modified for classification fine-tuning. Loading pre-trained weights rather than training from random initialization gives the model a strong linguistic foundation before we specialize it for spam detection.

14.4.1 Downloading GPT-2

The `download_and_load_gpt2` function accepts a model size string and a local directory path, then retrieves the corresponding files from OpenAI's public storage:

```
def download_and_load_gpt2(model_size, models_dir):
    # Validate model size
    allowed_sizes = ("124M", "355M", "774M", "1558M")
    if model_size not in allowed_sizes:
        raise ValueError(f"Model size {model_size} not in {allowed_sizes}")
    # Downloads GPT-2 model files from OpenAI public blob storage
```

14.4.2 Loading TensorFlow Checkpoints

OpenAI released GPT-2 weights as TensorFlow checkpoints. The following function loads them into a nested Python dictionary that we can then copy into our PyTorch model:

```
def load_gpt2_params_from_tf_ckpt(ckpt_path, settings):
    # Recursively loads TensorFlow checkpoint variables into nested params dictionary
    # Handles token embeddings, positional embeddings, transformer blocks, and layers
```

14.5 Modifying the GPT Architecture for Classification

With pre-trained weights loaded, we make two targeted changes to the architecture: replacing the output head and selecting which token's representation to classify.

14.5.1 The Classification Head

A **classification head** is a small linear layer added on top of a pre-trained language model to adapt it for classification tasks by mapping hidden states to class logits. To adapt GPT-2 for text classification, we replace the original 768→50257 output layer with a 768→2 linear layer that outputs logits for two classes—spam and not spam. More generally, for any classification fine-tuning task, the original output layer that maps to vocabulary size is replaced with a smaller layer that maps to the number of target classes.

In code, this replacement looks like:

```
model.output_head = Linear(in_features=768, out_features=num_classes)
```

The new classification head takes an input of size 768 and outputs 2 values for spam or not-spam classification.

14.5.2 Which Token's Output Do We Use?

Rather than aggregating all output positions, we focus fine-tuning on the last output token, since it contains all the information accumulated over the full sequence:

```
outputs[:, -1, :]
```

Extracting the last output token from a 4-token sequence yields a vector with two class logits—for example, -3.5898 and 3.9902 for spam/not-spam classification.

14.6 Freezing Layers and Selective Unfreezing

Loading pre-trained weights gives us a strong starting point, but updating every parameter during fine-tuning risks destroying the general language representations the model learned during pre-training—a phenomenon sometimes called catastrophic forgetting. Parameter-efficient fine-tuning addresses this by updating only a subset of parameters while freezing the rest, reducing both the number of trainable parameters and memory requirements.

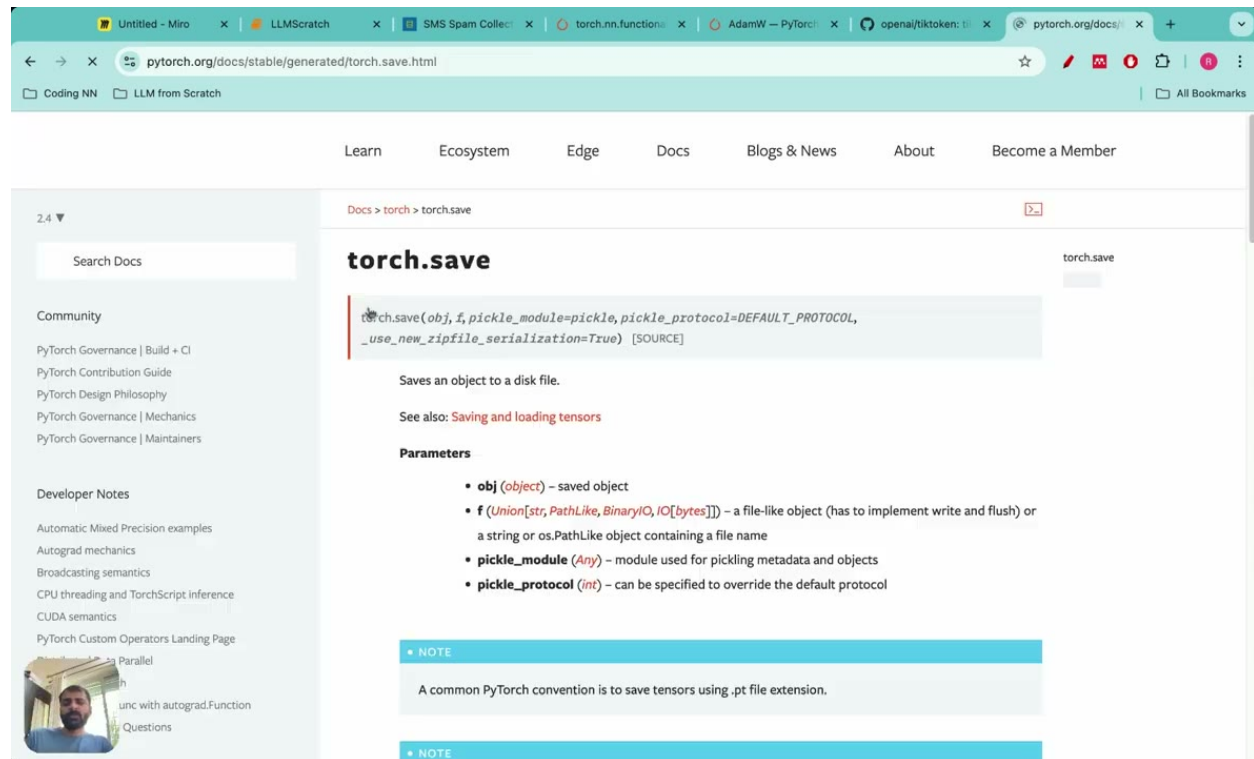


Figure 14.2: Side-by-side comparison of GPT-2 output head (768→50257 vocabulary) versus a new classification head (768→2 classes), with the transformer stack remaining unchanged.

14.6.1 12.6.1 What to Freeze and What to Unfreeze

Three components are fine-tuned: the classification head, the final (12th) transformer block, and the final layer normalization module. All other layers remain frozen throughout training.

Unfreezing the final transformer block:

```
for param in model.transformer.h[-1].parameters():
    param.requires_grad = True
```

Unfreezing the final layer normalization:

```
for param in model.ln_f.parameters():
    param.requires_grad = True
```

Before training begins, the parameters in the new output head, final transformer block, and final layer normalization are still at their initial (random or pre-trained) values and have not been trained on the classification dataset. The training loop will update exactly these parameters while leaving the rest of the network frozen.

14.6.2 12.6.2 Parameter-Efficient Methods: LoRA and QLoRA

Two popular parameter-efficient fine-tuning methods go further than simple layer freezing. **LoRA** fine-tunes two smaller matrices that approximate the larger weight matrix instead of updating all

weights directly. **QLoRA** extends this idea by quantizing the LoRA adapter weights to lower precision, making it even more memory-efficient. Both methods are instances of the broader parameter-efficient fine-tuning paradigm.

14.7 Evaluation Metrics: Accuracy and Loss

Before writing the training loop we need two evaluation utilities: one for accuracy and one for loss.

14.7.1 12.7.1 Classification Accuracy

Classification accuracy measures the percentage of correct predictions across a dataset, computed as:

$$\text{Accuracy} = \frac{\text{correct_predictions}}{\text{total_number_of_examples}}$$

Predicted labels are the class predictions generated by applying `argmax` to model logits; **target labels** represent the true values in the dataset.

The following function iterates over a data loader and accumulates correct predictions:

```
def calculate_accuracy(loader, num_batches=None):
    if num_batches is None:
        num_batches = len(loader)
    else:
        num_batches = min(num_batches, len(loader))
    correct_predictions = 0
    num_examples = 0
    for batch in loader:
        logits = model(batch)
        predicted_labels = argmax(logits)
        target_labels = batch.labels
        correct_predictions += sum(predicted_labels == target_labels)
        num_examples += len(batch)
    accuracy = correct_predictions / num_examples
    return accuracy
```

The `num_batches` parameter allows evaluation on a subset of the data during training, saving computation time.

14.7.2 12.7.2 Cross-Entropy Loss

Categorical cross-entropy loss is defined as the negative sum over all class labels of $y_i \cdot \log(p_i)$, where y_i is the true value and p_i is the predicted probability. In PyTorch, this is computed with `torch.nn.functional.cross_entropy()`.

14.8 The Fine-Tuning Training Loop

With the dataset, model, and metrics in place, we can write the training loop. The model passed to the training function is a GPT model class with a modified architecture that includes a classification head on top.

14.8.1 12.8.1 Epochs and Batches

One epoch is one complete pass through the entire dataset. Training typically runs for several epochs, updating model parameters after each batch via backpropagation.

14.8.2 12.8.2 Evaluation Frequency

Printing metrics after every single batch would flood the console and slow training. Two parameters control evaluation cadence:

- **Evaluation frequency** specifies how many batches must be processed before printing training and validation loss metrics. In our setup, training loss and validation loss are printed every 50 batches using the `evaluate_model` function.
- **Evaluation iteration** specifies the number of batches to use for each evaluation pass; using fewer batches (e.g., 5 or 10) saves computation time.

14.8.3 12.8.3 Per-Epoch Accuracy Reporting

At the end of each epoch, training accuracy and validation accuracy are calculated and printed, giving a higher-level view of learning progress than the per-batch loss alone.

14.8.4 12.8.4 Training Loop Structure

Each epoch proceeds as follows:

1. For each batch, pass the input through the model.
2. Extract the last token's output: `outputs[:, -1, :]`.
3. Compute cross-entropy loss against the target labels using `torch.nn.functional.cross_entropy()`.
4. Backpropagate and update the unfrozen parameters.
5. Every 50 batches, evaluate and print training loss and validation loss.
6. At the end of the epoch, evaluate and print training accuracy and validation accuracy.

The remaining steps after architecture setup are to implement loss and accuracy evaluation utilities, fine-tune the model, and test it on new data.

14.9 Saving and Loading the Fine-Tuned Model

After training, we save the model weights so they can be reloaded for inference or further fine-tuning without repeating the training process. PyTorch provides `torch.save` for this purpose:

```
torch.save(obj, f, pickle_module=pickle, pickle_protocol=DEFAULT_PROTOCOL,
           _use_new_zipfile_serialization=True)
```

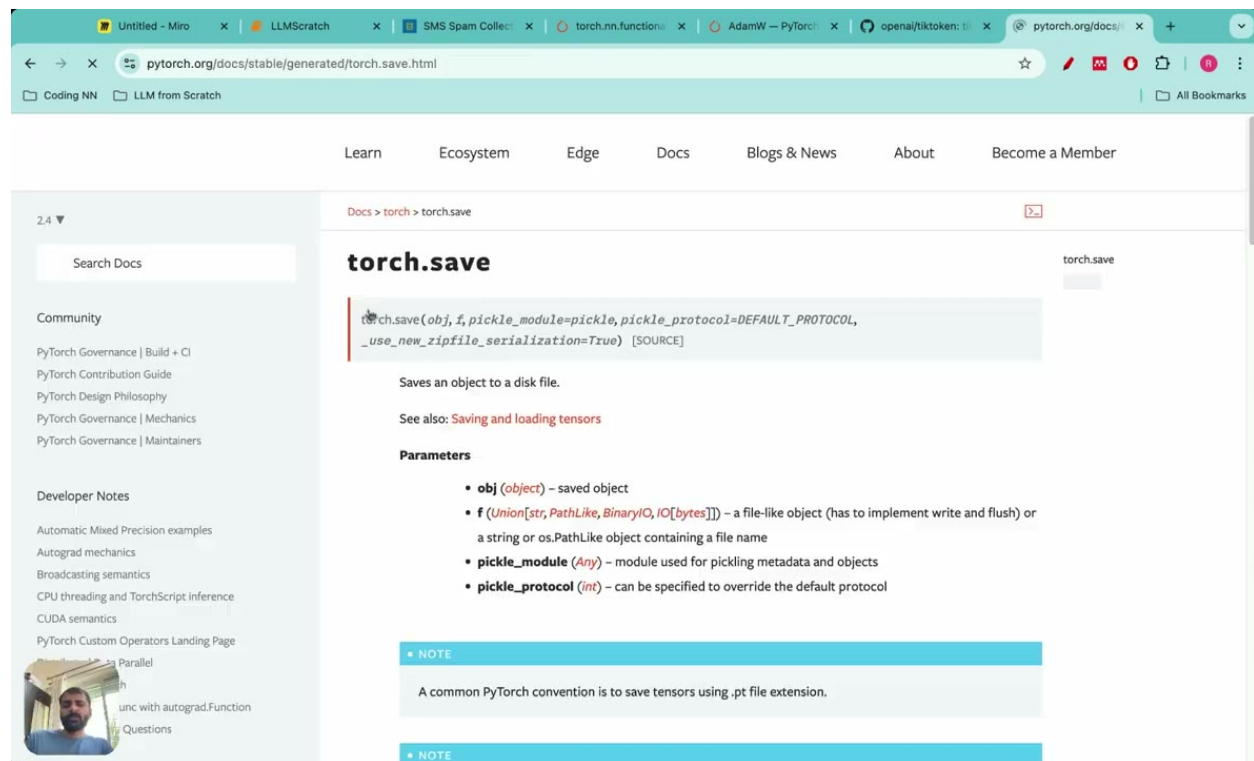


Figure 14.3: Timeline diagram showing one training epoch divided into batches, with loss printed every 50 batches and accuracy printed at the end of each epoch.

Fine-tuning changes model parameters through retraining on a specific dataset so the model performs well on that particular task; saving the fine-tuned weights captures the result of that process in a file that can be loaded later.

14.10 Evaluating on the Test Set and Running Inference

The final stage of the project involves testing the fine-tuned model on data it has never seen before. The held-out test set from Section 12.2.3 is the right tool for this evaluation: because it was never used during training or hyperparameter tuning, it provides an unbiased estimate of real-world performance.

14.10.1 12.10.1 Why a Held-Out Test Set Matters

Using the test set during training would leak information into the learning process and produce an optimistically biased accuracy estimate. Keeping it strictly separate ensures that the final reported accuracy reflects how the model will behave on genuinely unseen messages.

14.10.2 12.10.2 Inference on New Text

For inference on a single new message, the pipeline is:

1. Tokenize the input text using TikToken.

2. Pass the token IDs through the model and extract the last token's output: `outputs[:, -1, :]`.
3. Apply `argmax` to the two logits to obtain the predicted label.

This pipeline confirms the value of fine-tuning: without it, the pre-trained GPT model could not correctly answer spam classification prompts even when given explicit instructions.

14.11 Putting It All Together

Classification fine-tuning is one category of fine-tuning, focused on adapting models for classification tasks. The pattern established here—load a foundation model, swap the output head, freeze most layers, fine-tune a small subset, and evaluate rigorously—applies to any classification problem you might encounter: sentiment analysis, topic labelling, intent detection, and more.

14.11.1 12.11.1 Comparison with Instruction Fine-Tuning

Instruction fine-tuning requires the model to generate free-form text conditioned on a natural-language instruction, which demands more compute and a much larger labelled dataset. Classification fine-tuning constrains the output space to a fixed set of labels, making training faster, evaluation simpler, and deployment more predictable. The cost is flexibility: the model can only handle a narrow set of prompts.

For scenarios where you need both efficiency and flexibility, parameter-efficient methods like LoRA and QLoRA offer a middle ground by fine-tuning only a small number of adapter parameters rather than full layer weights.

14.12 Summary

In this chapter we built a complete classification fine-tuning pipeline for spam detection. Starting from the SMS Spam Collection dataset, we balanced it to address class imbalance and split it into train, validation, and test sets. We wrapped the data in PyTorch Dataset and DataLoader objects, using TikToken for tokenization and padding all sequences to a uniform length.

On the model side, we loaded pre-trained GPT-2 weights, replaced the vocabulary-sized output head with a two-class classification head, and selectively unfroze the final transformer block and final layer normalization for fine-tuning. We implemented accuracy and cross-entropy loss utilities, wrote a training loop with configurable evaluation frequency, and saved the resulting model. Finally, we evaluated on the held-out test set and ran inference on new messages using the last output token.

The next chapter extends these ideas to instruction fine-tuning, where the model learns to follow natural-language directives rather than predict a fixed class label.

Chapter 15

Instruction Fine-Tuning: Building a Personal Assistant

A pre-trained language model can complete sentences and generate coherent prose, but it cannot reliably *follow instructions*. This chapter closes that gap.

We will walk through the complete instruction fine-tuning pipeline from end to end. The journey has three stages: preparing the dataset, fine-tuning the model, and evaluating what we built.

15.1 Why Instruction Fine-Tuning?

Instruction fine-tuning is the process of training a pre-trained LLM with a specific dataset to teach it to correctly follow instructions. The mechanism is straightforward: collect a large set of instruction-response pairs and use them to update the model's weights. The result is a model that has been modified to understand instruction input-output pairs from a specific dataset. Intuitively, pre-training gives the model a rich internal representation of language; fine-tuning steers that representation toward the behavior you want.

Fine-tuning serves two main purposes: training the LLM to follow instructions and training on domain-specific data relevant to the deploying organization. To make this concrete, consider a healthcare virtual assistant. Fine-tuning it with domain-specific instructions ensures it understands medical terminology, follows healthcare guidelines, and personalizes responses based on patient history. More generally, building a personal assistant requires the LLM to follow instructions such as correcting grammar, removing filler words, making text concise, and adjusting tone.

One practical advantage: fine-tuning on a specific dataset after loading pre-trained weights requires significantly less computational time than training from scratch.

15.2 The Three-Stage Pipeline

Stage 1 — Dataset preparation. Download the instruction data, format it using the Alpaca prompt template, and create training, validation, and test data loaders. Significant time should be

spent on batching the dataset and creating data loaders, because correct implementation of this step simplifies all subsequent steps.

Stage 2 — Fine-tuning. Load a pre-trained GPT-2 checkpoint and run the supervised training loop on the instruction data.

Stage 3 — Evaluation. Extract model responses on the test set, perform qualitative inspection, and run automated scoring using a larger LLM.

Across all three stages, the pipeline spans nine steps in total: data preparation, fine-tuning, evaluation, response extraction, qualitative evaluation, and quantitative scoring.

15.3 The Instruction Dataset

15.3.1 What the Data Looks Like

The instruction dataset used for fine-tuning consists of 1,100 instruction input and output pairs. Each entry contains three keys: instruction, input, and output.

Example 1: - *Instruction:* Convert 45 km to meters - *Response:* 45 km is 45000 meters

Example 2: - *Instruction:* Provide a synonym for bright - *Response:* A synonym for bright is radiant

Example 3: - *Instruction:* Edit the following sentence to remove all passive voice. “The song was composed by the artist.” - *Response:* The artist composed the song.

Notice that Example 3 uses the input field to supply the sentence being edited, while Examples 1 and 2 do not. This distinction matters when we format the prompt.

For comparison, the Stanford Alpaca repository contains a much larger dataset of 52,000 instruction-output pairs.

15.3.2 Loading and Splitting the Data

The fine-tuning process begins by loading a training dataset consisting of instruction-response pairs. The test split is reserved for evaluating the model’s performance after fine-tuning.

15.4 The Alpaca Prompt Format

Raw instruction-response pairs cannot be fed directly to the model. We need a consistent text template that tells the model where the instruction ends and where the response should begin. The dominant convention is the **Alpaca prompt format**: a specific formatting convention for converting instruction-input-output pairs into prompts for fine-tuning large language models, maintained by Stanford in the Stanford Alpaca repository.

The full template reads:

Below is an instruction that describes a task paired with an input that provides further context. Write a response that appropriately completes the request.

Instruction: {instruction}

Input: {input}

Response: {output}

When the input field is empty, the Input: section is omitted entirely, keeping the prompt clean and avoiding confusion from an empty section.

For contrast, the Phi-3 prompt style takes a different approach: the instruction and input are fused together in a user field, and the output is placed in an assistant field. Both styles are valid; we use Alpaca throughout this chapter because it is the more widely documented convention.

15.4.1 Implementing `format_input`

The following function converts a dataset entry into an Alpaca-formatted prompt string:

Source:

```
def format_input(entry):
    instruction_text = (
        "Below is an instruction that describes a task paired with an input that"
        "provides further context. Write a response that appropriately completes the"
    )
    instruction = entry['instruction']
    input_text = entry.get('input', '')
    if input_text:
        prompt = f"{instruction_text}\n\nInstruction:\n{instruction}\n\nInput:\n{input_text}"
    else:
        prompt = f"{instruction_text}\n\nInstruction:\n{instruction}"
    return prompt
```

The function returns only the *input* side of the prompt—the instruction and any context—without the response. During training, the response is appended separately so the model can learn to predict it token by token.

15.5 Data Batching: From Text to Tensors

Converting raw entries into model-ready batches requires five steps:

1. Format each entry using the Alpaca prompt template.
2. Tokenize the formatted string into token IDs.
3. Pad sequences to equal length within each batch.
4. Create target token IDs by shifting the input sequence right by one position.
5. Replace padding tokens in the targets with -100 .

Let us work through each step.

15.5.1 Step 1: Format with the Prompt Template

Each entry in the dataset is passed through `format_input` to produce a fully formatted string.

15.5.2 Step 2: Tokenization

Tokenization is the process of converting data into a numerical representation by converting sentences into token IDs. The formatted prompt string is tokenized before being passed to the model.

15.5.3 Step 3: Padding to Equal Length

Because sequences in a batch can have different lengths, we must equalize them. Padding is the process of adjusting the length of token ID sequences so that all samples in a batch have the same length. The end-of-text token—token ID 50256 in GPT-2—is used as the padding symbol. Batching the dataset ultimately means converting multiple data samples into a batch where each sample is represented as a numerical array (row) with uniform dimensions.

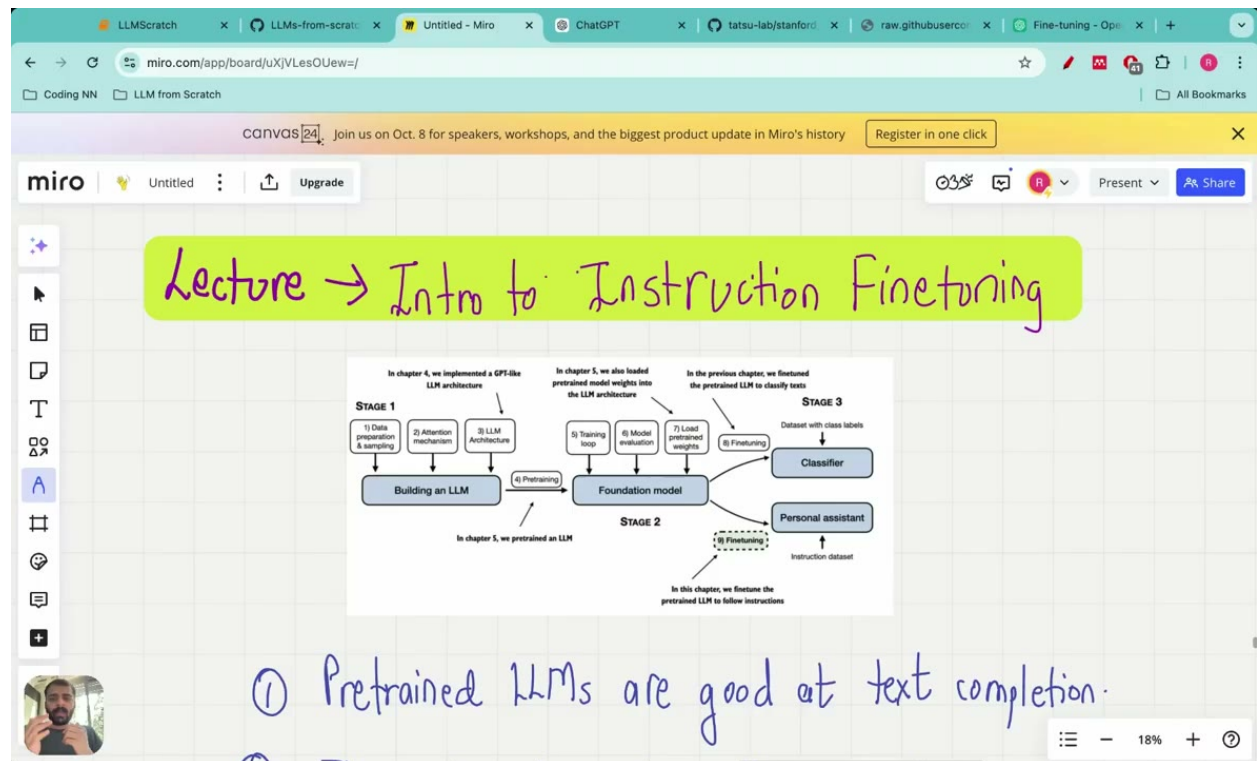


Figure 15.1: Three sequences of different lengths padded with token ID 50256 to form a uniform rectangular batch matrix for processing.

15.5.4 Step 4: Creating Target Sequences

During instruction fine-tuning, the LLM is trained using next-token prediction, where the entire formatted prompt serves as both input and target pairs. The target sequence is the input sequence shifted to the right by one position: target token IDs are created by shifting the input tokens right by one position and appending a padding token at the end to maintain equal length between input and target sequences.

Source:

```
targets = inputs[1:] # Shift inputs right by one by removing first element
```

An end-of-text token is appended to complete the target sequence.

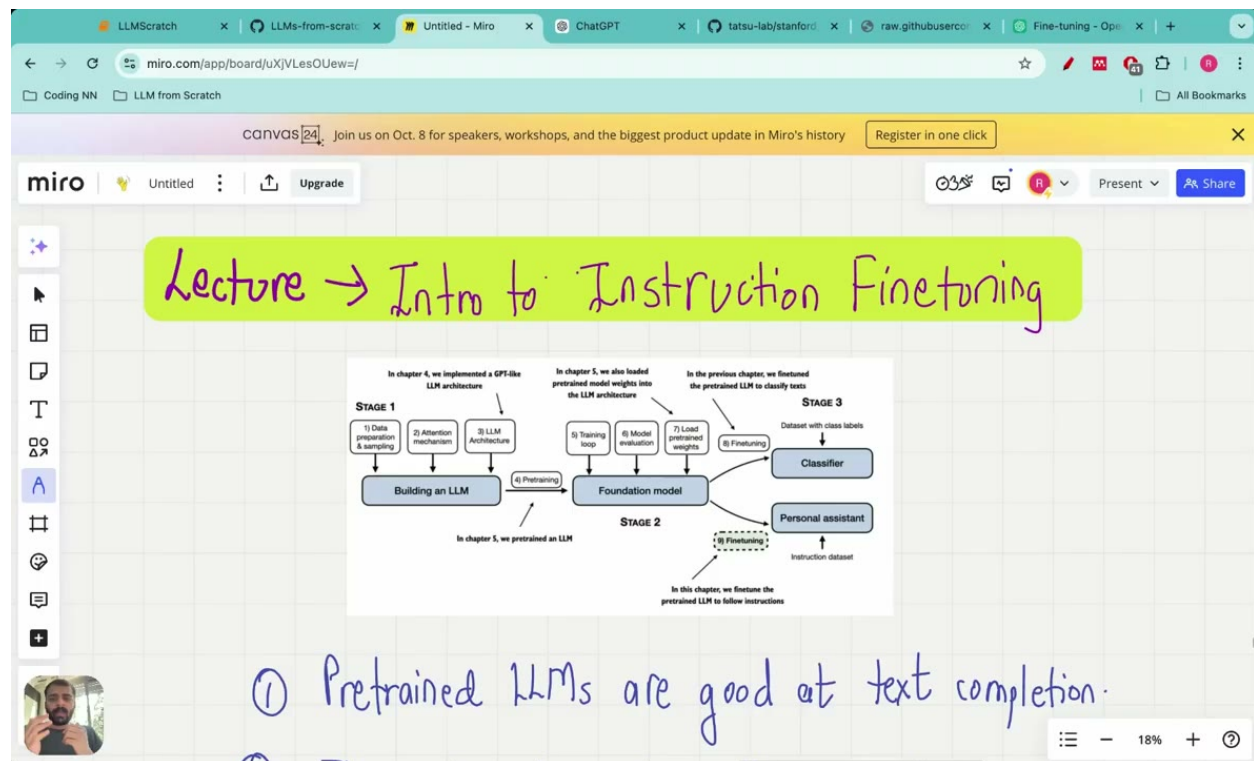


Figure 15.2: Input and target token sequences for instruction finetuning, showing the one-position rightward shift with end-of-text token appended.

15.5.5 Step 5: Replacing Padding Tokens with -100

Padding tokens in the target sequence must not contribute to the loss. PyTorch’s cross-entropy loss function has a built-in `ignore_index` parameter; when a target token ID equals `-100`, that position is excluded from the loss calculation entirely.

There is one exception: the *first* occurrence of the end-of-text token in the target is kept as-is, because it is a legitimate prediction target—the model should learn to emit end-of-text when it finishes a response. Additionally, the token IDs corresponding to the instruction and input portions of the target are also replaced with `-100`, so the loss is computed only over the response tokens.

15.5.6 The Custom Collate Function

A custom collate function takes a batch of sequences and requires a padding token ID (50256) and a target device (cpu) as inputs:

Source:

```
def custom_collate_function(batch, padding_token_id, ignore_index):
    # Get inputs and targets
    # Create mask for all padding token indices
    mask = (target_tensor == padding_token_id)
    # Ignore the first occurrence of padding_token_id
    mask[0] = False
    # Replace all remaining padding tokens with ignore_index (-100)
```

```
target_tensor[mask] = ignore_index
return input_tensor, target_tensor
```

In summary, the `collate` function converts tokens to token IDs, pads sequences to equal length within the batch, replaces padding tokens (50256) with `-100`, and returns input and target tensors ready for training.

15.6 Creating the Data Loaders

Data loaders are an efficient way to collect different batches sequentially and access them in an iterative manner during model training.

First, select the compute device:

```
# Source:
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

Then wrap the training data in a dataset object:

```
# Source:
train_dataset = InstructionDataset(training_data)
```

A `DataLoader` wraps the dataset and calls the custom `collate` function to assemble batches on each iteration. Stage one of instruction fine-tuning focuses entirely on preparing the dataset itself.

15.7 Loading the Pre-Trained Model

With the data pipeline in place, we turn to the model. A pre-trained GPT-2 model with 355 million weights serves as the foundation for instruction fine-tuning:

```
# Source:
from gpt_download3 import download_and_load_gpt2

BASE_CONFIG = {
    "vocab_size": 50257,
    "context_length": 1024,
    "drop_rate": 0.0,
    "qkv_bias": True
}

model_configs = {
    "gpt2-small(124M)": {"emb_dim": 768, "n_layers": 12, "n_heads": 12},
    "gpt2-medium(355M)": {"emb_dim": 1024, "n_layers": 24, "n_heads": 16},
    "gpt2-large(774M)": {"emb_dim": 1280, "n_layers": 36, "n_heads": 20},
    "gpt2-xl(1558M)": {"emb_dim": 1600, "n_layers": 48, "n_heads": 25},
}
```

```
CHOOSE_MODEL = "gpt2-medium_355M"
```

Fine-tuning is necessary to improve the model's ability to comprehend and appropriately respond to instruction-following requests.

15.8 The Fine-Tuning Training Loop

15.8.1 Loss Function

The loss function used for fine-tuning is cross-entropy loss—the same function used during pre-training. Conceptually, cross-entropy loss is the negative of the logarithm of the probability assigned to the correct token, and it is applied to calculate the loss on each batch of data.

15.8.2 Optimizer

The optimizer is **AdamW**, the Adam optimizer with weight decay, implemented as `torch.optim.AdamW`.

15.8.3 Parameter Updates

The backward pass calculates the gradient of the loss—the partial derivative of the loss with respect to all model parameters. Gradient descent then uses those gradients to update the parameters. The update rule is:

$$w_{\text{new}} = w_{\text{old}} - \alpha \cdot \frac{\partial L}{\partial w}$$

where w_{new} is the updated weight, w_{old} is the previous weight, α is the learning rate, and $\partial L / \partial w$ is the partial derivative of the loss with respect to w .

15.8.4 Monitoring Training

The dataset processing, batching, and data loaders are all in place before the training loop begins. During training, we track both training loss and validation loss. The `evaluation_iter` parameter specifies after how many iterations the loss on the evaluation dataset is calculated. Fine-tuning the LLM on instruction data modifies the weights and parameters of the GPT model to understand instruction input-output pairs from the dataset.

15.8.5 Saving and Loading the Fine-Tuned Model

After training completes, save the model's state dictionary so it can be reloaded later without re-running the training loop:

```
# Source:
torch.save(model.state_dict(), 'gpt2medium355_million_sft.pth')
```

To reload the weights:

Source:

```
model.load_state_dict(torch.load('gpt2medium355_million_sft.pth'))
```

With the weights saved, we can move on to evaluation without repeating the training run.

15.9 Evaluating the Fine-Tuned Model

Evaluation has three sub-steps: extracting model responses, performing qualitative inspection, and running automated quantitative scoring.

15.9.1 Step 1: Extracting Responses

After fine-tuning on the training portion of the instruction dataset, the model's performance is evaluated on the test set. The following procedure loops over the test data, generates output from the fine-tuned LLM, converts token IDs back to text, and isolates the response by removing the instruction input from the output:

Source:

```
# Iterates through all instruction-input pairs in the test data,  
# calls the generate function on each pair, and appends the model response  
# to the instruction_data_with_response file.
```

15.9.2 Step 2: Qualitative Evaluation

The simplest evaluation method is human preference comparison, where a human uses their own intuition and understanding to benchmark or compare LLMs. Reading through a sample of model responses gives an immediate sense of whether fine-tuning has produced coherent, instruction-following behavior. To improve performance, you can also experiment with different prompts or instruction formats to guide model responses more effectively.

15.9.3 Step 3: Automated LLM-Based Scoring

For scalable evaluation, we use a larger language model as a judge. This approach—Method 3 LLM evaluation—involves comparing the true expected output against the model response by asking a large language model to assign a score based on that comparison. Implementing this automated evaluation using a larger pre-trained LLM is the seventh step of the evaluation stage. Aggregating these scores across the test set yields a quantitative measure of fine-tuning quality.

15.10 Running Llama 3 Locally with Ollama

To use a larger LLM as a scoring judge without relying on a remote API, we can run one locally. Ollama is an efficient application for running large language models on your laptop, handling model downloading, quantization, and inference in a single lightweight binary.

Importantly, Ollama is a tool for LLM inference only and does not support training or fine-tuning.

15.10.1 Starting Ollama

```
# Source:
ollama run llama3
```

To run Ollama as a background server so it can be called from Python:

```
# Source:
ollama serve
```

15.11 Putting It All Together

1. **Dataset.** 1,100 instruction-response pairs, each with instruction, input, and output fields.
2. **Formatting.** The Alpaca prompt template wraps each entry into a structured string.
3. **Batching.** Tokenization, padding with token ID 50256, target shifting, and -100 masking are handled by the custom collate function.
4. **Data loaders.** PyTorch DataLoader objects iterate over training, validation, and test splits.
5. **Model.** GPT-2 medium (355M parameters) loaded from a pre-trained checkpoint.
6. **Training.** Cross-entropy loss over response tokens only, optimized with AdamW.
7. **Saving.** Model weights persisted to disk with `torch.save`.
8. **Response extraction.** The model generates responses for all test entries.
9. **Evaluation.** Qualitative human review and automated LLM scoring via Ollama.

The instruction fine-tuning pipeline spans nine steps in total, and we have now walked through every one of them.

15.12 Summary and What Comes Next

This chapter introduced supervised instruction fine-tuning and showed how to build the entire pipeline from raw data to a scored, evaluated personal assistant. The key ideas are:

- **Instruction fine-tuning** teaches a pre-trained model to follow instructions by training on instruction-response pairs.
- **The Alpaca prompt format** provides a consistent template that separates instruction, input, and response.
- **Data batching** requires careful handling of padding and the -100 ignore index to ensure the loss is computed only over response tokens.
- **Cross-entropy loss and AdamW** are the same tools used in pre-training, applied here to the fine-tuning objective.
- **Evaluation** combines human judgment and automated LLM scoring for a complete picture of model quality.

The supervised instruction fine-tuning approach covered here is the foundation for more advanced alignment techniques. Natural next steps include experimenting with different instruction formats to improve response quality, scaling to larger datasets such as the full Stanford Alpaca 52,000-entry corpus, and exploring reinforcement learning from human feedback (RLHF) to further align model behavior with human preferences.

Chapter 16

Summary, Reflections, and Next Steps

This final chapter steps back from the code and the mathematics to survey the full landscape of what we have built. By now you have traveled from raw text all the way to a fine-tuned language model that can classify emails and follow instructions. Before pointing toward what comes next, it is worth pausing to appreciate the arc of that journey—and to make sure the key ideas are firmly in place.

16.1 The Three-Stage Pipeline at a Glance

The development of a large language model follows a clear, sequential structure. Understanding that structure is as important as understanding any individual component, because it tells you *where* each technique lives and *why* it is needed.

Stage 1 — Foundation and Architecture. Everything begins with data. The preprocessing pipeline for an LLM is fundamentally different from the pipelines you would build for a regression or classification model. The goal is not to predict a label attached to an example; the goal is to predict the next token. That single objective shapes every design decision that follows.

The pipeline works in steps. First, raw text is broken into tokens. Those tokens are mapped to integer token IDs, which are then projected into a high-dimensional vector space to capture semantic meaning. The resulting representations are called *token embeddings*. Because the order of tokens matters for predicting what comes next, positional embeddings are added to the token embeddings, producing *input embeddings*—the final output of the preprocessing stage. For reference, GPT-2 uses an embedding dimension of 768 throughout this process.

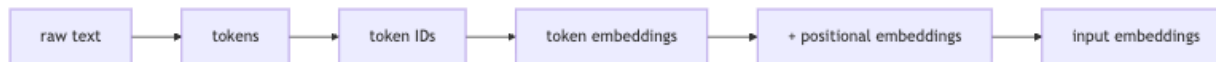


Figure 16.1: Data preprocessing pipeline transforming raw text through tokenization, embedding, and positional encoding stages to produce input embeddings for the model.

Stage 2 — Pre-training. With input embeddings in hand, the model is trained to predict the next token across an enormous corpus. This stage involves implementing the loss function, running backward-pass optimization over the hundreds of millions (or billions) of parameters a typical LLM

contains, and optionally bootstrapping from pre-trained weights—for example, loading OpenAI’s GPT-2 weights to accelerate convergence.

Stage 3 — Fine-tuning. A pre-trained model is a powerful general-purpose text predictor, but it does not yet perform well on specific tasks. Fine-tuning adapts the model to a target domain or behavior. The two main flavors explored in this series are classification fine-tuning (e.g., spam detection) and instruction fine-tuning (e.g., building a personal assistant).

16.2 Stage 1 in Depth: Architecture

16.2.1 The Attention Mechanism

The attention mechanism is the engine that gives large language models their power. To understand why it is necessary, consider what token embeddings alone provide: a representation of the semantic meaning of each individual token, with no information about how that token relates to the others around it. For next-token prediction, that relational information is essential—you need to know how every token connects to every other token in the context.

Attention solves this by assigning an *importance score* to every other token when processing a given token. The mechanism’s goal is to transform the input embedding vectors into richer *context vectors* that encode these relationships. A context vector is strictly more informative than the corresponding input embedding because it carries information about the token’s relationship to every other token in the sentence.

The computational workflow proceeds as follows:

1. Multiply the input embeddings by three trainable weight matrices to produce **queries**, **keys**, and **values**.
2. Multiply queries by the transpose of the keys to obtain raw **attention scores**.
3. Scale the scores by the square root of the key dimension.
4. Apply **dropout** after scaling.
5. Apply a **causal mask** so that each token can only attend to tokens that precede it, preventing information leakage from future tokens.
6. Pass the masked scores through a **softmax** to obtain **attention weights**.
7. Multiply the attention weights by the values to produce the **context vector matrix**.

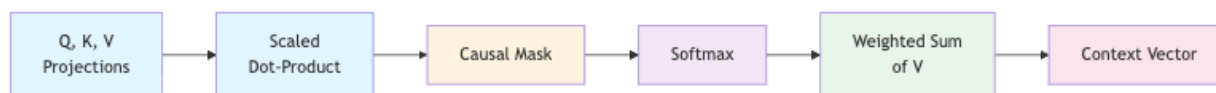


Figure 16.2: Single attention head computation pipeline: input projections through scaled dot-product attention with causal masking, softmax normalization, and weighted aggregation to produce the final context vector.

A single attention head produces one context vector matrix. Real LLMs run many such heads in parallel, because different heads can learn to capture different types of dependencies and long-range relationships within large paragraphs. The context vector matrices from all heads are then concatenated and projected to produce the final output context vector matrix.

Intuitively, this is the revolution: the model learns, entirely from data, which tokens should attend to which other tokens. That learned transformation from input embedding matrices to context vector matrices is what drives the power of modern LLMs.

16.2.2 The Transformer Block and Full Architecture

The full LLM architecture wraps the attention mechanism inside a larger structure. At the top level, input tokens are converted to embeddings, passed through a stack of *transformer blocks*, then through a final layer normalization and a linear projection that outputs *logits*—one score per vocabulary token—used to predict the next token.

Each transformer block contains:

- A **layer normalization** layer
- A **masked multi-head attention** module (where the Q/K/V projections live)
- A **dropout** layer
- **Shortcut (residual) connections**
- A second **layer normalization** layer
- A **feedforward neural network**
- A second **dropout** layer

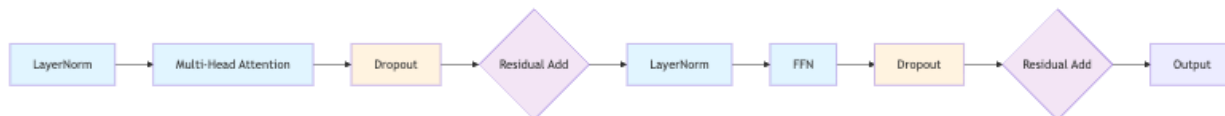


Figure 16.3: Transformer block architecture showing the sequential flow of LayerNorm, Multi-Head Attention, Dropout, and residual connections, followed by LayerNorm, FFN, Dropout, and another residual connection.

GPT-2 stacks 12 such blocks sequentially; larger models use more. Within each block there can be 12 or 24 individual attention heads. Trainable parameters are distributed across token embeddings, positional embeddings, layer normalization scale and shift parameters, the query/key/value weight matrices in every attention head, the feedforward network weights, and the final output projection.

16.3 Stage 2 in Depth: Pre-training

Pre-training is where the model acquires its general knowledge of language. The training objective is straightforward: minimize the cross-entropy loss between the model’s predicted next token and the actual next token in the training corpus.

At its simplest, the gradient update is vanilla gradient descent:

$$w_{i+1} = w_i - \alpha \frac{\partial L}{\partial w_i}$$

In practice, more sophisticated optimizers—Adam or Adam with weight decay—are used instead. With L2 regularization, the weight update takes the form $W_i^{(l)} = W_i - \lambda \frac{\partial L}{\partial W}$, and pre-trained weights from GPT-2 can serve as initialization rather than starting from random values.

The scale of real pre-training is sobering. Models like GPT-2, GPT-3, and GPT-4 are trained on datasets containing millions of news articles, blogs, and books, and the compute cost for training such models exceeds one million dollars. This is precisely why loading pre-trained weights—rather than training from random initialization—is such a practical accelerant.

16.4 Stage 3 in Depth: Fine-tuning

Fine-tuning takes a pre-trained model and specializes it. Two approaches were implemented in this series.

Classification fine-tuning trains the model to assign inputs to discrete categories—for example, deciding whether an email is spam or not spam. The architecture is modified so that the final layer produces a class prediction rather than a distribution over vocabulary tokens.

Instruction fine-tuning trains the model on a dataset of (instruction, input, output) triples so that it learns to follow natural-language directives. A concrete example: given the instruction “convert to passive voice” and the input “the chef cooks the meal every day,” the model should produce “the meal is cooked every day by the chef”.

Together, these two projects—an LLM classifier that distinguishes spam from non-spam, and a personal assistant that follows instructions—represent the two hands-on deliverables of the series.

[GAP: Specific discussion of LoRA and QLoRA (parameter-efficient fine-tuning) — the chapter summary mentions these techniques but no supporting claims were supplied in the source material.]

16.5 Evaluating What You Have Built

Training a model is only half the work; you also need to know whether it is any good. Three evaluation strategies were covered.

Benchmark evaluation uses standardized test suites. MMLU (Measuring Massive Multitask Language Understanding) is one prominent example, applying 57 distinct tests to probe an LLM’s knowledge across a wide range of domains.

Human evaluation has raters compare and score the outputs of different models directly. This is the gold standard but is expensive and slow.

LLM-based evaluation uses a powerful model to judge a weaker one. In practice, a larger LLM compares the model’s response against the ground-truth output and assigns a score out of 100. The series demonstrated this using Llama 3 8B Instruct—a fine-tuned model with 8 billion parameters—accessed locally through a tool called Ollama.

16.6 What Was Built from Scratch

It is worth being explicit about the scope of what was implemented. The series produced three concrete artifacts:

1. A **next-token prediction LLM** built entirely from scratch.
2. An **email classification LLM** produced by fine-tuning the above.
3. An **instruction-following LLM** produced by a second round of fine-tuning.

Every code block was explained in detail, and nothing was imported from a high-level library that would hide the underlying mechanics. The series was inspired by Sebastian Rushka’s book *Build a Large-Language Model*, and a single shared code file was developed and refined across all lectures.

The philosophy throughout was to understand the nuts and bolts of how large language models are built, rather than simply running pre-packaged applications. Students who complete the series know how to build an entire large language model completely from scratch.

16.7 Reflections on the Core Ideas

Looking back, a handful of insights stand out as load-bearing.

The preprocessing pipeline is the foundation. Everything the model learns is mediated by how text is tokenized, embedded, and positioned. Understanding *why* each step exists is prerequisite to understanding everything else.

Attention is the key innovation. The transformation from static token embeddings to dynamic context vectors, achieved through learned query/key/value projections, is what allows LLMs to model the rich dependencies in language. Without this mechanism, scaling up a model would not produce the qualitative improvements we observe.

Pre-training and fine-tuning are complementary, not competing. Pre-training gives the model broad linguistic competence at enormous cost; fine-tuning then steers that competence toward a specific task at a fraction of the cost. The two stages are designed to work together.

Understanding the mechanics at small scale—which is exactly what this series provides—is the prerequisite for working effectively with large-scale systems.

16.8 Next Steps and Further Directions

Completing this series is a beginning, not an end. Several natural directions open up from here.

Larger datasets and bigger models. The architectural patterns you have learned—transformer blocks, multi-head attention, residual connections—scale directly. GPT-2 uses 12 transformer blocks and up to 24 attention heads per block; larger models simply use more of both. Applying what you know to larger corpora and deeper networks is a straightforward, if expensive, extension.

Parameter-efficient fine-tuning. Full fine-tuning updates every parameter in the model. Techniques like LoRA (Low-Rank Adaptation) and QLoRA (Quantized LoRA) reduce the number of parameters that need to be updated during fine-tuning, making it feasible to adapt large models on modest hardware. [GAP: Specific claims about LoRA/QLoRA mechanics and hyperparameters were not supplied in the source material.]

Evaluation rigor. The three evaluation strategies covered—benchmark suites like MMLU, human evaluation, and LLM-based evaluation—each have strengths and weaknesses. As you build more capable models, investing in robust evaluation becomes increasingly important.

Research literacy. The series was designed to teach fundamental research methodology in machine learning. With the foundations in place, you are now equipped to read primary literature—original papers on attention, transformer architectures, pre-training objectives, and fine-tuning strategies—and understand what the authors are actually doing.

16.9 Closing Thoughts

There is something genuinely clarifying about building a system from scratch. When you have written every line of the tokenizer, implemented the attention mechanism yourself, watched the loss curve descend during pre-training, and seen your fine-tuned model correctly classify a spam email or rephrase a sentence into passive voice, the technology stops being a black box. It becomes a set of engineering choices, each with a reason.

The attention mechanism is the driving engine of LLMs, but the broader insight is that the entire system—from the embedding dimension of 768 to the cross-entropy loss to the Adam optimizer—is a collection of deliberate, understandable decisions. None of it is magic, and all of it can be studied, modified, and improved.

That is the real takeaway from this series: not just that you can build a large language model from scratch, but that doing so gives you the conceptual vocabulary to engage seriously with one of the most consequential technologies of our time.

Appendix A

GPT Model Configuration Reference

Before writing a single line of model code, it pays to have all the numbers in one place. This chapter is that place. It covers the full family of GPT-2 model sizes, the specific configuration dictionary used throughout this series, and the training hyperparameters that appear in the data-loading and optimization code. Treat it as a reference you can flip back to whenever a number looks unfamiliar.

A.0.1 The GPT_CONFIG_124M Dictionary

Every component of the model we build—the embedding layer, the transformer blocks, the attention heads, the output projection—is parameterized by a single Python dictionary. Here is the canonical version used throughout this series:

```
GPT_CONFIG_124M = {  
    "vocab_size": 50257,           # Vocabulary size  
    "context_length": 1024,       # Context length  
    "emb_dim": 768,               # Embedding dimension  
    "n_heads": 12,                # Number of attention heads  
    "n_layers": 12,               # Number of layers  
    "drop_rate": 0.1,             # Dropout rate  
    "qkv_bias": False             # Query-Key-Value bias  
}
```

This dictionary encodes the architecture of GPT-2 small, the 124-million-parameter variant. Every key corresponds to a design decision with downstream consequences for memory, compute, and model quality. The sections below unpack each field in turn.

A.0.2 Hyperparameter Definitions

A.0.2.1 vocab_size — Vocabulary Size

The vocabulary size determines how many distinct tokens the model can recognize and generate. GPT-2 uses a vocabulary of **50,257 tokens**. This number is not arbitrary: it is the output of the

Byte-Pair Encoding (BPE) tokenizer that OpenAI trained for GPT-2, and it must match exactly between the tokenizer and the model’s embedding table.

A.0.2.2 `context_length` — Maximum Sequence Length

The context length sets the maximum number of tokens the model can attend to at once. For GPT-2 small, the full context length is **1,024 tokens**. A longer context allows the model to “see” more of a document when predicting the next token, but attention matrices grow quadratically with sequence length. For this reason, the tutorials in this series reduce the context length to **256 tokens** during training, so that the code can execute on a laptop without running out of memory. The production value remains 1,024; 256 is purely a pedagogical convenience.

A.0.2.3 `emb_dim` — Embedding Dimension

The embedding dimension is the size of the vector space into which each token ID is converted. For GPT-2 small, every token is represented as a **768-dimensional vector**. This dimensionality flows through every transformer block: the residual stream, the attention projections, and the feed-forward layers all operate in this 768-dimensional space.

A.0.2.4 `n_heads` — Number of Attention Heads

`n_heads` is the number of attention heads present in each transformer block. GPT-2 small uses **12 attention heads**. Each head operates on a slice of the embedding dimension ($768 / 12 = 64$ dimensions per head), allowing the model to attend to different aspects of the input in parallel.

A.0.2.5 `n_layers` — Number of Transformer Blocks

`n_layers` is the number of transformer blocks stacked in the model. GPT-2 small has **12 transformer blocks**, each containing a multi-head self-attention sublayer followed by a feed-forward sublayer, with layer normalization and residual connections around each.

A.0.2.6 `drop_rate` — Dropout Rate

The dropout rate is a regularization parameter. GPT-2 small sets it to **0.1**, meaning 10% of activations are randomly zeroed out during training to help prevent overfitting. When experimenting or debugging, it is common to set dropout to zero so that the model’s behavior is fully deterministic and easier to inspect.

A.0.2.7 `qkv_bias` — Query-Key-Value Bias

This boolean controls whether the linear projections that produce the query, key, and value matrices include a bias term. The GPT-2 small configuration sets this to **False**—a deliberate design decision that reduces the parameter count slightly and matches the original GPT-2 architecture.

Note on the loaded-weights variant. When loading pretrained OpenAI GPT-2 weights, `qkv_bias` must be set to **True**, because the released checkpoints include those bias terms. The False default is appropriate only when training from scratch.

A.0.3 The Full GPT-2 Model Family

GPT-2 was released in four sizes. The table below summarizes their architectural hyperparameters.

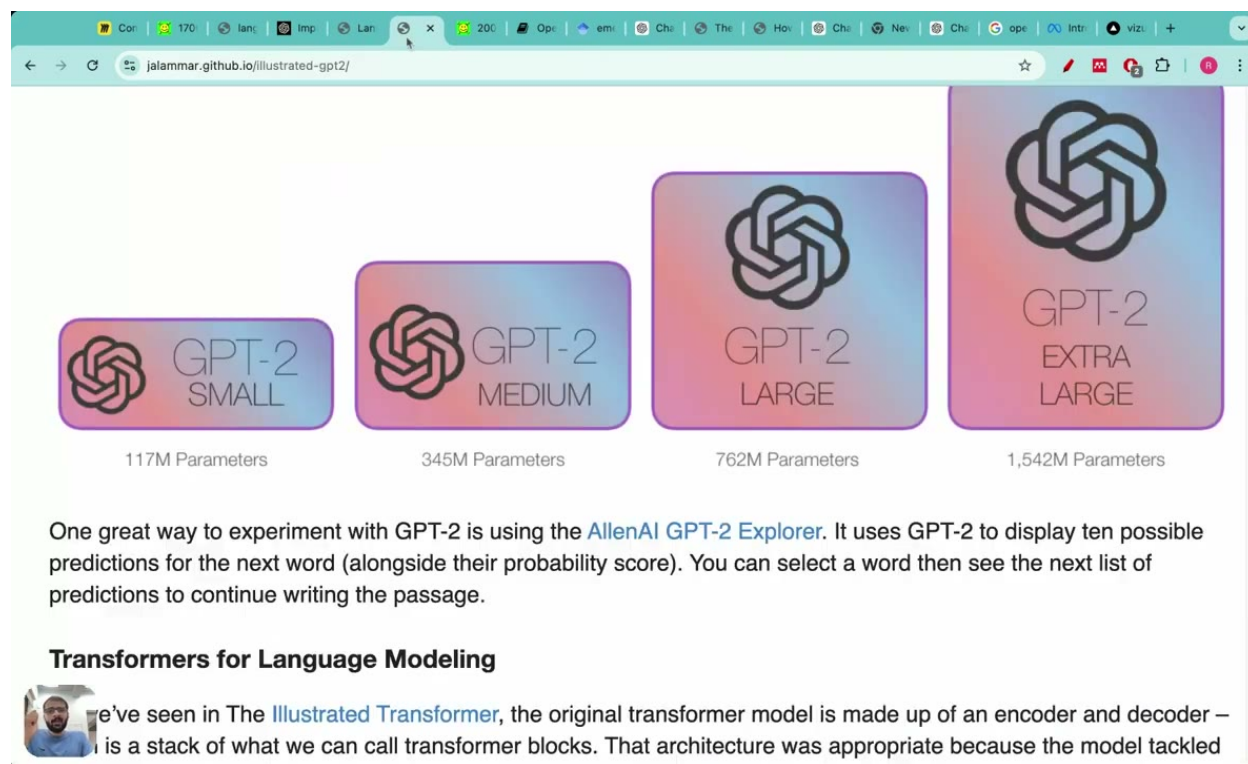


Figure A.1: Four GPT-2 model variants showing increasing parameter counts: Small (117M), Medium (345M), Large (762M), and Extra Large (1,542M).

A.0.3.1 GPT-2 Small (124M)

Hyperparameter	Value
Parameters	124M
Vocabulary size	50,257
Context length	1,024
Embedding dimension	768
Transformer layers	12
Attention heads	12
Dropout rate	0.1
QKV bias	False

This is the model built and trained from scratch throughout the series.

A.0.3.2 GPT-2 Medium (345M)

Hyperparameter	Value
Parameters	345M
Context length	1,024
Embedding dimension	1,024
Transformer layers	24
Attention heads	16

Relative to GPT-2 small, medium doubles both the embedding dimension and the number of layers, roughly tripling the parameter count.

A.0.3.3 GPT-2 Large (774M)

Hyperparameter	Value
Embedding dimension	1,280
Transformer layers	36
Attention heads	20

A.0.3.4 GPT-2 XL (1558M)

Hyperparameter	Value
Embedding dimension	1,600
Transformer layers	48
Attention heads	25

The pattern across the family is consistent: larger models increase embedding dimension, layer count, and head count together. The vocabulary size of 50,257 and the context length of 1,024 remain constant across all four variants.

A.0.4 Training Hyperparameters

Beyond the model architecture, several hyperparameters govern how data is fed to the model during training. These appear in the data-loader construction code used throughout the series.

A.0.4.1 Context Size During Training

As noted above, the tutorials use a context length of **256 tokens** rather than the full 1,024, purely to reduce computational requirements. This value is used consistently as `max_length` in both the training and validation data loaders.

A.0.4.2 Data Loader Configuration

The training data loader is constructed with the following settings:

Parameter	Value
batch_size	2
max_length	256
stride	256
drop_last	True
shuffle	True
num_workers	0

The validation data loader uses the same batch_size and sequence parameters but without shuffling:

Parameter	Value
batch_size	2
max_length	256
stride	256

Setting stride equal to max_length means there is no overlap between consecutive windows: each chunk of 256 tokens is drawn from a non-overlapping position in the text. Setting drop_last=True ensures that every batch has exactly the same size, avoiding shape surprises during training.

[GAP: Learning rate and weight decay values are referenced in the chapter summary but no claims in the supplied set specify their numeric values. These should be added when the corresponding claims are available.]

A.0.5 Quick-Reference Summary

The table below consolidates everything into a single lookup.

Key	GPT-2 Small (series default)	Notes
vocab_size	50,257	Fixed across all GPT-2 sizes
context_length	1,024 (full) / 256 (tutorial)	Reduced for laptop execution
emb_dim	768	Vector size per token
n_heads	12	Heads per transformer block
n_layers	12	Number of transformer blocks
drop_rate	0.1	Set to 0.0 when debugging
qkv_bias	False	True when loading OAI weights
Training batch_size	2	
Training max_length	256	
Training stride	256	Non-overlapping windows

With these numbers in hand, every subsequent chapter has a concrete anchor. When the code instantiates an embedding layer of size $50,257 \times 768$, loops over 12 transformer blocks, or clips sequences to 256 tokens, each choice can be traced directly back to this reference.

Appendix B

Environment Setup: PyTorch, TikToken, TensorFlow, and Ollama

Before writing a single line of model code, you need a working environment. This chapter walks through every tool you will need throughout the book: PyTorch for tensor operations and training, tiktoken for tokenization, TensorFlow (briefly) for loading pre-trained GPT-2 weights, tqdm for download progress tracking, and Ollama for running open-source LLMs locally during evaluation. Getting these pieces in place now will save you from interruptions later.

B.0.1 Device Selection: CPU, CUDA, and Apple Silicon

One of the first decisions your code must make at runtime is which hardware device to use for computation. PyTorch supports three backends relevant here: CPU, CUDA (for NVIDIA GPUs), and MPS (Metal Performance Shaders, for Macs with Apple Silicon chips).

The standard one-liner for selecting between CPU and CUDA is:

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

On a MacBook with an Apple Silicon chip, you can go one step further and set the device to `torch.device('mps')` for faster execution. Apple Silicon is approximately 2× faster than the Apple CPU for these workloads, making the extra line well worth it if you have the hardware.

The device variable is passed to every tensor and model call so that all computation happens in the same memory space. Mixing devices — for example, keeping your model on the GPU while your data stays on the CPU — is a common source of runtime errors, so establishing a consistent device-selection strategy early pays dividends throughout the project.

B.0.1.1 Moving Data to the Device During Loading

A subtle but important optimization concerns *when* you transfer tensors to the target device. The recommended approach is to perform the transfer inside a custom collate function, which runs as part of the data-loading pipeline rather than inside the training loop. Because the collate function executes in background worker processes, the transfer happens concurrently with the forward and backward passes, preventing it from blocking the GPU during model training.

B.0.2 TensorFlow and tqdm: Loading GPT-2 Weights

Later in the book you will load the original GPT-2 weights released by OpenAI. There is a small compatibility wrinkle: OpenAI originally saved those weights using TensorFlow, while the code in this series uses PyTorch. The weights therefore cannot be loaded directly with `torch.load`, so TensorFlow must be installed as an intermediary.

You will need the following minimum versions:

- **TensorFlow** 2.15
- **tqdm** 4.66

tqdm is installed specifically to track the download progress of the weight files, which can be several gigabytes. A progress bar is a small quality-of-life improvement that makes it clear the download has not stalled.

[GAP: explicit pip/conda install commands for PyTorch, tiktoken, TensorFlow, and tqdm are not present in the supplied claims.]

B.0.3 Ollama: Running LLMs Locally

Once you have trained or fine-tuned a model, you will want to compare its outputs against a strong reference. Ollama is a lightweight application designed to run large language models on a laptop, and it supports several well-known open-source models:

- **Llama 3** by Meta
- **Phi 3** by Microsoft
- **Mistral**
- **Gemma 2**

One important distinction: Ollama is an inference-only tool. It works with already pre-trained models and does not support training or fine-tuning. Think of it as a convenient local server for querying models, not a training framework.

B.0.3.1 Installing Ollama

Ollama is available for Mac OS, Linux, and Windows. The Mac OS installer weighs in at 177 MB, so the application itself downloads quickly. The model files are a different story — the Llama 3 8-billion-parameter model is 4.7 GB and takes roughly 15 to 20 minutes to download on a typical connection.

B.0.3.2 Running Ollama on Mac

On Mac, a single terminal command downloads (on first run) and starts the model:

```
ollama run llama3
```

When Ollama has successfully loaded Llama 3, right-hand-side arrows appear in the terminal, indicating the model is ready for interaction.

B.0.3.3 Running Ollama on Windows

On Windows the process requires two commands in sequence: first start the server, then launch the model.

```
ollama serve
```

Then, in the same or a second terminal:

```
ollama run llama3
```

B.0.3.4 Keeping Ollama Running

One operational detail to keep in mind: Ollama must remain running in the terminal for any downstream Python code to reach it. Closing the terminal window stops the server. A good habit is to open a dedicated terminal tab for Ollama and leave it untouched while you work in other tabs.

B.0.3.5 Talking to Ollama from Python

Typing prompts directly in the terminal is fine for quick checks, but systematic evaluation calls for a programmatic approach. Rather than invoking `ollama run` each time, the recommended method is to interact with the model through an API from Python. You can verify that the Ollama session is reachable by checking whether `ollama_running` equals `true` in your verification code.

[GAP: the specific Python API calls (e.g., the requests or ollama Python package usage) for interacting with Ollama programmatically are not present in the supplied claims.]

B.0.4 Summary

With this setup complete, you have everything you need to move forward:

Tool	Purpose	Key version / note
PyTorch	Tensors, training, inference	Device: CPU / CUDA / MPS
tiktoken	Tokenization	[GAP: version not specified in claims]
TensorFlow	Loading GPT-2 weights	2.15
tqdm	Download progress bars	4.66
Ollama	Local LLM inference for evaluation	Inference only

The next chapter begins building the model itself. Having your device selection sorted out now means you can pass device into model and data-loader code without revisiting this setup.

Appendix C

Datasets Used in This Series

Every machine learning project lives or dies by its data, and building a language model from scratch is no exception. This chapter catalogs every dataset that appears throughout this series—what each one contains, where it comes from, how large it is, and what role it plays. Having this reference in one place will save you from hunting through later chapters when you need to recall why a particular dataset was chosen or how to obtain it.

We cover four distinct categories: a small literary text used for pre-training demonstrations, a spam-classification corpus used for classification fine-tuning, a compact instruction-following dataset used for instruction fine-tuning, and the massive corpora used to train GPT-3 in the real world. The first three are datasets you will actually work with in code; the last provides context for understanding the scale gap between educational examples and production systems.

C.0.1 The Verdict: Pre-Training Dataset

The primary dataset for pre-training demonstrations throughout this series is *The Verdict*, a short story by Edith Wharton. Published in 1908 and now in the public domain, it is freely available to download and is included directly in the companion repository for this series.

The choice of such a small text is deliberate: a short work is used specifically to ensure fast execution on laptops, so you can run every pre-training experiment on consumer hardware in a reasonable amount of time.

In concrete terms, the text file contains **20,479 total characters**. When tokenized using byte pair encoding (specifically the tiktoken encoder), the story produces **5,145 tokens**. That is an extremely short corpus by the standards of language model training—a production model would be trained on hundreds of billions of tokens—but for learning how the data pipeline, model architecture, and training loop all fit together, it is entirely suitable.

[Diagram: Comparison of model sizes: The Verdict (5,145 tokens) versus GPT-3 (300 billion tokens), displayed on a logarithmic scale to visualize the vast difference in scale.]

C.0.2 SMS Spam Collection: Classification Fine-Tuning Dataset

When the series moves from pre-training to fine-tuning for classification, a different dataset takes center stage: the **SMS Spam Collection**. This corpus demonstrates how a pre-trained language model can be adapted to perform binary text classification—distinguishing spam messages from legitimate ones (commonly called “ham”).

The dataset contains **5,572 messages** in total, of which 425 spam messages were manually extracted from the Grumbletext website. It is distributed as a tab-separated values file named `SMSSpamCollection.tsv`, which can be loaded directly into a pandas `DataFrame` for manipulation.

One practical challenge with spam datasets is class imbalance: legitimate messages typically outnumber spam by a wide margin. To address this, the dataset is balanced before training. After balancing, each class contains **747 instances**, yielding 1,494 examples in total for the classification experiments.

C.0.3 Instruction Fine-Tuning Dataset

The third dataset appears in the instruction fine-tuning portion of the series, where the goal is to teach a model to follow natural-language instructions rather than simply continue text. The dataset used for these demonstrations consists of **1,100 instruction–input–output pairs**.

Each example has three fields: an instruction describing the task, an optional input providing context, and an output giving the expected response. This format is the backbone of instruction fine-tuning and mirrors the structure used by much larger, publicly available datasets.

For comparison, the **Stanford Alpaca** repository contains **52,000 instruction–output pairs**—roughly **50 times larger** than the 1,100-pair dataset used here. The smaller dataset is chosen for the same reason *The Verdict* is used for pre-training: it keeps training times manageable on a laptop while still illustrating every step of the fine-tuning pipeline.

[Diagram: Comparison of dataset sizes: this series uses 1,100 instruction-following pairs versus Stanford Alpaca’s 52,000 pairs.]

C.0.4 GPT-3 Pre-Training Corpora: Real-World Scale

To ground your intuition about what production pre-training actually looks like, it is worth examining the data that went into GPT-3. In total, GPT-3 was trained on **300 billion tokens**, assembled from several sources and weighted differently during training.

C.0.4.1 Common Crawl

The dominant source is **Common Crawl**, a free and open repository of web crawl data that has maintained over **250 million pages** spanning **17 years** since 2007. GPT-3 draws **410 billion tokens** from Common Crawl, constituting **60%** of the entire training dataset—making it the backbone of the training run.

C.0.4.2 WebText2

The second major source is **WebText2**, an enhanced version of the original WebText corpus covering Reddit submissions from 2005 to 2020. GPT-3 uses **19 billion words** from WebText2, accounting for **22%** of the training data.

C.0.4.3 Books and Wikipedia

The remaining **18–19%** of GPT-3’s training data comes from books and Wikipedia. Within the books component, the Books1 corpus alone contributes **12 billion tokens** at an **8% weight**.

The table below summarizes the full composition of GPT-3’s training data:

Source	Tokens	Weight
Common Crawl	410 billion	60%
WebText2	19 billion words	22%
Books1	12 billion	8%
Books + Wikipedia (remainder)	—	~10%
Total	300 billion tokens	100%

It is worth noting that the weighting scheme matters as much as raw token counts. A source can be sampled more or less frequently than its size alone would suggest, allowing the training process to emphasize higher-quality text—such as curated books or filtered web pages—relative to noisier sources.

C.0.5 Downloading and Licensing

The Verdict is in the public domain and is included directly in the series repository. The SMS Spam Collection is a well-known benchmark dataset available for research use, and the 1,100-pair instruction dataset is provided alongside the series materials. Common Crawl is a free and open repository, though working with it at full scale requires storage and compute resources well beyond a typical laptop setup.

For the purposes of this series, you will only need *The Verdict* and the SMS Spam Collection. Everything else is either provided in the repository or serves as reference material describing production-scale systems rather than something you will run locally.

C.0.6 Summary

Dataset	Size	Purpose
<i>The Verdict</i> (Edith Wharton, 1908)	20,479 chars / 5,145 tokens	Pre-training demonstrations
SMS Spam Collection	5,572 messages (balanced: 747 per class)	Classification fine-tuning
Instruction dataset	1,100 pairs	Instruction fine-tuning

Dataset	Size	Purpose
Stanford Alpaca	52,000 pairs	Reference / comparison
GPT-3 corpora	300 billion tokens total	Production-scale context

With these datasets cataloged—and a clear sense of why each was chosen—you are ready to move into the chapters that build the data-loading and tokenization pipelines that feed them into a model.